book 13

infrastructure: docker, databases, and testing

an engineering handbook

Raeliosol

version 2.2, compiled 10 2026

Contents

1. the toolchain: docker on windows 5
   1.1. what runs where 5
   1.2. the compiler-free lane, and its one exception 5
   1.3. lua, two runtimes, never crossed 6
   1.4. the road through the book 6
2. images and layers 8
3. containers: processes, signals, pid 1 10
   3.1. pid 1 and the signals that never came 10
   3.2. shutdown in reverse startup order 10
   3.3. what pid 1 owes, and what these processes owe instead 11
4. compose and one click local systems 12
   4.1. healthchecks, the difference between started and working 12
   4.2. depends_on with a condition, and why up --wait means it 13
   4.3. volumes and teardown discipline 14
5. container networking 16
   5.1. the bridge and the embedded dns server 16
   5.2. one published port, on purpose 16
   5.3. the subject space is the other network 17
6. sqlite: schemas and normalization 18
   6.1. the second migration, additive by design 19
7. sqlite: transactions and isolation 21
   7.1. a read transaction is a snapshot 21
   7.2. the lock ladder, shared to reserved to exclusive 22
   7.3. busy_timeout, the difference between error and wait 24
8. sqlite: wal, locking, crash semantics 27
   8.1. the mode is a property of the file 27
   8.2. checkpointing, the price of the sidecar 28
   8.3. backup without a backup api 30
   8.4. crash semantics, by experiment 31
9. sqlite: the 3.53 feature set 35
   9.1. json in sql 35
   9.2. alter column constraints, the 3.53.1 addition 36
   9.3. reindex 38
   9.4. money as integer cents 38
   9.5. strict tables 39
10. sqlite in production: embedded kvs and game saves 41
   10.1. three stores, one engine, three shapes 41
   10.2. the embedded key value shape 41
   10.3. where it ends 42
11. duckdb: the embedded analytical engine 43
   11.1. olap is a different question 43
   11.2. embedded from go, pinned 43
   11.3. parquet in, parquet out 44
   11.4. csv, sniffed then pinned 47
   11.5. scans that never load the file 49
   11.6. meeting the data: describe and summarize 51
   11.7. the appender, the fast way in 52
   11.8. mvcc, readers that never wait 54

12. duckdb and sqlite together: choosing the engine 57
12.1. attach, two engines in one process 57
12.2. writing back through the attach 58
12.3. affinity, loose typing meets strict columns 60
12.4. one statement to parquet 62
12.5. two engines, one answer 63
12.6. the cli lane, same binary 66
12.7. choosing the engine 68
12.8. the capstone, four stores named 68
12.9. the v2.0 horizon 68
13. mongodb and aggregation pipelines 70
13.1. the seam first 70
13.2. idempotent writes 70
13.3. the pipeline vocabulary 71
13.4. proven against the real engine 73
14. nats: pub sub, request reply, queue groups 75
14.1. subjects are a hierarchy 75
14.2. request reply is one publish with a return address 77
14.3. queue groups: load balancing with no broker config 78
14.4. the broker in process 80
15. jetstream: streams, consumers, kv, kafka compared 82
15.1. durable consumers are offsets with a name 82
15.2. kv: a bucket with a ttl 85
15.3. kafka, compared honestly 86
16. testing without docker 88
16.1. the three substitutions 88
16.2. the gate itself is a build tag 90
16.3. what each gate proves 90
16.4. where expected values come from 91
17. mocks, seeding, contract tests 93
17.1. the ladder, best first 93
17.2. the fake earns its keep by being checked 93
17.3. seeding to reach the rare paths 94
17.4. contract tests, names both sides answer to 95
18. migrations, test driven 97
18.1. discovery, order, and the names that fail loudly 97
18.2. apply, record, and the atomicity that matters 100
18.3. why not a library 102
19. version control at scale 104
19.1. git's object model, in brief 104
19.2. the failure, measured 104
19.3. the changeset, a wal entry 104
19.4. refs as compare and swap 107
19.5. derived data, blame and path history 110
19.6. three tiers, one rpc shape 112
19.7. the migration, measured 113
19.8. piper, sapling, and what oss can adopt 113
20. capstone part 1: services and topology 115
20.1. the shape 115
20.2. the wire, one page 115
20.3. the chat interior 116

20.4. the analytics interior, the columnar twin 118
20.5. boot, shared by all six 123
21. capstone part 2: chat and notifications, one click 125
21.1. htmx, vendored and pinned 125
21.2. the one command 127
21.3. topic matrix 130
21.4. pinned sources 133

1. the toolchain: docker on windows

This book is the third engineering handbook in the row. It takes the three things a backend or a game server actually sits on, containers, databases, and tests, and builds one system out of them: a peer to peer chat and notification microservice suite that comes up with a single `docker compose up -d`. Every chapter before the capstone is one layer of that system, and every layer obeys one rule about verification: it either runs without the docker daemon, or it is explicitly gated behind the daemon and never silently skipped.

1.1. what runs where

Two gates, and the split is a design rule, not a convenience.

- `make verify` runs everywhere and touches no docker daemon. Persistence is SQLite or DuckDB on a temp directory, messaging is an embedded in-process NATS server, and the mongo consumer runs against an in-memory fake. A test that needs the real daemon does not belong in this gate.
- `make verify-infra-docker` preflights `docker info` and refuses to skip. With the daemon present it brings the capstone up with `docker compose up -d --wait`, runs the docker-tagged suite against the running stack, and tears down with `down -v` even on failure. With the daemon absent it exits nonzero with a plain message saying what to start.

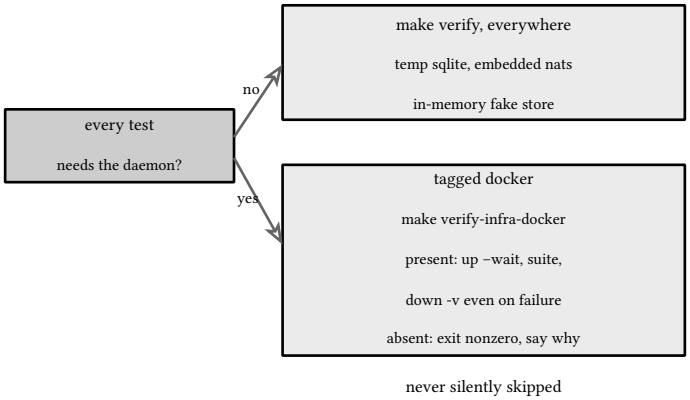


figure 1: the two-gate split as a routing decision, and neither gate silently skips

1.2. the compiler-free lane, and its one exception

Every sqlite driver in this repo is chosen so that no consumer needs a C compiler at build or test time.

- Go uses `modernc.org/sqlite`, a pure Go translation of the SQLite amalgamation (the single-file C source sqlite publishes). It is the first third-party dependency this repo's go modules ever took, spent deliberately: the sqlite lanes keep `CGO_ENABLED=0` true and cross compilation stays boring.
- C# uses `Microsoft.Data.Sqlite` with the `SQLitePCLRaw.bundle_e_sqlite3` package, which ships a prebuilt native engine next to the managed driver.
- JavaScript uses `node:sqlite`, built into node 26, so the capstone adds no dependency at all.
- Lua uses LuaJIT's FFI over a dll this repo builds itself with mingw.

The analytical engine is the one exception, and it is fenced. Duckdb's go driver is cgo by nature, so the duckdb lanes build against the pinned official dll from `make duckdb-tools`, confined to the `duckdb_use_lib` tagged sample packages and one debian compose stage in the capstone, with zig standing in as the cgo compiler on the host because the driver's prebuilt static engine libraries are MSVC objects. On linux, which is what that compose stage builds, the driver's default static bindings compile with no tag and no dll. A plain `go test ./...` compiles none of it, so every lane that is not deliberately duckdb stays compiler-free.

The single place that owns a compiler is the toolchain. `make sqlite-tools` builds SQLite 3.53.4 and LuaJIT from pinned source with mingw, the same compiler that builds Lua 5.5 for book 9, and `make duckdb-tools` fetches the duckdb 1.5.5 cli and libduckdb with sha256 pins against install.duckdb.org, warming the sqlite extension into the home cache for the attach chapter. The outputs live in `tools/sqlite/build/`, `tools/luajit/build/`, and `tools/duckdb/build/`, all gitignored, and the build scripts assert their versions before they finish, so a wrong download fails the build instead of producing a quietly different engine.

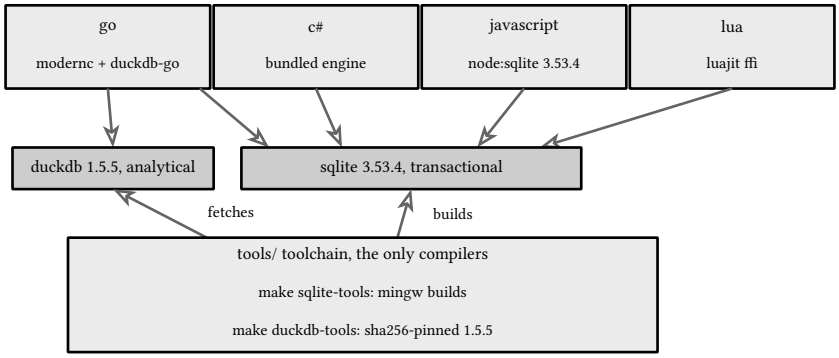


figure 2: four runtimes, two pinned engines, the compilers confined to the toolchain box

NOTE: why 3.53.4 and 1.5.5, pinned once each

Pinning one version per engine across every runtime keeps the database chapters honest. When book 13, ch 9 (infrastructure: docker, databases, and testing, sqlite: the 3.53 feature set) says a JSON function or an ALTER TABLE capability exists, it exists in the engine the Go, C#, and Lua suites assert on. Node ships its own sqlite build rather than the pinned amalgamation: it read 3.53.1 on node 26.3.0, the one measured delta, and reads 3.53.4 on node 26.10.0, caught up to the pin, and the javascript capstone asserts that exact string either way, because driver-bundled engines drift. The duckdb chapters do the same for 1.5.5: the go module tag encodes it, and the probes assert `select version()` returns it exactly. Driver-bundled engines drift. The pinned amalgamation and the pinned dll do not.

1.3. lua, two runtimes, never crossed

Book 9 teaches Lua 5.5 from the manual and its runner is a 5.5 interpreter built from lua.org source. This book adds LuaJIT for exactly one job, the sqlite wrapper in the persistence chapters, and LuaJIT speaks Lua 5.1. The split is a rule the make targets enforce: the 5.5 runner never executes FFI code, the LuaJIT runner never executes the 5.5 samples, and the wrapper itself stays 5.1 clean. No goto, no integer division, nothing from 5.2 and later.

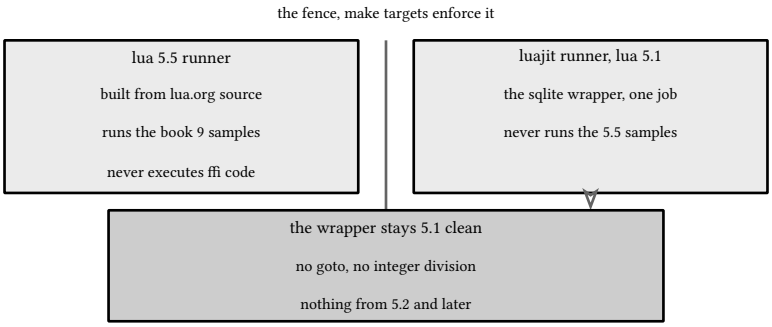


figure 3: two fenced lua lanes, the wrapper confined to the 5.1 subset

1.4. the road through the book

Chapters 2 through 5 build the container half: what an image is, what a container actually is on the host, how compose wires a multi-service system, and how the bridge network and embedded DNS let the services find

each other. Chapters 6 through 12 are the storage half: sqlite from schema design and normalization through transactions, WAL, and the 3.53 feature set into production, then duckdb as the embedded analytical engine and the two engines run together under one choosing rule. Chapters 13 through 15 add the other two stores: MongoDB for aggregation pipelines, NATS for messaging, and JetStream for the durable log and key value layers, with Kafka treated as the comparison it needs to be rather than a second implementation. Chapters 16 through 18 are the testing half: what runs docker-free, when a mock is the right call and when it is a lie, and a migration runner built test first.

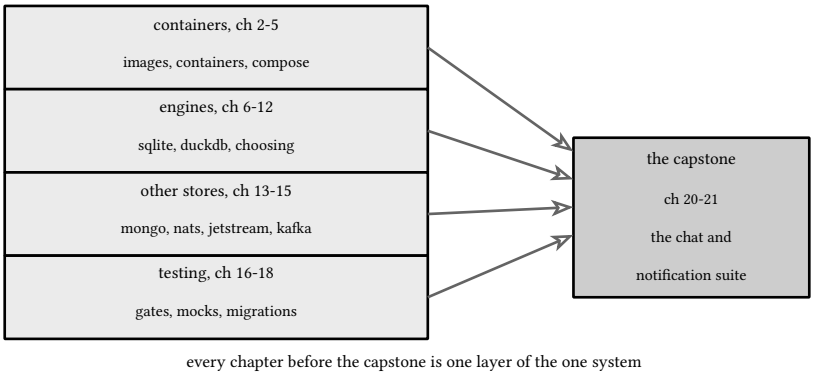


figure 4: four halves stack up and converge on the capstone

Then the capstone, book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 1: services and topology) and book 13, ch 21 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click), composes all of it into the chat and notification suite.

2. images and layers

A docker image is a tarball of filesystem layers plus a manifest that says which layer goes on top of which. Each layer is the directory diff one build step produced, so two images that share a base and a dependency step share those layers on disk and in the registry, and a rebuild that changes only source code downloads nothing it already has. The whole art of a fast build loop is ordering the file copies so the layers that change rarely sit under the layers that change often.

The capstone builds five services out of one dockerfile by handing the build stage an argument that picks the command to compile:

```
# one image for all five services: the build argument picks the binary
# the thin runtime copy gets. multi stage keeps the compiler out of the
# shipped layer; CGO_ENABLED=0 keeps the binary static so distroless-
# style minimal bases stay possible.
FROM golang:1.27-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
ARG SERVICE=web
RUN CGO_ENABLED=0 go build -o /out/service ./cmd/${SERVICE}
```

listing 1: the build stage: dependencies first, source second, one binary out (source: infrastructure/capstone/Dockerfile, lines 1-11)

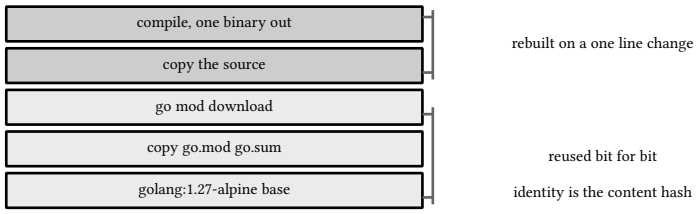


figure 5: the build stage as a layer stack, and what a one line source edit rebuilds

Three choices carry the weight. `COPY go.mod go.sum` before any source means the `go mod download` layer is built once and reused across every code edit, which is the difference between a two second rebuild and a two minute one. `CGO_ENABLED=0` keeps chapter 1's promise, the binary is static, so the runtime image needs no toolchain and no glibc version negotiation. And the `SERVICE` argument means one dockerfile, five images, no duplicated instructions drifting apart.

```
FROM alpine:3.22
# tini-free pid 1: the binaries install their own signal handlers, so
# exec form entrypoint hands SIGTERM straight to the go process
COPY --from=build /out/service /service
EXPOSE 8080
ENTRYPOINT ["/service"]
```

listing 2: the runtime stage: alpine, one binary, exec form entrypoint (source: infrastructure/capstone/Dockerfile, lines 13-18)

The runtime stage copies exactly one artifact into an alpine base. Alpine and not something thinner: the healthchecks in book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) shell out to busybox `wget`, which alpine ships and distroless images do not. Choosing a base by size alone quietly costs you the process you need for probing. The entrypoint is exec form, square brackets, which matters more than it looks and is the subject of book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1).

NOTE: layers are content addressed

A layer's identity is the hash of its contents. Two services built from the same stage share the compiled dependencies layer bit for bit, and `docker compose up --build` after a one line go change rebuilds only the source copy and the compile. Watching the layer ids scroll past on the second build is the cheapest way to verify the copy order is doing its job.

The image tags the compose file pins, `golang:1.27-alpine`, `alpine:3.22`, `nats:2.14.6-alpine`, `mongo:8.0`, and `debian:13-slim` for the analytics image, were verified with `docker manifest inspect` before landing in `compose.yml`, the same pin-then-assert discipline the sqlite version gets.

sources: docs.docker.com/reference/dockerfile for build stages, entrypoint forms, and build arguments, accessed 2026-09-10. Verified by `make verify-infra-docker`, which builds all five alpine images plus the one debian analytics image and runs the tagged suite against them, run green 2026-09-20.

3. containers: processes, signals, pid 1

Strip the branding and a container is a process with walls: linux namespaces give it its own view of the network stack, the process tree, and the filesystem, and cgroups give it a resource ceiling. On Windows the docker daemon runs these processes inside a small linux vm, which is why `docker info` from git-bash talks to a named pipe and why file mounts cross a vm boundary. The walls change what the process can see. Nothing about it stops being a process, and that single fact drives everything in this chapter.

3.1. pid 1 and the signals that never came

Inside the container's pid namespace, the entrypoint process is pid 1. Pid 1 is special in unix: the kernel will not deliver default-disposition signals to it. If the shell is pid 1, because the dockerfile used shell form `ENTRYPOINT /service`, then `docker stop` sends SIGTERM, the shell catches it, does nothing, and the container dies only when docker escalates to SIGKILL after the grace period. The process never heard the polite knock.

The fix is two lines working together. Exec form in the dockerfile:

```
ENTRYPOINT ["/service"]
```

listing 3: exec form: the binary is pid 1, no shell in between (source: infrastructure/capstone/Dockerfile, lines 18-18)

and a signal handler in the binary:

```go
func Signal() <-chan os.Signal {
  ch := make(chan os.Signal, 1)
  signal.Notify(ch, os.Interrupt, syscall.SIGTERM)
  return ch
}
```

listing 4: the two signals docker sends a stopping container (source: infrastructure/capstone/internal/boot/boot.go, lines 64-68)

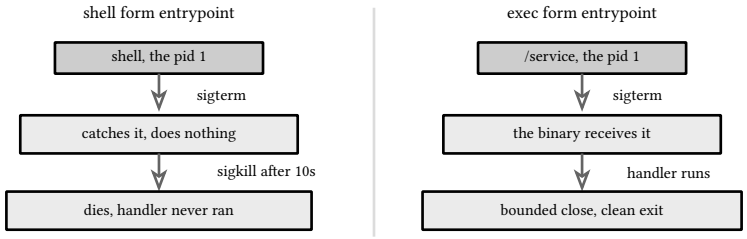


figure 6: shell form versus exec form: one kernel rule, two opposite outcomes

3.2. shutdown in reverse startup order

Every service main in the capstone follows one shape: bring things up in dependency order, park on the signal channel, then close in reverse. The chat service, which owns both a database file and a message connection, shows the full ladder:

```
func main() {
  db, err := store.Open(boot.Env("SQLITE_PATH", "/data/chat.db"))
  if err ≠ nil {
    log.Fatalf("chat: open store: %v", err)
  }
  nc, err := boot.Connect(boot.Env("NATS_URL", config.NATSURL))
  if err ≠ nil {
    log.Fatalf("chat: connect nats: %v", err)
  }
  svc, err := chat.New(db, nc)
  if err ≠ nil {
    log.Fatalf("chat: start: %v", err)
  }
  health := boot.ServeHealth(config.ChatHealthAddr)
  log.Print("chat: up")

  ←boot.Signal()

  boot.StopHealth(health)
  svc.Close()
  nc.Drain()
  db.Close()
  log.Print("chat: down")
}
```

listing 5: up, park, close in reverse: health server, responders, connection, file (source: infrastructure/capstone/cmd/chat/main.go, lines 14-37)

The order is the argument. The health endpoint goes first so compose stops routing traffic to a container that is going away. The NATS responders drop next, then `Drain` waits for in-flight requests to finish before closing the connection, and the database closes last, after nothing is left that could write to it. Reversing any pair creates a window where a live request meets a closed resource.

sigterm → health off → responders off → drain → db close

figure 7: stop, in the order that leaves no stragglers

3.3. what pid 1 owes, and what these processes owe instead

Classic pid 1 duty also includes reaping orphaned children, which is why real init systems like tini exist. The capstone services never fork child processes, so there is nothing to reap, and shipping an init shim would be cargo cult. The honest checklist is: exec form so the binary is pid 1, a handler for interrupt and terminate, and a bounded close sequence. One subprocess does exist in this book, the crash simulation in book 13, ch 8 (infrastructure: docker, databases, and testing, sqlite: wal, locking, crash semantics), and it is a test child, not a container entrypoint.

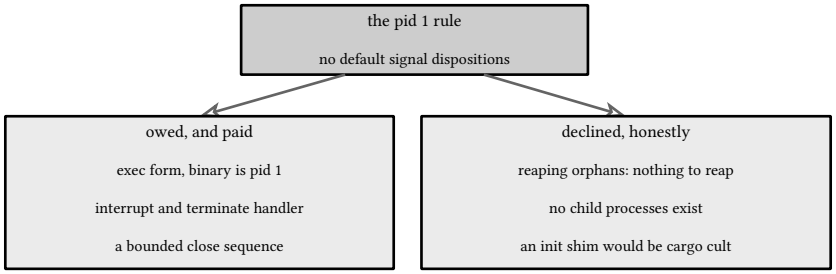


figure 8: what pid 1 owes here, and what these processes honestly decline

sources: docs.docker.com/reference/dockerfile for the shell versus exec entrypoint behavior, accessed 2026-09-10, and the compose stop grace period default of ten seconds from the compose reference, accessed 2026-09-09. Verified by `make verify-infra-docker`, whose teardown stops all six services through this path, run green 2026-09-20.

4. compose and one click local systems

One container proves a process works. A system is several containers that find each other, start in a defensible order, and report when they are actually ready, and compose is the declarative file that says all of it. The capstone's `compose.yml` is eight services and four volumes, and the claim it backs is strong: `docker compose up -d --wait` returning means the chat system works, not merely that eight containers exist.

4.1. healthchecks, the difference between started and working

A container that has a process in it is "running" even if that process is still dialing its broker or waiting on its database. The healthcheck closes that gap: a command the engine runs inside the container on an interval, and a container is healthy only when the command succeeds. The capstone compresses the shared timing into one yaml anchor:

```yaml
x-hc: &hc
  interval: 2s
  timeout: 2s
  retries: 30
  start_period: 3s
```

listing 6: one anchor, seven of eight services: probe every 2s, 30 retries, 3s of startup grace (source: infrastructure/capstone/compose.yml, lines 12-16)

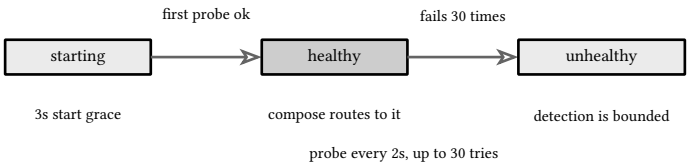


figure 9: container states under the shared probe, from started to working

The brokers first, since everything else depends on them:

```yaml
  nats:
    image: nats:2.14.6-alpine
    command: ["-js", "-sd", "/data", "-m", "8222"]
    volumes:
      - natsdata:/data
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8222/healthz"]
      <<: *hc
  mongo:
    image: mongo:8.0
    volumes:
      - mongodata:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 2s
      timeout: 5s
      retries: 30
      start_period: 10s
```

listing 7: nats probes its own monitoring endpoint, mongo pings itself through mongosh (source: infrastructure/capstone/compose.yml, lines 19-36)

The nats image exposes a healthz endpoint on its monitoring port, and the alpine base carries busybox wget to fetch it. The mongo image carries mongosh, so its probe is a real ping through the real client, and it is the one service

outside the anchor, setting its own slower timing, a 5s timeout and a 10s start period, because a cold database needs more grace than a binary answering one route. My own services each serve a `/healthz` http endpoint from `internal/boot`, one route per process, so the probe asserts the process can still answer, which is the definition of alive the operator wants.

4.2. depends_on with a condition, and why up --wait means it

```
  chat:
    <<: *go-service
    build:
      context: .
      args:
        SERVICE: chat
    environment:
      NATS_URL: nats://nats:4222
      SQLITE_PATH: /data/chat.db
    volumes:
      - chatdata:/data
    depends_on:
      nats:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8081/healthz"]
      <<: *hc
```

listing 8: chat waits for a healthy nats, and carries its own probe on 8081 (source: infrastructure/capstone/compose.yml, lines 37-53)

The `condition: service_healthy` form is the load-bearing one. Without a condition, depends_on only orders container starts, which on a slow mongo means the history service starts against a dependency that is seconds from existing and dies, or worse, retries in a loop forever. With conditions, the dependency graph is a readiness graph:

```
  web:
    <<: *go-service
    build:
      context: .
      args:
        SERVICE: web
    environment:
      NATS_URL: nats://nats:4222
    ports:
      - "8080:8080"
    depends_on:
      chat:
        condition: service_healthy
      presence:
        condition: service_healthy
      notify:
        condition: service_healthy
      history:
        condition: service_healthy
      analytics:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:8080/healthz"]
      <<: *hc
```

listing 9: the web tier waits for all five services to be healthy before it can be healthy itself (source: infrastructure/capstone/compose.yml, lines 115-138)

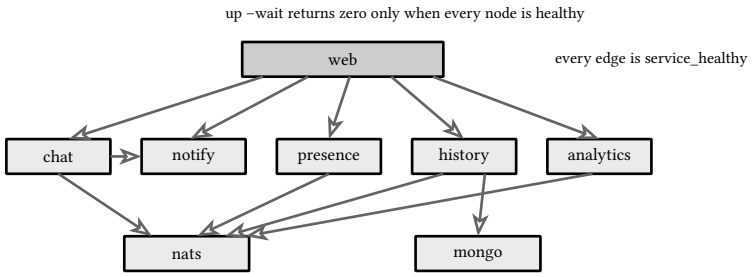


figure 10: the readiness graph the conditions build, and what up –wait waits for

`--wait` turns the whole command synchronous: compose brings the graph up and returns zero only when every container reports healthy. That is what licenses the capstone's one-click claim, and it is why the docker-gated suite in book 13, ch 21 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click) can start testing the moment `up` returns.

4.3. volumes and teardown discipline

Four named volumes hold the nats jetstream store, the durable log book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) builds on, the mongo files, the chat sqlite database, and the analytics duckdb file. The make target that drives this stack tears down with `down -v` even when the tests fail, wrapped in a shell trap, because a half-torn stack with stale volumes is exactly the kind of local drift that makes the next run fail for reasons that are not in the code. The failure mode the trap guards against is simple: a red suite that also leaves a running stack, which the next `up --wait` then passes against stale state.

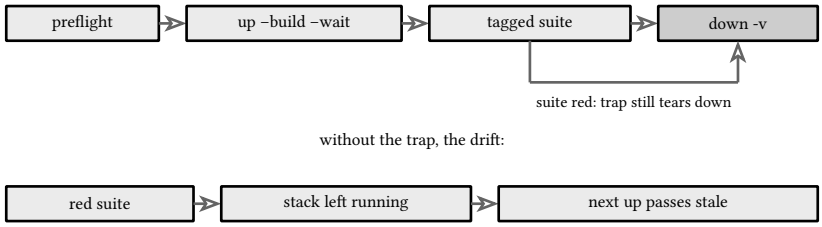


figure 11: the make target lifecycle, the trap that tears down whatever happened

WARNING: the healthcheck is not a monitoring system

A healthcheck is a liveness contract for orchestration, one command, one interval, no history. It tells compose whether to route to this container. It is not metrics, not alerting, and not a substitute for the logging the services already do. Keeping the probe tiny and dependency-free, `wget` on localhost, is what keeps restarts cheap.

sources: docs.docker.com compose file reference for healthcheck, depends_on conditions, and the `--wait` flag, accessed 2026-09-09 and 2026-09-10. Verified by `make verify-infra-docker`, which runs exactly this lifecycle, up, wait, test, down with volumes, green 2026-09-10.

5. container networking

Every compose project gets its own bridge network, created implicitly, and the containers on it can reach each other by service name. That single sentence is most of container networking, and the mechanism under it is worth one chapter because it is the difference between a stack that starts and one that starts then falls over waiting for a hostname that only exists outside.

5.1. the bridge and the embedded dns server

The bridge network is a virtual switch inside the docker engine. Each container gets a virtual interface on it with a private address, and traffic between containers on the same bridge never leaves the host. Alongside the switch the engine runs an embedded dns server, and every container's resolver points at it. When the chat service resolves `nats`, that name never reaches the internet, the embedded server answers with the nats container's address on the bridge.

That is why the capstone's configuration is bare names everywhere:

```
    environment:
      NATS_URL: nats://nats:4222
      SQLITE_PATH: /data/chat.db
    volumes:
      - chatdata:/data
```

listing 10: the chat service finds its broker by service name, inside the project bridge (source: infrastructure/capstone/compose.yml, lines 43-47)

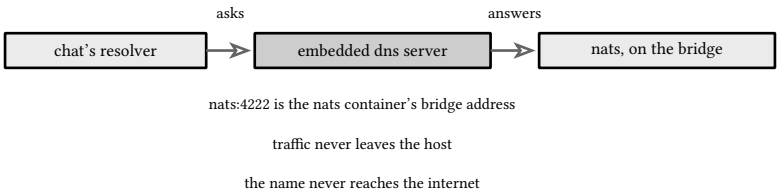


figure 12: resolving nats inside the project bridge, no further than the engine

`NATS_URL: nats://nats:4222` is not a placeholder that works by luck. The host `nats` is the compose service name, resolved by the embedded server, and every service in the file uses the same pattern with zero ip addresses anywhere. The same stanza carries the one thing that does punch out of the bridge: the sqlite path on a named volume, so the database file outlives the container.

5.2. one published port, on purpose

```
    environment:
      NATS_URL: nats://nats:4222
    ports:
      - "8080:8080"
```

listing 11: only the web tier publishes a host port (source: infrastructure/capstone/compose.yml, lines 121-124)

The other seven services are reachable inside the bridge and invisible from the host. That is a design rule, not thrift: the web tier is the system's only public surface, and keeping the brokers and workers unpublished means the host's port space stays clean and nothing can bypass the front door. When a test needs mongo or nats directly, the docker-gated suite reaches the system through the web port like every other client.

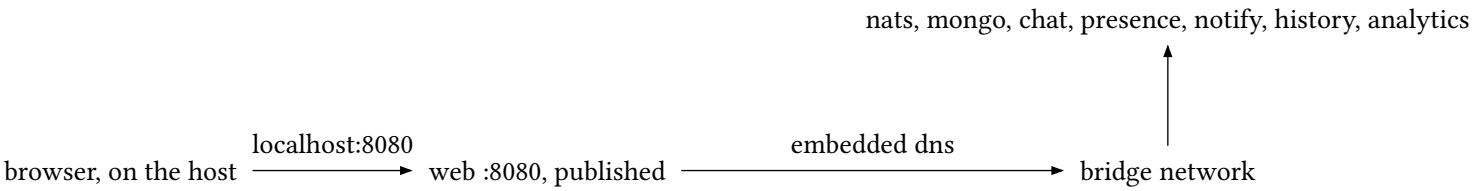


figure 13: who can talk to whom

5.3. the subject space is the other network

The bridge moves bytes between processes. The application-level network is the nats subject tree, `chat.room.<room>`, `notify.user.<user>`, `presence.beat`, and the request subjects, and the capstone's services never address each other by host at all beyond the one broker url. A message published on `chat.room.general` is routed by the broker to whoever expressed interest, on whichever container that subscriber landed. The two networks compose: dns finds the broker once at startup, subjects do everything after.

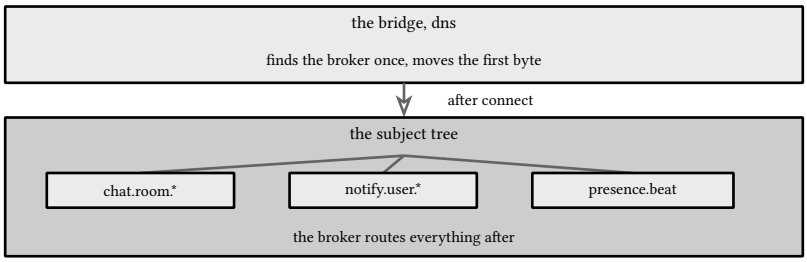


figure 14: two stacked networks: the bridge connects once, the subject tree routes everything after

NOTE: why the unit tests never need dns

The embedded server from book 13, ch 14 (infrastructure: docker, databases, and testing, nats: pub sub, request reply, queue groups) hands tests a real client url, so the same code path, connect by url, dial, subscribe, runs in-process and in-container. The bridge and its dns server exist to make that one url resolvable in production. Nothing else in the capstone knows networking exists.

sources: docs.docker.com engine network drivers for bridge networking and the embedded dns server, accessed 2026-09-09. Verified by `make verify-infra-docker`, whose suite reaches the whole system through the single published port, green 2026-09-10.

6. sqlite: schemas and normalization

The chat service persists to one sqlite file, and the schema it uses is a small normalization argument in sql. The naive chat table is one wide row per message: sender text, room text, body, timestamp. It works on day one and rots on day two, because a sender's display name is stored on every message they ever sent, and changing it means rewriting history instead of one row.

The normalized shape stores each name once and references it by id:

```
-- the chat schema, normalized: names live once in users and rooms, every
-- message references them by id, so a renamed room or user is one update
-- and no message rewrites. foreign keys are on at open time, the store
-- package owns that pragma.
create table users (
  id   integer primary key,
  name text not null unique
);

create table rooms (
  id   integer primary key,
  name text not null unique
);

create table messages (
  id      integer primary key,
  room_id integer not null references rooms(id),
  user_id integer not null references users(id),
  body    text not null,
  ts      integer not null
);
```

listing 12: the first migration: names live once, messages reference them (source: infrastructure/capstone/internal/migrate/migrations/0001_init.sql, lines 1-21)

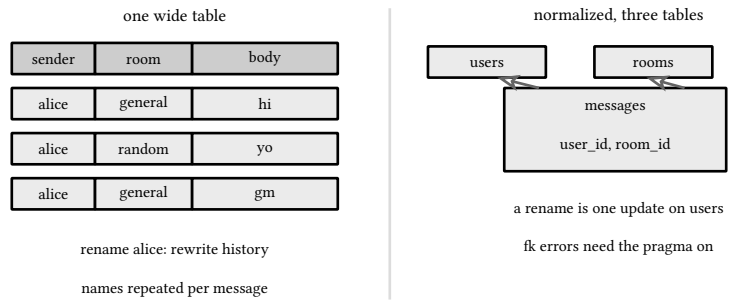


figure 15: the wide row against the normalized shape, and what a rename costs

One detail in the `id integer primary key` columns matters beyond this chapter: every sqlite table has a hidden rowid, a row number the storage b-tree keys on, and a column declared `integer primary key` is that rowid, so an id is the row's position, not a second stored copy. A `without rowid` table, the keyed store's shape in book 13, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves), drops the hidden column and clusters the b-tree on the declared primary key instead.

Three tables, and the cost of the join is paid back twice. A rename is one update on `users`. Room membership queries count distinct ids instead of comparing text. And the foreign key clauses, `references rooms(id)`, turn a class of application bugs, messages pointing at a room that does not exist, into database errors, but only if the connection asks for them:

```
  if _, err := db.Exec(
    "pragma busy_timeout=5000; pragma foreign_keys=on;"); err ≠ nil {
    db.Close()
    return nil, err
  }
```

listing 13: foreign key enforcement is a per-connection pragma, not a file property (source: infrastructure/capstone/internal/store/store.go, lines 43-47)

That listing is the trap most sqlite schemas hit once. The `references` clause is parsed always and enforced never, unless the connection sets `pragma foreign_keys=on`. The capstone's `store.Open` sets it on every open, and the test that pins it inserts a message with invented ids and expects the rejection.

6.1. the second migration, additive by design

Schemas change after data exists. The second migration ships what a live database can absorb without a rewrite:

```
-- the second migration ships after the first ran somewhere: it adds the
-- index the recent messages query wants and a rollup view over the same
-- join, both additive, both things a live database can absorb while the
-- chat service keeps writing.
create index messages_room_id_id on messages(room_id, id);

create view room_counts as
select r.name as room,
       count(*) as messages,
       count(distinct m.user_id) as talkers
from rooms r
join messages m on m.room_id = r.id
group by r.name;
```

listing 14: an index the recent query wants, and a rollup view over the join (source: infrastructure/capstone/internal/migrate/migrations/0002_rollup.sql, lines 1-13)

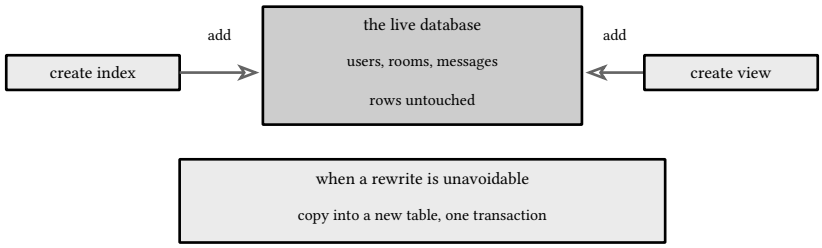


figure 16: migration 0002 lands on a live database, adding without touching a row

The index serves the exact access path the chat service runs, newest messages of one room, which is a range scan on `(room_id, id)`. The view is a named query, and the chapter's probes exercise it through the migration suite, which asserts the view returns one row after one message lands. Neither statement touches existing rows, which is the rule for migrations on live files: add, never rewrite, and when a rewrite is unavoidable, do it as a copy into a new table inside one transaction, the pattern book 13, ch 18 (infrastructure: docker, databases, and testing, migrations, test driven) builds the runner for.

The normalized shape also travels. book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine) takes this exact schema, both migrations verbatim, attaches the file to duckdb, and asks the attached tables the same questions the view answers. The two engines must agree on every row, which is the cheapest proof that normalization bought nothing at the price of portability.

NOTE: denormalize on purpose or not at all

> Normalization is the default because it is the safe default. The capstone denormalizes nowhere: message counts are computed, rooms are joined. If a room counter column ever became worth it, the chapter's rule would be to add it as a maintained, derived column written in the same transaction as the message, never as a second source of truth the application remembers to update.

sources: sqlite.org pragma reference for the per-connection foreign_keys behavior, accessed 2026-09-10, and sqlite.org lang pages for index and view semantics. Verified by the migrate and store suites, 10 tests, including the foreign key rejection and the view row count, run green under `make verify` 2026-09-10.

7. sqlite: transactions and isolation

Sqlite's concurrency story is small enough to hold in your head and strict enough to bite anyone who holds it loosely. This chapter pins the actual behavior with executed probes against the 3.53.4 engine the repo links, every claim a test that fails if the engine changes its mind.

7.1. a read transaction is a snapshot

A deferred transaction, what go's `database/sql` gives you with `Begin`, takes no lock at all until the first statement runs. A read statement takes the shared lock and, more importantly, fixes the transaction's view: everything committed after that first read is invisible until the transaction ends. That is repeatable read, with no isolation level to configure, and the probe pins both halves:

```
func TestReaderSeesOnlyCommittedState(t *testing.T) {
  path := filepath.Join(t.TempDir(), "iso.db")
  writer := open(t, path, "pragma journal_mode=wal", "pragma busy_timeout=2000")
  reader := open(t, path, "pragma journal_mode=wal", "pragma busy_timeout=2000")
  exec(t, writer, "create table t(x integer)")
  exec(t, writer, "insert into t values (1)")

  // open the reader's snapshot before the writer changes anything
  readTx, err := reader.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  var before int
  if err := readTx.QueryRow("select count(*) from t").Scan(&before); err ≠ nil {
    t.Fatal(err)
  }

  // in wal mode the writer commits without waiting for the reader
  exec(t, writer, "insert into t values (2)")
  if got := count(t, writer, "t"); got ≠ 2 {
    t.Fatalf("writer sees %d rows", got)
  }

  var mid int
  if err := readTx.QueryRow("select count(*) from t").Scan(&mid); err ≠ nil {
    t.Fatal(err)
  }
  if mid ≠ before {
    t.Fatalf("reader's open transaction saw %d rows, want the snapshot's %d", mid, before)
  }
  readTx.Rollback()

  if after := count(t, reader, "t"); after ≠ 2 {
    t.Fatalf("fresh read after commit saw %d rows, want 2", after)
  }
}
```

listing 15: the snapshot opens at first read and survives another connection's commit (source: infrastructure/samples/ch07/isolation_test.go, lines 70-105)

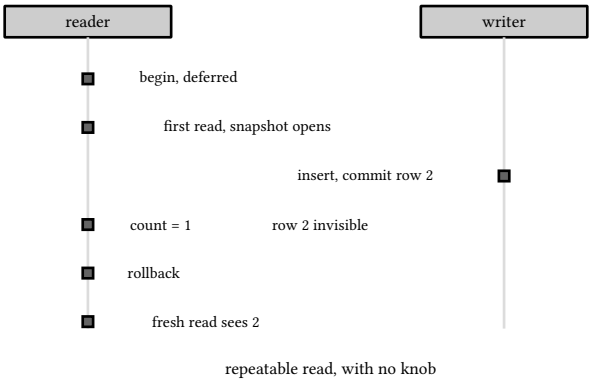


figure 17: the snapshot interleaving, two connections on one timeline

The reader's open transaction counts one row after the writer committed a second, and a fresh read after the rollback sees two. The writer in this probe runs in wal mode, where commits do not wait for readers, which is the next chapter's whole subject.

7.2. the lock ladder, shared to reserved to exclusive

Rollback journal mode, the non-wal default, walks a ladder. A reader holds shared. A writer's insert takes reserved, which coexists with any number of shared locks, so writers really can write while readers read. The commit is where it ends: committing needs exclusive, and exclusive waits for every shared lock to release:

```
func TestRollbackCommitNeedsExclusive(t *testing.T) {
  path := filepath.Join(t.TempDir(), "rollback.db")
  writer := open(t, path, "pragma journal_mode=delete", "pragma busy_timeout=0")
  reader := open(t, path, "pragma journal_mode=delete", "pragma busy_timeout=0")
  exec(t, writer, "create table t(x integer)")
  exec(t, writer, "insert into t values (1)")

  readTx, err := reader.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  if err := readTx.QueryRow("select count(*) from t").Scan(new(int)); err ≠ nil {
    t.Fatal(err)
  }

  wtx, err := writer.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  // the insert takes the reserved lock, which coexists with readers
  if _, err := wtx.Exec("insert into t values (2)"); err ≠ nil {
    t.Fatalf("insert under shared readers: %v", err)
  }
  err = wtx.Commit()
  if err == nil || !strings.Contains(err.Error(), "locked") {
    t.Fatalf("commit under an open reader = %v, want SQLITE_BUSY", err)
  }
  readTx.Rollback()
  wtx.Rollback()

  // once the reader is gone the same sequence commits
  if _, err := writer.Exec("insert into t values (3)"); err ≠ nil {
    t.Fatal(err)
  }
  if got := count(t, writer, "t"); got ≠ 2 {
    t.Fatalf("rows = %d, want 2", got)
  }
}
```

listing 16: the insert lands under readers, the commit does not, until the reader closes (source: infrastructure/samples/ch07/isolation_test.go, lines 110-147)

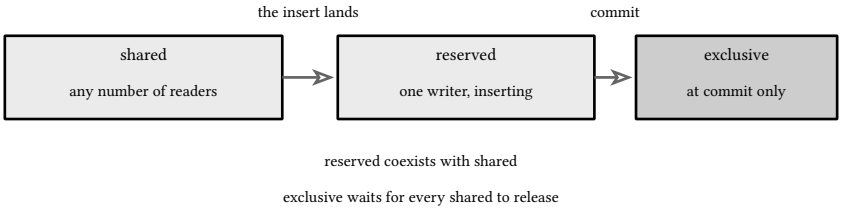


figure 18: the lock ladder a writer climbs, and what blocks at the top

Two behaviors live in that probe. With `busy_timeout=0` the blocked commit surfaces as an immediate `SQLITE_BUSY` error, and after the reader rolls back the same sequence commits. The nuance most people miss is that the write itself succeeded, only the durability step was refused, so an application that retries the wrong thing, the insert instead of the commit, corrupts nothing but duplicates plenty.

7.3. busy_timeout, the difference between error and wait

busy_timeout tells sqlite to park instead of failing when a lock is held, and the probe measures it happening:

```
func TestBusyTimeoutWaitsForTheReader(t *testing.T) {
  path := filepath.Join(t.TempDir(), "wait.db")
  writer := open(t, path, "pragma journal_mode=delete", "pragma busy_timeout=3000")
  reader := open(t, path, "pragma journal_mode=delete", "pragma busy_timeout=3000")
  exec(t, writer, "create table t(x integer)")
  exec(t, writer, "insert into t values (1)")

  readTx, err := reader.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  if err := readTx.QueryRow("select count(*) from t").Scan(new(int)); err ≠ nil {
    t.Fatal(err)
  }
  go func() {
    time.Sleep(400 * time.Millisecond)
    readTx.Rollback()
  }()

  start := time.Now()
  if _, err := writer.Exec("insert into t values (2)"); err ≠ nil {
    t.Fatal(err)
  }
  elapsed := time.Since(start)
  if elapsed < 350*time.Millisecond {
    t.Fatalf("commit returned in %v, before the reader released", elapsed)
  }
  if got := count(t, writer, "t"); got ≠ 2 {
    t.Fatalf("rows = %d, want 2", got)
  }
}
```

listing 17: the writer's commit parks roughly 400ms until the goroutine releases the reader (source: infrastructure/samples/ch07/isolation_test.go, lines 151-181)

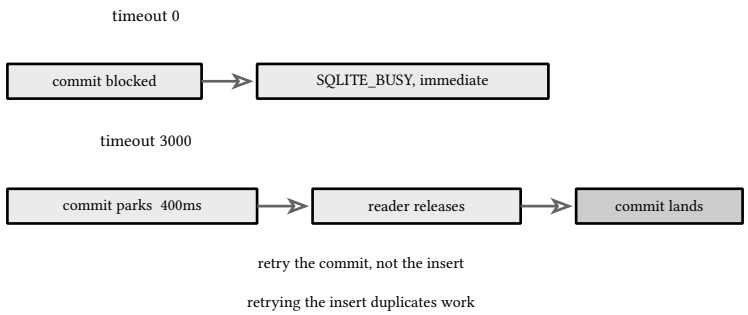


figure 19: one blocked commit, two settings: error at once or a bounded wait

The elapsed-time assertion is the honest part: the test would fail if the commit sneaked in before the reader released, and it would fail if the wait were unbounded, because the deadline goroutine would leave the reader open and the test would time out instead.

Writers serialize with each other in every mode, and the last probe covers both sides of that: a zero-timeout second writer gets the error, a waiting one gets the lock after the first commits:

```
func TestSecondWriterWaitsOrFails(t *testing.T) {
  path := filepath.Join(t.TempDir(), "lock.db")
  first := open(t, path, "pragma journal_mode=wal")
  second := open(t, path, "pragma journal_mode=wal", "pragma busy_timeout=3000")
  exec(t, first, "create table t(x integer)")

  tx, err := first.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  if _, err := tx.Exec("insert into t values (1)"); err ≠ nil {
    t.Fatal(err)
  }

  // zero timeout: busy surfaces as an error, not a wait
  patient, _ := sql.Open("sqlite", path)
  t.Cleanup(func() { patient.Close() })
  if _, err := patient.Exec("pragma busy_timeout=0"); err ≠ nil {
    t.Fatal(err)
  }
  _, impatientErr := patient.Exec("insert into t values (99)")
  if impatientErr == nil {
    t.Fatal("want immediate SQLITE_BUSY with busy_timeout=0")
  } else if !strings.Contains(impatientErr.Error(), "locked") &&
    !strings.Contains(impatientErr.Error(), "BUSY") {
    t.Fatalf("unexpected error: %v", impatientErr)
  }

  // release, then the waiting writer goes through
  if err := tx.Commit(); err ≠ nil {
    t.Fatal(err)
  }
  done := make(chan error, 1)
  go func() {
    _, err := second.Exec("insert into t values (2)")
    done ← err
  }()
  select {
  case err := ←done:
    if err ≠ nil {
      t.Fatal(err)
    }
  case ←time.After(5 * time.Second):
    t.Fatal("second writer never got the lock")
  }
  // rows 1 and 2 landed; 99 never did
  if got := count(t, first, "t"); got ≠ 2 {
    t.Fatalf("rows = %d, want 2", got)
  }
}
```

listing 18: immediate busy without a timeout, ordered handoff with one (source: infrastructure/samples/ch07/isolation_test.go, lines 186-235)

That pair is the whole case for the capstone's `store.Open` setting `busy_timeout=5000`: transient contention between the chat writer and an occasional administrative write becomes a five second wait, not an error the user sees.

WARNING: isolation is per transaction, not per connection

> A connection with no open transaction sees the latest committed state on every query, which is why the probe rolls back before its final count. Treating a pooled `*sql.DB` as if it had one long-lived snapshot, which go's pool makes it not have anyway, is the classic way to be surprised by sqlite's "anomalies" that are actually two different transactions doing exactly what they said.

sources: sqlite.org lockingv3 for the shared, reserved, exclusive ladder and the wal comparison, accessed 2026-09-10. Verified by the ch07 probe suite, 5 tests, executed against the modernc 3.53.4 engine and cross-checked against the pinned amalgamation binary in tools/sqlite/build, green under `make verify` 2026-09-10.

8. sqlite: wal, locking, crash semantics

Rollback mode's commit waits for readers, book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation) established that. Write-ahead logging removes the wait by changing where the commit writes: instead of modifying the database file in place under an exclusive lock, a wal writer appends the changed pages to a sidecar `-wal` file and returns. Readers walk the database file plus the sidecar, each transaction pointed at the point-in-time snapshot it started at. One writer at a time still, but readers never block a writer and a writer never blocks readers.

8.1. the mode is a property of the file

```
func TestWalModeSticksAndPersists(t *testing.T) {
  path := filepath.Join(t.TempDir(), "wal.db")
  db := open(t, path)
  var mode string
  if err := db.QueryRow("pragma journal_mode=wal").Scan(&mode); err ≠ nil {
    t.Fatal(err)
  }
  if mode ≠ "wal" {
    t.Fatalf("journal_mode = %s, want wal", mode)
  }
  if _, err := db.Exec("create table t(x integer); insert into t values (1)"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := os.Stat(path + "-wal"); err ≠ nil {
    t.Fatalf("no -wal sidecar after writes: %v", err)
  }
  db.Close()

  // journal mode is durable in the database header: a fresh open
  // reports wal without being told
  again, err := sql.Open("sqlite", path)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer again.Close()
  var remode string
  if err := again.QueryRow("pragma journal_mode").Scan(&remode); err ≠ nil {
    t.Fatal(err)
  }
  if remode ≠ "wal" {
    t.Fatalf("journal_mode on reopen = %s, want wal", remode)
  }
}
```

listing 19: wal on, the sidecar appears on write, and a fresh open reports wal unprompted (source: infrastructure/samples/ch08/wal_test.go, lines 33-65)

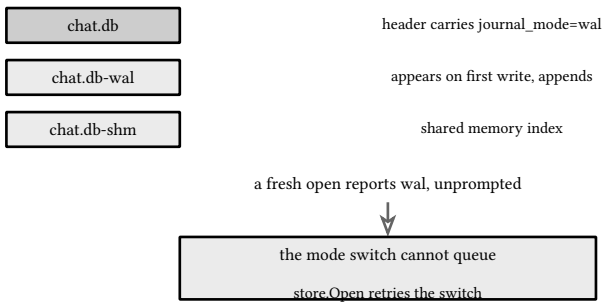


figure 20: the wal trio on disk, and the one pragma that refuses to queue

The reopen assertion matters operationally: `journal_mode` is stored in the database header, so setting it once is enough for every future connection, including ones made by tools that never ran the pragma. That is why `store.Open` can set it defensively on every open at zero cost:

```
func Open(path string) (*sql.DB, error) {
  db, err := sql.Open("sqlite", path)
  if err ≠ nil {
    return nil, err
  }
  // sqlite refuses to queue the journal-mode switch behind another
  // connection, so a brief retry covers two services racing to create
  // the same fresh file
  for attempt := 0; ; attempt++ {
    var mode string
    err = db.QueryRow("pragma journal_mode=wal").Scan(&mode)
    if err == nil {
      break
    }
    if attempt ≥ 20 {
      db.Close()
      return nil, fmt.Errorf("store: set wal: %w", err)
    }
    sleepBrief()
  }
  if _, err := db.Exec(
    "pragma busy_timeout=5000; pragma foreign_keys=on;"); err ≠ nil {
    db.Close()
    return nil, err
  }
  if _, err := migrate.Up(db, migrate.FS); err ≠ nil {
    db.Close()
    return nil, fmt.Errorf("store: migrate: %w", err)
  }
  return db, nil
}
```

listing 20: open: wal with a retry, busy timeout, foreign keys, then migrations (source: infrastructure/capstone/internal/store/store.go, lines 23-53)

The retry loop around the switch is the one place sqlite refuses to queue: the journal-mode change cannot wait behind another connection, so two services racing to create the same fresh file retry briefly instead of failing.

8.2. checkpointing, the price of the sidecar

The sidecar grows until something folds it back. Sqlite checkpoints automatically when it reaches a threshold, and the operator's lever is the explicit form:

```
func TestCheckpointTruncateResetsTheSidecar(t *testing.T) {
  path := filepath.Join(t.TempDir(), "ckpt.db")
  db := open(t, path, "pragma journal_mode=wal")
  if _, err := db.Exec("create table t(x integer)"); err ≠ nil {
    t.Fatal(err)
  }
  for i := 0; i < 100; i++ {
    if _, err := db.Exec("insert into t values (?)", i); err ≠ nil {
      t.Fatal(err)
    }
  }
  walBefore, err := os.Stat(path + "-wal")
  if err ≠ nil {
    t.Fatal(err)
  }
  var busy, log, check int
  if err := db.QueryRow("pragma wal_checkpoint(truncate)").Scan(&busy, &log, &check); err ≠ nil {
    t.Fatal(err)
  }
  if busy ≠ 0 {
    t.Fatalf("checkpoint busy = %d, readers held the wal", busy)
  }
  walAfter, err := os.Stat(path + "-wal")
  if err ≠ nil {
    t.Fatal(err)
  }
  if walAfter.Size() ≥ walBefore.Size() {
    t.Fatalf("wal still %d bytes after truncate checkpoint (was %d)",
      walAfter.Size(), walBefore.Size())
  }
  var n int
  if err := db.QueryRow("select count(*) from t").Scan(&n); err ≠ nil {
    t.Fatal(err)
  }
  if n ≠ 100 {
    t.Fatalf("rows = %d after checkpoint", n)
  }
}
```

listing 21: truncate checkpoint resets the sidecar without losing a row (source: infrastructure/samples/ch08/wal_test.go, lines 69-106)

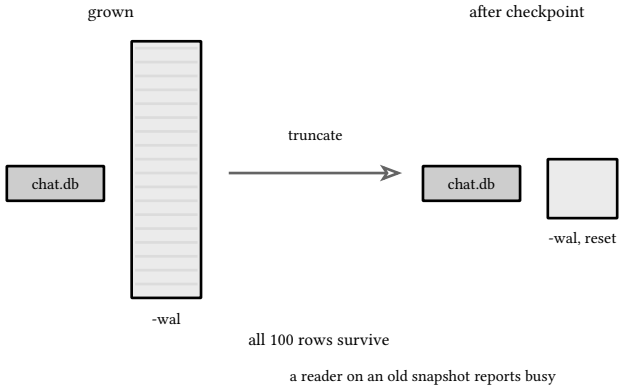


figure 21: the sidecar before and after a truncate checkpoint

The probe asserts the sidecar shrinks and all hundred rows survive. The `busy` return value is the other lesson, and a second probe pins both halves: a checkpoint cannot finish while a reader holds an old snapshot, reported as

nonzero busy, and the same checkpoint returns zero busy once the reader releases, the rare case where wal still makes someone wait.

8.3. backup without a backup api

`VACUUM INTO` writes a consistent snapshot of the whole database to a fresh path, in one statement, without holding locks on the target and without driver-specific backup machinery:

```
func TestVacuumIntoBackup(t *testing.T) {
  dir := t.TempDir()
  path := filepath.Join(dir, "live.db")
  db := open(t, path, "pragma journal_mode=wal")
  if _, err := db.Exec("create table t(x integer)"); err ≠ nil {
    t.Fatal(err)
  }
  for i := 0; i < 50; i++ {
    if _, err := db.Exec("insert into t values (?)", i); err ≠ nil {
      t.Fatal(err)
    }
  }
  backup := filepath.Join(dir, "backup.db")
  if _, err := db.Exec("vacuum into ?", backup); err ≠ nil {
    t.Fatal(err)
  }

  bak, err := sql.Open("sqlite", backup)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer bak.Close()
  var n int
  if err := bak.QueryRow("select count(*) from t").Scan(&n); err ≠ nil {
    t.Fatal(err)
  }
  if n ≠ 50 {
    t.Fatalf("backup rows = %d, want 50", n)
  }
  var check string
  if err := bak.QueryRow("pragma integrity_check").Scan(&check); err ≠ nil {
    t.Fatal(err)
  }
  if check ≠ "ok" {
    t.Fatalf("backup integrity = %s", check)
  }
  // the backup is a fresh standalone file, no wal sidecar came along
  if _, err := os.Stat(backup + "-wal"); !os.IsNotExist(err) {
    t.Fatal("backup carries a wal sidecar")
  }
}
```

listing 22: vacuum into: the backup opens, passes integrity_check, and carries no sidecar (source: infrastructure/samples/ch08/wal_test.go, lines 110-150)

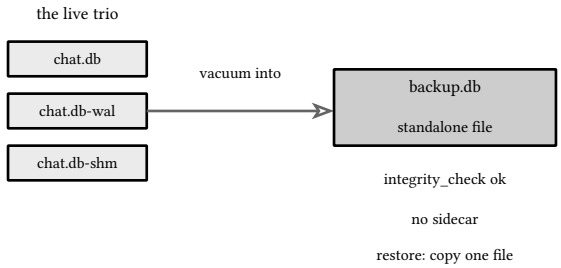


figure 22: vacuum into: the live trio against the one-file backup

The last assertion is the easy one to forget: a snapshot taken this way is a standalone database, not a wal trio, so restoring it is copying one file.

8.4. crash semantics, by experiment

The durability claim, a crash leaves the old state or the new, never half, is testable. The probe forks a child process that opens the database, begins, inserts, and exits hard, `os.Exit(3)`, no commit, no close. It is as close to pulling power as a test suite honestly gets:

```
func TestCrashBeforeCommitKeepsCommittedState(t *testing.T) {
  if os.Getenv("CH08_CRASH_CHILD") == "1" {
    crashChild()
    return
  }
  dir := t.TempDir()
  path := filepath.Join(dir, "crash.db")
  db := open(t, path, "pragma journal_mode=wal")
  if _, err := db.Exec("create table t(x integer); insert into t values (1)"); err ≠ nil {
    t.Fatal(err)
  }
  db.Close()

  cmd := exec.Command(os.Args[0], "-test.run=TestCrashBeforeCommitKeepsCommittedState")
  cmd.Env = append(os.Environ(), "CH08_CRASH_CHILD=1", "CH08_DB="+path)
  out, err := cmd.CombinedOutput()
  if err == nil {
    t.Fatalf("child exited cleanly, it was supposed to crash:\n%s", out)
  }

  after, err := sql.Open("sqlite", path)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer after.Close()
  var n int
  if err := after.QueryRow("select count(*) from t").Scan(&n); err ≠ nil {
    t.Fatalf("reopen after crash: %v", err)
  }
  if n ≠ 1 {
    t.Fatalf("rows after crash = %d, want the committed 1", n)
  }
  var check string
  if err := after.QueryRow("pragma integrity_check").Scan(&check); err ≠ nil {
    t.Fatal(err)
  }
  if check ≠ "ok" {
    t.Fatalf("integrity after crash = %s", check)
  }
  // and the database still accepts writes: recovery finished
  if _, err := after.Exec("insert into t values (2)"); err ≠ nil {
    t.Fatalf("post crash write: %v", err)
  }
}
```

listing 23: parent seeds, child dies mid transaction, parent reopens and checks (source: infrastructure/samples/ch08/wal_test.go, lines 156-199)

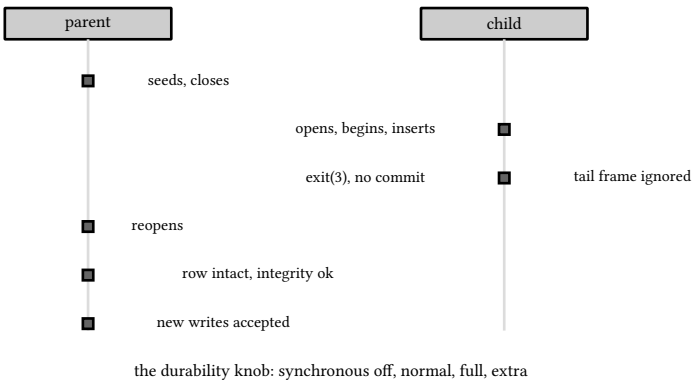


figure 23: the crash experiment, parent and child on one timeline

The parent then asserts three things in order: the committed row is still there, `integrity_check` says ok, and the database accepts new writes, meaning recovery actually ran, not merely that nothing broke. Behind the scenes the reopening connection found the sidecar, saw an uncommitted tail frame, and ignored it, which is the entire crash safety story in one sentence.

The `synchronous` pragma is the knob that trades commit latency against how fanatical the fsyncs are, and the probe pins the readback values for all four levels, off through extra, so the settings in prose always mean something concrete:

```
func TestSynchronousLevels(t *testing.T) {
  db := open(t, ":memory:")
  for _, want := range []struct {
    pragma string
    value  int
  }{
    {"pragma synchronous=off", 0},
    {"pragma synchronous=normal", 1},
    {"pragma synchronous=full", 2},
    {"pragma synchronous=extra", 3},
  } {
    if _, err := db.Exec(want.pragma); err ≠ nil {
      t.Fatalf("%s: %v", want.pragma, err)
    }
    var got int
    if err := db.QueryRow("pragma synchronous").Scan(&got); err ≠ nil {
      t.Fatal(err)
    }
    if got ≠ want.value {
      t.Fatalf("%s reads back %d, want %d", want.pragma, got, want.value)
    }
  }
}
```

listing 24: the four synchronous levels and their numeric readback (source: infrastructure/samples/ch08/wal_test.go, lines 230-252)

The capstone runs the default, full, because a chat log is not worth a data race against physics, and the raft store in the patterns book makes the same choice for harder reasons.

NOTE: wal is not a server

Wal mode adds reader and writer concurrency inside one process boundary, many connections in one or several processes on one machine. It does not turn sqlite into a networked database, and the file must stay on a filesystem with working locking semantics, which is why the compose stack gives the chat service a named volume and exactly one writer.

sources: sqlite.org wal.html for the architecture, checkpoints, and the reader and writer concurrency guarantees, lockingv3 for the comparison ladder, lang/vacuum for `VACUUM INTO`, accessed 2026-09-10. Verified by the ch08 probe suite, 6 tests including the subprocess crash and the reader-blocked checkpoint, green under `make verify` 2026-09-10.

9. sqlite: the 3.53 feature set

This book pins one engine, 3.53.4, across every runtime that touches it, and this chapter is what that pin buys: a tour of the feature set where every capability is asserted present and the absent ones are asserted absent. Each probe ran against both the modernc driver and the amalgamation binary in `tools/sqlite/build`, and where the two could disagree they did not: same version string, same behavior.

9.1. json in sql

Json is a first class type in sql, not a blob the application parses. Extraction, patching, pretty printing, and the `json_each` table valued function:

```
func TestJsonFunctions(t *testing.T) {
  db := open(t)
  var extracted int
  if err := db.QueryRow(`select json_extract('{"a": 1}', '$.a')`).Scan(&extracted); err ≠ nil {
    t.Fatal(err)
  }
  if extracted ≠ 1 {
    t.Fatalf("json_extract = %d", extracted)
  }

  var pretty string
  if err := db.QueryRow(`select json_pretty('[1,{"b":2}]')`).Scan(&pretty); err ≠ nil {
    t.Fatal(err)
  }
  if !strings.Contains(pretty, "\"b\": 2") {
    t.Fatalf("json_pretty = %q", pretty)
  }

  var patched string
  if err := db.QueryRow(`select json_patch('{"a":1}', '{"b":2}')`).Scan(&patched); err ≠ nil {
    t.Fatal(err)
  }
  if patched ≠ `{"a":1,"b":2}` {
    t.Fatalf("json_patch = %s", patched)
  }

  var keys int
  if err := db.QueryRow(
    `select count(*) from json_each('{"a":1,"b":2}')`).Scan(&keys); err ≠ nil {
    t.Fatal(err)
  }
  if keys ≠ 2 {
    t.Fatalf("json_each rows = %d", keys)
  }
}
```

listing 25: extract, pretty, patch, and json_each, all plain queries (source: infrastructure/samples/ch09/features_test.go, lines 40-74)

The pattern that earns its keep is the path into a column. An events table that grew a json payload column answers structural questions without a schema change and without unmarshalling in go:

```
func TestJsonColumnAndPath(t *testing.T) {
  db := open(t)
  if _, err := db.Exec(`create table events(
    id integer primary key,
    payload text not null
  )`); err ≠ nil {
    t.Fatal(err)
  }
  for _, p := range []string{
    `{"kind": "join", "user": "alice"}`,
    `{"kind": "msg", "user": "bob"}`,
  } {
    if _, err := db.Exec("insert into events(payload) values (?)", p); err ≠ nil {
      t.Fatal(err)
    }
  }
  var n int
  if err := db.QueryRow(
    `select count(*) from events where json_extract(payload, '$.kind') = 'join'`,
  ).Scan(&n); err ≠ nil {
    t.Fatal(err)
  }
  if n ≠ 1 {
    t.Fatalf("json path filter matched %d rows", n)
  }
}
```

listing 26: a where clause that reaches inside a json column (source: infrastructure/samples/ch09/features_test.go, lines 78-103)

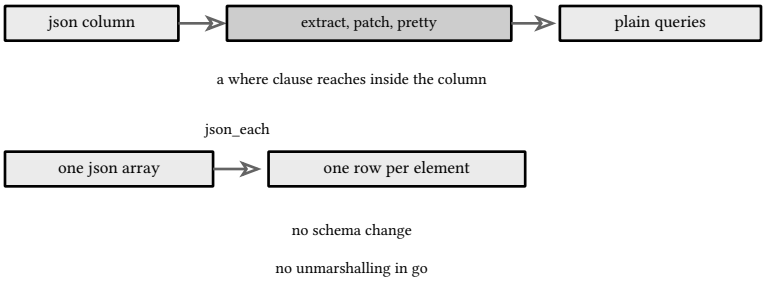


figure 24: json stays in sql: paths and streams, no unmarshalling in go

9.2. alter column constraints, the 3.53.1 addition

Before 3.53, editing a column's constraints meant the twelve step table rebuild from the sqlite documentation. Now the not null constraint is editable in place, and the probe exercises the whole life: set, enforced rejection, drop:

```
func TestAlterColumnConstraints(t *testing.T) {
  db := open(t)
  if _, err := db.Exec("create table t(a integer primary key, b text)"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec("insert into t values (1, 'x')"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec("alter table t alter column b set not null"); err ≠ nil {
    t.Fatalf("set not null: %v", err)
  }
  var notnull int
  if err := db.QueryRow(
    `select "notnull" from pragma_table_info('t') where name = 'b'`).Scan(&notnull); err ≠ nil {
    t.Fatal(err)
  }
  if notnull ≠ 1 {
    t.Fatalf("pragma notnull = %d after set", notnull)
  }

  // the enforced constraint rejects the write
  if _, err := db.Exec("insert into t(a) values (2)"); err == nil {
    t.Fatal("null insert accepted under set not null")
  }

  if _, err := db.Exec("alter table t alter column b drop not null"); err ≠ nil {
    t.Fatalf("drop not null: %v", err)
  }
  if err := db.QueryRow(
    `select "notnull" from pragma_table_info('t') where name = 'b'`).Scan(&notnull); err ≠ nil {
    t.Fatal(err)
  }
  if notnull ≠ 0 {
    t.Fatalf("pragma notnull = %d after drop", notnull)
  }

  // default clause editing is absent in this engine: the statement
  // must fail, and a migration that needs it must rebuild instead
  if _, err := db.Exec("alter table t alter column b set default 'z'"); err == nil {
    t.Fatal("set default parsed, but the pinned engine rejects it")
  }
}
```

listing 27: set not null, feel it reject a null, drop it again, and watch the default variant fail (source: infrastructure/samples/ch09/features_test.go, lines 109-150)

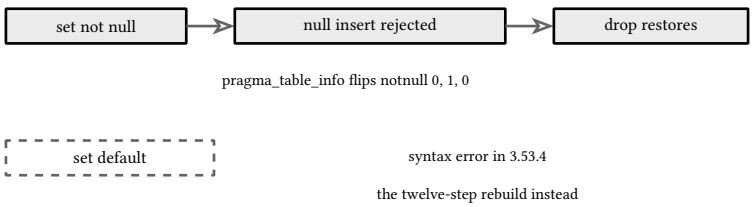


figure 25: the constraint life cycle in place, and the one edit this engine refuses

The last assertion is the honest one. `ALTER COLUMN ... SET DEFAULT` is a syntax error in this exact engine, probed in both runtimes, so a migration needing it must fall back to the rebuild, and the chapter says so rather than hoping. The pragma readback through `pragma_table_info` shows the constraint flipping, and the column is addressed as `"notnull"` quoted, because 3.53 makes the bare word a keyword again in these statements.

9.3. reindex

Index rebuilding runs by index name and by collation name:

```
func TestReindex(t *testing.T) {
  db := open(t)
  if _, err := db.Exec(
    "create table t(a text collate nocase); create index ix on t(a collate nocase)"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec("insert into t values ('Q')"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec("reindex ix"); err ≠ nil {
    t.Fatalf("reindex by name: %v", err)
  }
  if _, err := db.Exec("reindex nocase"); err ≠ nil {
    t.Fatalf("reindex by collation: %v", err)
  }
  if _, err := db.Exec("reindex lower(a)"); err == nil {
    t.Fatal("expression reindex parsed, but the pinned engine rejects it")
  }
}
```

listing 28: reindex by name and by collation, with the expression form pinned absent (source: infrastructure/samples/ch09/features_test.go, lines 154-172)

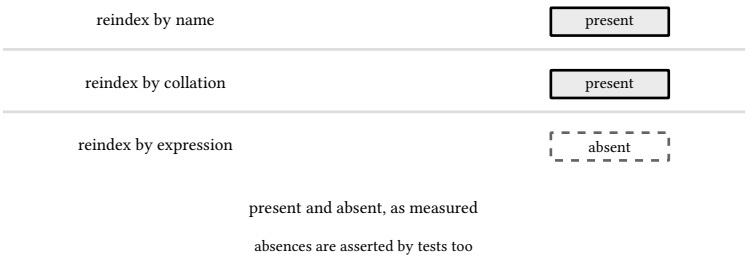


figure 26: reindex, one row of the feature matrix: what runs and what this engine refuses

9.4. money as integer cents

Binary floating point cannot hold a tenth, so money in floats drifts, and the drift is one query long:

```
func TestMoneyAsIntegerCents(t *testing.T) {
  db := open(t)
  var floatSum float64
  // one hundred copies of 19.99
  if err := db.QueryRow(
    "with recursive n(i) as (select 1 union all select i+1 from n where i < 100) " +
      "select sum(19.99) from n").Scan(&floatSum); err != nil {
    t.Fatal(err)
  }
  if floatSum == 1999.0 {
    t.Fatalf("float sum landed exactly on 1999, the drift premise is wrong: %v", floatSum)
  }
  t.Logf("sum(19.99 x 100) as float = %v", floatSum)

  var centSum int64
  if err := db.QueryRow(
    "with recursive n(i) as (select 1 union all select i+1 from n where i < 100) " +
      "select sum(1999) from n").Scan(&centSum); err != nil {
    t.Fatal(err)
  }
  if centSum != 199900 {
    t.Fatalf("cent sum = %d, want 199900", centSum)
  }
}
```

listing 29: one hundred espressos at 19.99: the float sum misses, the cent sum is exact (source: infrastructure/samples/ch09/features_test.go, lines 205-228)

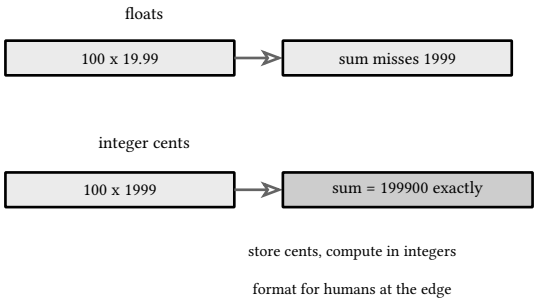


figure 27: the same hundred espressos, summed as floats and as cents

The probe asserts the float sum is not exactly 1999, the drift premise is real and not folklore, then asserts the integer cent sum is exact. The rule it backs: store cents, add and compare in integers, format for humans at the edge where the decimal point is a display concern. The game shop in book 15 prices stock in whole gold for the same reason, no rounding anywhere near a balance.

9.5. strict tables

`STRICT` turns affinity coercion, sqlite's silent type massage, into an error at the boundary:

```go
func TestStrictTables(t *testing.T) {
  db := open(t)
  if _, err := db.Exec("create table ledger(" +
    "amount_cents integer not null, " +
    "note text not null) strict"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec(
    "insert into ledger(amount_cents, note) values (1999, 'espresso')"); err ≠ nil {
    t.Fatal(err)
  }
  if _, err := db.Exec(
    "insert into ledger(amount_cents, note) values ('1999', 'numeric text')"); err ≠ nil {
    t.Fatalf("strict rejected numeric text in an integer column: %v", err)
  }
  if _, err := db.Exec(
    "insert into ledger(amount_cents, note) values ('not a number', 'x')"); err == nil {
    t.Fatal("strict table accepted non numeric text for an integer column")
  }
}
```

listing 30: strict rejects what affinity would have coerced, with one nuance pinned (source: infrastructure/samples/ch09/features_test.go, lines 235-254)

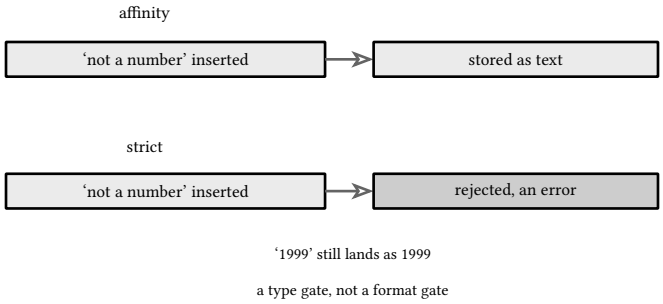


figure 28: strict against affinity, same inserts, opposite outcomes

The nuance is in the test on purpose: strict integer columns still convert text that looks like an integer, `'1999'` lands as 1999. The gate is on type families, not on string formatting, and a probe that claimed otherwise would be describing a different database.

NOTE: why assert the absences

The build that links into this book's tests is fixed, but the one that links into a reader's next job is not. Tests that pin rejected syntax, `SET DEFAULT`, expression `REINDEX`, document the boundary of the feature set as measured, so upgrading the pin in book 13, ch 1 (infrastructure: docker, databases, and testing, the toolchain: docker on windows) reruns these probes and any new capability shows up as a deliberate test change, not a silent drift.

sources: sqlite.org changes for the 3.53.x timeline, json1, lang/ altertable, lang/reindex, pragma, and stricttables pages, all accessed 2026-09-10 and probed against the pinned 3.53.4 binary the same day. Verified by the ch09 probe suite, 8 tests, green under `make verify` 2026-09-10.

10. sqlite in production: embedded kvs and game saves

“Sqlite in production” draws a knowing smile from people who have only met it as the thing under their phone. The honest position is narrower and stronger: sqlite is the answer whenever the data belongs to one machine, one writer, one process family, and it is a liability the moment that stops being true. Four capstones across this corpus make the argument concrete, this book’s chat file the fourth to ship a sqlite file.

10.1. three stores, one engine, three shapes

The raft store in the patterns book, book 12, ch 16 (design patterns, concurrency, and distributed systems, capstone: raft replicated configuration service), persists the consensus triple, term, vote, and log, as two tables over a wal file with `synchronous=full`, because a replicated log’s promise is “the acknowledged entry survives the crash”, and the store opens with the same `busy_timeout` retry dance this book’s `store.Open` uses. The interesting design choice is honesty about write shape: the log can truncate and diverge, so the store rewrites the log table whole inside one transaction instead of maintaining a mirroring tail that can drift.

The game save store, book 15, ch 12 (game systems and architecture, serialization and save systems), writes the whole campaign state, sheet, pets, loadout, upgrades, counters, into six tables in one transaction, drops and recreates on every save, and opens read-only for loads. A save file wants snapshot semantics, not incremental mutation, and the transaction gives the atomicity the temp-file rename trick used to provide. The profile seed rides along so a loaded run draws the same futures a never-saved run would have, determinism preserved across persistence.

And the dsa capstone, book 10, ch 44 (practical data structures and algorithms, capstone: storage engine toolkit), uses sqlite as the reference implementation in a comparator suite: the lsm engine and sqlite loaded with the same deterministic keyspace must answer identically on every key and every negative, which makes the database an executable specification for a storage engine, the most demanding use of the three.

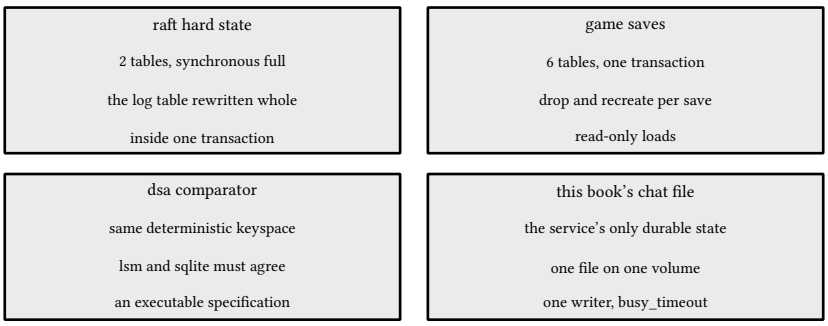


figure 29: one engine, four production shapes

10.2. the embedded key value shape

Stripped of sql, sqlite is a b-tree keyed store with transactions, and a `create table kv(k text primary key, v blob) without rowid` table over it is a durable kv with single-digit-millisecond writes and crash safety for free. The raft hard state table is exactly this shape, two rows wide. The rules that make the shape production-grade are the ones this book already built:

```
wal on open, one writer process, busy_timeout for the
administrative second writer, vacuum into for backups, and the
crash probe from sqlite-wal in the test suite so the guarantee is
asserted, not assumed.
```

fragment, illustrative, not executed

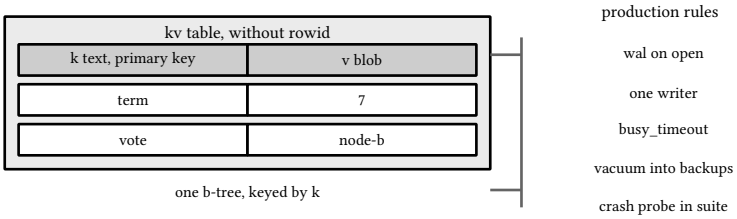


figure 30: a without-rowid table is a durable kv, two rows wide in the raft case

10.3. where it ends

The boundary is equally plain. One machine: the file is a file, and nothing in it survives the machine's disk dying, so durability beyond the machine is replication, which is why the raft store exists and why it does not try to be a networked sql server. One writer at a time: wal lets readers pile on, but writes serialize, and a second writer arriving over the network is a sign the design wanted a client-server database. And no nfs, no cloud-synced folder, nothing that breaks the locking semantics the wal chapter quietly depends on.

The analytical workload is the fourth crossing, and it stays embedded. When the questions stop being point lookups and become scans and aggregates over the same data, the answer is still not a server, it is the other embedded engine: book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine) draws the decision table, and the capstone runs both engines side by side, each behind its own service and its own file.

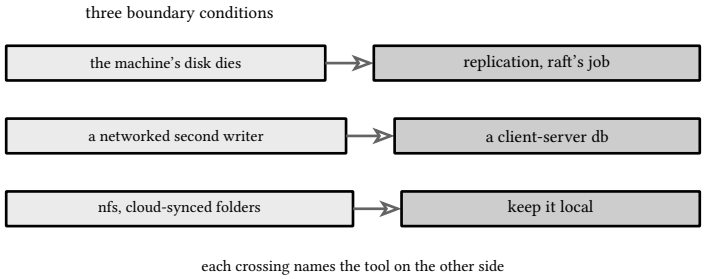


figure 31: where sqlite ends, and what owns the far side of each line

NOTE: the register's claim, graded

The analysis book's employment claims track where the resume says sqlite and the repos say sqlite, and the embedded stores in this chapter's three cross references are the evidence: a replicated log, a game save, and an engine comparator are three different production loads on the same library, each with the failure modes probed rather than assumed. The capstone adds the fourth: a service's only durable state, one file on one volume.

sources: sqlite.org wal.html and the crash semantics chapters behind the probes already cited, accessed 2026-09-10; the three cross referenced stores live in books 10, 11, and 14 and are tested by their own suites under `make verify`. Verified by the capstone store suite, 4 tests, and the ch07 and ch08 probes, green 2026-09-10.

11. duckdb: the embedded analytical engine

The sqlite half of this book, chapters 6 through 10, answered the transactional question: many small reads and writes, one writer at a time, correctness measured per transaction. This chapter opens the analytical half with the engine this repo pins for it, duckdb 1.5.5, released 2026-07-22 and the current stable 1.x, with 1.4.5 carrying the long term support line. Like sqlite it is embedded, one library inside the process, no server and no wire protocol. Unlike sqlite its storage is columnar and its execution is vectorized, built for scans and aggregates over millions of rows instead of point access to a handful.

11.1. olap is a different question

An oltp workload touches a few rows and wants each transaction durable. An olap workload touches most of the table and wants a number: revenue per region, messages per hour, distinct senders per room. The queries are fewer and heavier, and the storage layout that serves them is not the one sqlite chose.

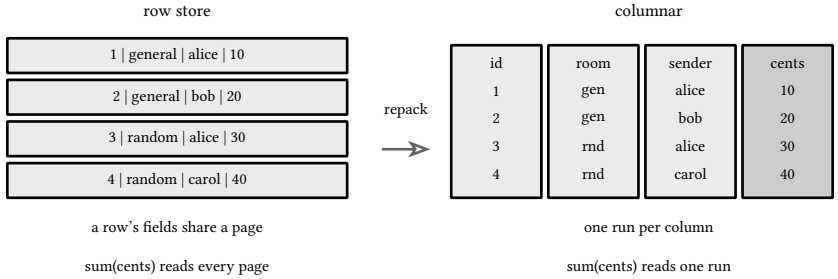


figure 32: the same four rows, row store pages against columnar runs

A row store keeps a row's fields together on the page, so summing one column means reading every page of the table to collect a few values from each. A columnar store keeps each column's values together, so the same sum reads one contiguous run and never loads the rest. Vectorized execution finishes the argument: the engine walks columns in fixed size batches through tight loops over a single type, which is why a group by over two million rows of a two column file answers in milliseconds. The trade sits on the other side of the ledger: touching whole rows one at a time, the oltp shape, means reassembling them from column runs, which is why this book carries both engines instead of replacing one with the other.

11.2. embedded from go, pinned

The driver is `github.com/duckdb/duckdb-go/v2` at tag `v2.10505.0`, and the tag encodes the engine: 10505 is 1.5.5. It registers the `database/sql` driver name `duckdb`, takes a DSN like `store.duckdb?threads=2` or, for a read only seat at a file somebody else owns, `store.duckdb?threads=2&access_mode=read_only`, and speaks the same Query, Exec, and placeholder api as every other driver in this book.

One platform fact shapes the sample lane. The driver's prebuilt static engine libraries are MSVC objects, so on windows the probes ride the `duckdb_use_lib` build tag and link the pinned official dll from `make duckdb-tools`, with zig as the cgo compiler because it speaks gcc flags while emitting msvc objects. On linux, which is what the analytics docker image in the capstone builds, the default static bindings compile with no tag and no dll. The plain `go test ./...` lane compiles none of it, so the compiler-free property the toolchain chapter claims survives everywhere except the deliberate duckdb lanes.

The first probe is the pin itself, the same discipline the sqlite chapters apply to `sqlite3_libversion`:

```
// The engine version pin, exact string: the same dll the c lane and
// the cli lane link, so a version drift breaks here first.
func TestEngineVersion(t *testing.T) {
  db := open(t, "")
  var version string
  if err := db.QueryRow("select version()").Scan(&version); err ≠ nil {
    t.Fatal(err)
  }
  if version ≠ "v1.5.5" {
    t.Fatalf("version() = %s, want v1.5.5", version)
  }
  t.Logf("version() = %s", version)
}
```

listing 31: the engine version pin, exact string, the same dll the cli lane links (source: infrastructure/samples/ch11/duckdb_test.go, lines 42-54)

`make duckdb-tools` also installs the pinned `duckdb.exe`, and the cli is the interactive face of the same engine: point it at a file, describe it, summarize it, pipe a script's csv output somewhere else. The cli gets its own probe in the next chapter, and the version string it prints when a script asks is asserted there too, so the dll, the driver, and the cli cannot drift apart quietly.

11.3. parquet in, parquet out

Parquet is the columnar file format the analytical world already agreed on, and duckdb treats it as a first class citizen: `COPY` any query to a parquet file, read one back with `read_parquet`, and never hand marshal a row. The round trip probe writes four rows with a stated compression and reads them back:

```
func TestParquetRoundTrip(t *testing.T) {
  dir := t.TempDir()
  p := pathOf(dir, "events.parquet")
  db := open(t, "")
  exec(t, db, `create table events(
    id bigint,
    kind varchar,
    ts timestamp,
    amount double
  )`)
  // values chosen so float64 comparison stays exact: halves only
  exec(t, db, `insert into events values
    (1, 'join',   timestamp '2026-01-02 03:04:05', 1.5),
    (2, 'msg',    timestamp '2026-02-03 04:05:06', -2.25),
    (3, 'leave',  timestamp '2026-03-04 05:06:07', 10.0),
    (4, 'msg',    timestamp '2026-04-05 06:07:08', 0.0)`)

  // zstd: the columnar compression this corpus pins for parquet output
  exec(t, db, `copy events to '`+p+`' (FORMAT parquet, COMPRESSION zstd)`)

  rows, err := db.Query(`select id, kind, ts::varchar, amount
    from read_parquet('` + p + `') order by id`)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer rows.Close()
  want := []struct {
    id     int64
```

```
    kind   string
    ts     string
    amount float64
  }{
    {1, "join", "2026-01-02 03:04:05", 1.5},
    {2, "msg", "2026-02-03 04:05:06", -2.25},
    {3, "leave", "2026-03-04 05:06:07", 10.0},
    {4, "msg", "2026-04-05 06:07:08", 0.0},
  }
  for rows.Next() {
    var id int64
    var kind, ts string
    var amount float64
    if err := rows.Scan(&id, &kind, &ts, &amount); err ≠ nil {
      t.Fatal(err)
    }
    w := want[id-1]
    if kind ≠ w.kind || ts ≠ w.ts || amount ≠ w.amount {
      t.Fatalf("row %d = (%s, %s, %v), want (%s, %s, %v)",
        id, kind, ts, amount, w.kind, w.ts, w.amount)
    }
  }
  if err := rows.Err(); err ≠ nil {
    t.Fatal(err)
  }
  var n int
  if err := db.QueryRow(`select count(*) from read_parquet('` + p + `')`).Scan(&n); err ≠ nil {
    t.Fatal(err)
  }
  if n ≠ len(want) {
    t.Fatalf("parquet rows = %d, want %d", n, len(want))
  }
}
```

listing 32: copy to parquet with zstd, read back with read_parquet, compared row for row (source: infrastructure/samples/ch11/parquet_test.go, lines 13-73)

Two details in that probe earn their lines. The compression is stated, `COMPRESSION zstd`, because the columnar win is half layout and half encoding, and this corpus pins the encoder instead of trusting a default. And the timestamp is cast to varchar on the way out: duckdb timestamp columns scan into go as `time.Time`, so a probe that wants a plain string assert casts inside the query rather than formatting in the test.

The write side has a partitioned form that changes where a column lives. `PARTITION_BY` writes one subdirectory per key value and drops the partition column from the files, and `hive_partitioning` reads the directory names back as a column, so the round trip restores what the write removed:

```
// PARTITION_BY writes one subdirectory per key value and drops the
// column from the files; hive_partitioning reads the directory names
// back as a column, so the round trip restores what the write removed.
func TestParquetPartitionedByHive(t *testing.T) {
  dir := t.TempDir()
  root := pathOf(dir, "parts")
  db := open(t, "")
  // 10 ids split over 3 groups: g = id % 3
  exec(t, db, `copy (select i as id, i % 3 as g from range(10) t(i))
    to '`+root+`' (FORMAT parquet, PARTITION_BY (g), COMPRESSION zstd)`)

  rows, err := db.Query(`select id, g
    from read_parquet('` + root + `/**/*.parquet', hive_partitioning = true)
    order by id`)
  if err != nil {
    t.Fatal(err)
  }
  defer rows.Close()
  seen := map[int64]int64{}
  for rows.Next() {
    var id, g int64
    if err := rows.Scan(&id, &g); err != nil {
      t.Fatal(err)
    }
    seen[id] = g
  }
  if err := rows.Err(); err != nil {
    t.Fatal(err)
  }
  if len(seen) != 10 {
    t.Fatalf("hive read returned %d rows, want 10", len(seen))
  }
  for id, g := range seen {
    if want := int64(id) % 3; g != want {
      t.Fatalf("id %d mapped to partition %d, want %d", id, g, want)
    }
  }

  // the partition column lives only in the path: one directory per
  // distinct value, values recoverable without reading a file
  var dirs int
  if err := db.QueryRow(`select count(distinct g)
    from read_parquet('` + root + `/**/*.parquet', hive_partitioning = true)`).
    Scan(&dirs); err != nil {
    t.Fatal(err)
  }
  if dirs != 3 {
    t.Fatalf("distinct partitions = %d, want 3", dirs)
  }
}
```

listing 33: partition by writes g into the path, hive partitioning reads it back (source: infrastructure/samples/ch11/parquet_test.go, lines 75-124)

The partition column ends up stored exactly once per directory name, which is what makes partition pruning work: a query filtering on `g` can skip whole directories without opening a file. The probe asserts the recovered mapping and that the three distinct values produced three directories.

11.4. csv, sniffed then pinned

Real analytical data arrives as csv, and duckdb's answer is the sniffer: `read_csv_auto` infers delimiter, quoting, and types from the file itself, and the inference is assertable because `describe` works over the sniffed relation:

```
func TestCsvAutoDetectInfersTypes(t *testing.T) {
  dir := t.TempDir()
  p := pathOf(dir, "people.csv")
  csv := `id,name,hired,note
1,"smith, john",2026-01-02,"said ""hi"" twice"
2,alice,2026-02-03,
3,"lee; ann",2026-03-04,ok
` + "\n"
  if err := os.WriteFile(filepath.Join(dir, "people.csv"), []byte(csv), 0o644); err ≠ nil {
    t.Fatal(err)
  }
  db := open(t, "")

  rows, err := db.Query(`describe select * from read_csv_auto('` + p + `')`)
  if err ≠ nil {
    t.Fatal(err)
  }
  cols := map[string]string{}
  for rows.Next() {
    var name, typ, null string
    var key, dflt, extra any
    if err := rows.Scan(&name, &typ, &null, &key, &dflt, &extra); err ≠ nil {
      t.Fatal(err)
    }
    cols[name] = typ
  }
  rows.Close()
  if err := rows.Err(); err ≠ nil {
    t.Fatal(err)
  }
  for name, want := range map[string]string{
    "id":    "BIGINT",
    "name":  "VARCHAR",
    "hired": "DATE",
    "note":  "VARCHAR",
  } {
    if cols[name] ≠ want {
      t.Fatalf("column %s inferred as %s, want %s", name, cols[name], want)
    }
  }

  // quoted commas and escaped quotes survive, and the trailing empty
  // field scanned as null, not as the empty string
  var name, note string
  var nulls int
  if err := db.QueryRow(`select name from read_csv_auto('` + p + `') where id = 1`).
    Scan(&name); err ≠ nil {
    t.Fatal(err)
  }
  if name ≠ `smith, john` {
    t.Fatalf("quoted comma field = %q", name)
  }
```

```
  if err := db.QueryRow(`select note from read_csv_auto('` + p + `') where id = 1`).
    Scan(&note); err ≠ nil {
    t.Fatal(err)
  }
  if note ≠ `said "hi" twice` {
    t.Fatalf("escaped quote field = %q", note)
  }
  if err := db.QueryRow(`select count(*) from read_csv_auto('` + p + `')
    where note is null`).Scan(&nulls); err ≠ nil {
    t.Fatal(err)
  }
  if nulls ≠ 1 {
    t.Fatalf("null note rows = %d, want 1", nulls)
  }
}
```

listing 34: the sniffer infers bigint, varchar, date, and the quoted and null cases survive (source: infrastructure/samples/ch11/csv_test.go, lines 15-81)

The inferred types are real: `id` comes back BIGINT, `hired` comes back DATE, quoted commas and doubled quotes survive, and a trailing empty field reads as null rather than the empty string. The sniffer is a guess, though, and the second probe is the file where it guesses wrong:

```
// The faulty csv: semicolon fields, single-quoted text, backslash-N for
// null. The sniffer finds the delimiter but keeps score as varchar
// holding the literal text \N, because nothing told it what null looks
// like. Explicit delim, quote, and nullstr pin the parse and the column
// goes back to bigint with real nulls.
func TestCsvExplicitOptionsFixTheParse(t *testing.T) {
  dir := t.TempDir()
  p := pathOf(dir, "scores.csv")
  csv := `id;label;score
1;'smith; john';\N
2;alice;10
3;bob;\N
` + "\n"
  if err := os.WriteFile(filepath.Join(dir, "scores.csv"), []byte(csv), 0o644); err ≠ nil {
    t.Fatal(err)
  }
  db := open(t, "")

  var sniffed string
  if err := db.QueryRow(`select column_type from (describe select * from read_csv_auto('` +
    p + `')) where column_name = 'score'`).Scan(&sniffed); err ≠ nil {
    t.Fatal(err)
  }
  if sniffed ≠ "VARCHAR" {
    t.Fatalf("sniffed score type = %s, want VARCHAR (the sniff is supposed to be wrong here)", sniffed)
  }
  var literal int
  if err := db.QueryRow(`select count(*) from read_csv_auto('` + p + `')
    where score = '\N'`).Scan(&literal); err ≠ nil {
    t.Fatal(err)
  }
  if literal ≠ 2 {
    t.Fatalf("literal backslash-N rows = %d, want 2", literal)
  }
```

```
  // the fix: three options, no sniffing
  fixed := `read_csv('` + p + `', delim = ';', quote = '''', nullstr = '\N')`
  var typed string
  if err := db.QueryRow(`select column_type from (describe select * from ` + fixed + `)
    where column_name = 'score'`).Scan(&typed); err ≠ nil {
    t.Fatal(err)
  }
  if typed ≠ "BIGINT" {
    t.Fatalf("fixed score type = %s, want BIGINT", typed)
  }
  var label string
  var nulls, total int64
  if err := db.QueryRow(`select label from ` + fixed + ` where id = 1`).Scan(&label); err ≠ nil {
    t.Fatal(err)
  }
  if label ≠ "smith; john" {
    t.Fatalf("single-quoted semicolon field = %q", label)
  }
  if err := db.QueryRow(`select count(*), count(score) from `+fixed).Scan(&total, &nulls); err ≠ nil {
    t.Fatal(err)
  }
  if total ≠ 3 || nulls ≠ 1 {
    t.Fatalf("rows = %d, non-null scores = %d, want 3 and 1", total, nulls)
  }
}
```

listing 35: semicolons, single quotes, and backslash-N: the sniffer keeps score as varchar holding the literal text (source: infrastructure/samples/ch11/csv_test.go, lines 83-142)

The sniffer finds the delimiter but nothing told it what null looks like, so `score` stays VARCHAR holding the literal text `\N`, and the probe asserts the wrongness before fixing it. The fix is three explicit options, `delim`, `quote`, and `nullstr`, and the column goes back to BIGINT with real nulls. The rule the pair of probes encodes: sniff first, describe the result, and pin options the moment the sniff is wrong, because a silent VARCHAR where a number should be poisons every downstream aggregate.

11.5. scans that never load the file

The claim that separates duckdb from an in-memory table is that the file is the table. The scan probe never creates one: it generates two million rows with `range()`, writes them once as parquet, and then groups and sums straight off disk:

```
func TestLargeFileScanAggregates(t *testing.T) {
  const rows = 2_000_000
  dir := t.TempDir()
  p := pathOf(dir, "big.parquet")
  db := open(t, "")

  start := time.Now()
  exec(t, db, `copy (select i as id, i % 10 as g from range(`+strconv.Itoa(rows)+`) t(i))
    to '`+p+`' (FORMAT parquet, COMPRESSION zstd)`)
  genDur := time.Since(start)

  start = time.Now()
  type group struct {
    g    int64
```

```
    n    int64
    sum  int64
    want int64
  }
  // ids g, g+10, ..., below 2,000,000: 200000 rows per group,
  // sum = 200000*g + 10 * sum(0..199999) = 200000*g + 199999000000
  var groups []group
  rowsQ, err := db.Query(`select g, count(*), sum(id)
    from read_parquet('` + p + `') group by g order by g`)
  if err ≠ nil {
    t.Fatal(err)
  }
  for rowsQ.Next() {
    var gr group
    if err := rowsQ.Scan(&gr.g, &gr.n, &gr.sum); err ≠ nil {
      t.Fatal(err)
    }
    gr.want = 200000*gr.g + 199_999_000_000
    groups = append(groups, gr)
  }
  if err := rowsQ.Err(); err ≠ nil {
    t.Fatal(err)
  }
  rowsQ.Close()
  scanDur := time.Since(start)

  if len(groups) ≠ 10 {
    t.Fatalf("groups = %d, want 10", len(groups))
  }
  var total int64
  for _, gr := range groups {
    if gr.n ≠ 200000 {
      t.Fatalf("group %d has %d rows, want 200000", gr.g, gr.n)
    }
    if gr.sum ≠ gr.want {
      t.Fatalf("group %d sums to %d, want %d", gr.g, gr.sum, gr.want)
    }
    total += gr.n
  }
  if total ≠ rows {
    t.Fatalf("scanned %d rows, want %d", total, rows)
  }
  t.Logf("%d rows: generate %v, scan+group %v (file-backed, no table loaded)",
    rows, genDur, scanDur)
}
```

listing 36: two million rows, no table ever created, the aggregate streams the file (source: infrastructure/samples/ch11/scan_test.go, lines 15-73)

On this machine the 2 million row parquet file writes in 121ms and the grouped scan answers in 17.6ms, and the engine is free to spill to temp files when a query's working set exceeds memory, which is the out-of-core behavior the chapter title promises. The correctness half is the honest half: the per group sums are checked against closed form arithmetic, `200000*g + 199999000000`, computed in the test and nowhere near the engine, so a fast wrong answer still fails.

11.6. meeting the data: describe and summarize

Exploration is a query genre of its own. DESCRIBE reports the column shape of any relation, and SUMMARIZE scans the values and reports min, max, average, count, and null share per column. Both are plain result sets, which makes them assertable, and SUMMARIZE works as a subquery with cells castable to varchar:

```
// summarize over known data: min, max, avg, count, null_percentage are
// cells in a result set, computed by one scan of the table.
func TestSummarizeReportsValueShape(t *testing.T) {
  db := open(t, "")
  exec(t, db, `create table t(a bigint, b varchar)`)
  exec(t, db, `insert into t values (2,'y'), (1,'x'), (5,'z'), (4,NULL)`)

  rows, err := db.Query(`select column_name, min::varchar, max::varchar, avg, count::varchar,
    null_percentage::varchar from (summarize t) order by column_name`)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer rows.Close()
  got := map[string][5]string{}
  for rows.Next() {
    var name, minCell, maxCell, countCell, pctCell string
    var avg sql.Null[string]
    if err := rows.Scan(&name, &minCell, &maxCell, &avg, &countCell, &pctCell); err ≠ nil {
      t.Fatal(err)
    }
    avgCell := ""
    if avg.Valid {
      avgCell = avg.V
    }
    got[name] = [5]string{minCell, maxCell, avgCell, countCell, pctCell}
  }
  if err := rows.Err(); err ≠ nil {
    t.Fatal(err)
  }
  for name, want := range map[string][5]string{
    "a": {"1", "5", "3.0", "4", "0.00"},
    "b": {"x", "z", "", "4", "25.00"},
  } {
    if got[name] ≠ want {
      t.Fatalf("summarize %s = %v, want %v", name, got[name], want)
    }
  }
}
```

listing 37: summarize as a subquery: min, max, avg, count, and the 25 percent null share on column b (source: infrastructure/samples/ch11/explore_test.go, lines 57-94)

Four rows of known data, and the probe checks the whole row of statistics against hand computed values, including the empty average cell for the varchar column with a null. The habit this probe models costs nothing on real files: point SUMMARIZE at a parquet path you just received and the shape of the data, types, ranges, and ragged null shares, is one query away.

11.7. the appender, the fast way in

Inserts are the slow path into a columnar engine because every statement pays parse, plan, and row reassembly. The appender is the fast path: it streams go values straight into columnar storage, reached through the driver as `sql.Conn.Raw` to a `*duckdb.Conn`, then `NewAppenderFromConn`. The probe loads the same 200,000 rows both ways:

```
func TestAppenderVersusBatchedExec(t *testing.T) {
  db := open(t, "")
  exec(t, db, `create table via_appender(a bigint, b varchar, c double)`)
  exec(t, db, `create table via_exec(a bigint, b varchar, c double)`)

  // the appender: one raw connection, rows appended then flushed
  conn, err := db.Conn(context.Background())
  if err ≠ nil {
    t.Fatal(err)
  }
  defer conn.Close()
  start := time.Now()
  var ap *duckdb.Appender
  if err := conn.Raw(func(dc any) error {
    var err error
    ap, err = duckdb.NewAppenderFromConn(dc.(driver.Conn), "", "via_appender")
    return err
  }); err ≠ nil {
    t.Fatal(err)
  }
  for i := 0; i < loadRows; i++ {
    a, b, c := rowTriple(i)
    if err := ap.AppendRow(a, b, c); err ≠ nil {
      t.Fatalf("append row %d: %v", i, err)
    }
  }
  if err := ap.Flush(); err ≠ nil {
    t.Fatal(err)
  }
  if err := ap.Close(); err ≠ nil {
    t.Fatal(err)
  }
  appenderDur := time.Since(start)
```

listing 38: the appender: one raw connection, rows appended, one flush (source: infrastructure/samples/ch11/load_test.go, lines 57-89)

```
  // batched exec: one multi-row insert statement per batch
  start = time.Now()
  var sb strings.Builder
  args := make([]any, 0, batchRows*3)
  for base := 0; base < loadRows; base += batchRows {
    sb.Reset()
    sb.WriteString("insert into via_exec values ")
    args = args[:0]
    for i := 0; i < batchRows; i++ {
      if i > 0 {
        sb.WriteByte(',')
      }
      sb.WriteString("(?, ?, ?)")
      a, b, c := rowTriple(base + i)
      args = append(args, a, b, c)
    }
    if _, err := db.Exec(sb.String(), args...); err != nil {
      t.Fatalf("batch at %d: %v", base, err)
    }
  }
  execDur := time.Since(start)

  // correctness: same row count, same checksum, both matching the
  // values computed in the test
  var appN, execN int
  if err := db.QueryRow("select count(*) from via_appender").Scan(&appN); err != nil {
    t.Fatal(err)
  }
  if err := db.QueryRow("select count(*) from via_exec").Scan(&execN); err != nil {
    t.Fatal(err)
  }
  if appN != loadRows || execN != loadRows {
    t.Fatalf("rows: appender %d, exec %d, want %d", appN, execN, loadRows)
  }
  wSum, wMax, wC := wantSum()
  aSum, aMax, aC := sumTriple(t, db, "via_appender")
  eSum, eMax, eC := sumTriple(t, db, "via_exec")
  if aSum != eSum || aMax != eMax || aC != eC {
    t.Fatalf("loads disagree: appender (%d, %s, %v) exec (%d, %s, %v)",
      aSum, aMax, aC, eSum, eMax, eC)
  }
  if aSum != wSum || aMax != wMax || aC != wC {
    t.Fatalf("loaded checksum (%d, %s, %v) != computed (%d, %s, %v)",
      aSum, aMax, aC, wSum, wMax, wC)
  }

  t.Logf("appender: %d rows in %v (%.0f rows/s)",
    appN, appenderDur, float64(appN)/appenderDur.Seconds())
  t.Logf("batched exec (%d-row inserts): %d rows in %v (%.0f rows/s)",
    batchRows, execN, execDur, float64(execN)/execDur.Seconds())
  t.Logf("appender advantage: %.0fx", execDur.Seconds()/appenderDur.Seconds())
}
```

listing 39: the same rows through 1,000 row batched Exec, then checksums against values computed in the test (source: infrastructure/samples/ch11/load_test.go, lines 91-142)

Measured on this machine: the appender loads 200,000 rows in 71ms, 2,809,297 rows per second, against 58,895 rows per second for the same rows through 1,000 row batched Exec statements, about 48 times. Both loads must

checksum identically, against each other and against the sums the test computes itself, so the fast path is proven to load the same data, not just load something quickly. The batched lane is not a strawman either: 1,000 row multi value inserts are what a reasonable `database/sql` program does, and the number is the size of the gap the appender closes.

11.8. mvcc, readers that never wait

The busy ladder chapter, book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation), built sqlite's side: in rollback mode a writer's commit parks behind any reader holding a shared lock, and `busy_timeout` is the knob that turns the failure into a bounded wait. Duckdb's file mvcc is the deliberate opposite. One writer and any number of readers coexist without blocking, because readers read a snapshot and the writer appends versions nothing points at yet. The probe holds a write transaction open on one connection and reads through a second the whole time:

```
// The mvcc contrast. Chapter 7 built sqlite's busy ladder: in rollback
// mode a writer's commit parks behind any reader holding a shared
// lock, and busy_timeout is the knob that turns the failure into a
// wait. Duckdb's file mvcc is the deliberate opposite: one writer and
// any number of readers coexist without blocking, because readers read
// a snapshot and the writer appends versions nothing points at yet.
// The probe holds a write transaction open on one connection and reads
// through a second connection the whole time; nothing waits.
package ch11

import (
  "testing"
  "time"
)

func TestReaderSnapshotWhileWriterHoldsTransaction(t *testing.T) {
  p := pathOf(t.TempDir(), "mvcc.duckdb")
  writer := open(t, p)
  reader := open(t, p)
  exec(t, writer, "create table t(x bigint)")
  exec(t, writer, "insert into t values (1)")

  tx, err := writer.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  if _, err := tx.Exec("insert into t values (2)"); err ≠ nil {
    t.Fatal(err)
  }
  // the writer's own view inside its transaction
  var wmid int
  if err := tx.QueryRow("select count(*) from t").Scan(&wmid); err ≠ nil {
    t.Fatal(err)
  }
  if wmid ≠ 2 {
    t.Fatalf("writer in tx sees %d rows, want 2", wmid)
  }

  // the reader, same file, while the write transaction is open: no
  // error, no block, and the pre-transaction snapshot
  start := time.Now()
  var rmid int
```

```
  if err := reader.QueryRow("select count(*) from t").Scan(&rmid); err ≠ nil {
    t.Fatalf("read during open write tx: %v", err)
  }
  readDur := time.Since(start)
  if rmid ≠ 1 {
    t.Fatalf("reader saw %d rows during the tx, want the snapshot's 1", rmid)
  }
  t.Logf("concurrent read returned in %v with the tx open", readDur)

  // a reader transaction opened now keeps seeing that snapshot even
  // after the writer commits, the same repeatable read ch07 pinned
  readTx, err := reader.Begin()
  if err ≠ nil {
    t.Fatal(err)
  }
  if err := readTx.QueryRow("select count(*) from t").Scan(new(int)); err ≠ nil {
    t.Fatal(err)
  }
  if err := tx.Commit(); err ≠ nil {
    t.Fatal(err)
  }
  var inSnap int
  if err := readTx.QueryRow("select count(*) from t").Scan(&inSnap); err ≠ nil {
    t.Fatal(err)
  }
  if inSnap ≠ 1 {
    t.Fatalf("reader tx saw %d rows after the commit, want its snapshot's 1", inSnap)
  }
  if err := readTx.Rollback(); err ≠ nil {
    t.Fatal(err)
  }

  // a fresh read on the same reader connection now sees the commit
  var after int
  if err := reader.QueryRow("select count(*) from t").Scan(&after); err ≠ nil {
    t.Fatal(err)
  }
  if after ≠ 2 {
    t.Fatalf("fresh read after commit saw %d rows, want 2", after)
  }
}
```

listing 40: the writer's transaction stays open, the reader reads, nothing parks (source: infrastructure/samples/ch11/mvcc_test.go, lines 3-85)

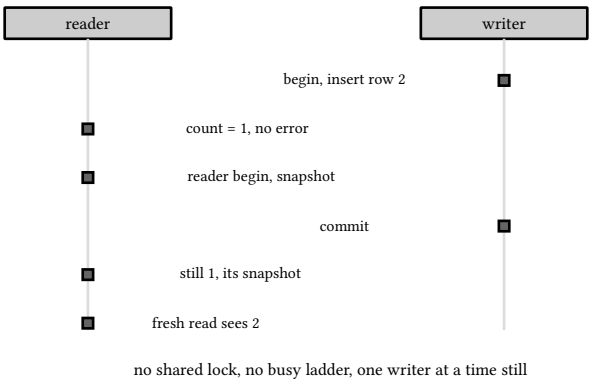


figure 33: the same interleaving the busy ladder chapter timed, here with no wait anywhere

Three assertions land in order. The reader on a second connection sees the pre transaction snapshot while the write transaction is open, with no error and no wait. A reader transaction started before the commit keeps seeing its snapshot after it, the same repeatable read the sqlite probe pinned, just without the ladder underneath. And a fresh read on the same reader connection sees the commit. Writers still serialize with each other, one writer at a time is as true here as it is in sqlite, so the difference is narrower than the marketing: it is readers against writers, which for an analytical workload, many long scans beside an occasional load, is exactly the contention that matters.

NOTE: the version pin fails loudly, in three places

The go module is pinned by tag `v2.10505.0`, whose number encodes the engine: 10505 is 1.5.5. The probe asserts `select version()` returns exactly `v1.5.5`, the cli script lane in the next chapter asserts the same string from the same pinned binary, and the c row cruncher in the c-os-cloud book links the same dll from the same `make duckdb-tools` output. When any leg drifts, one exact string check breaks first, which is the cheap end of the failure. The next chapter turns this into a re-pin checklist for the v2.0 preview already on the horizon.

sources: duckdb.org documentation for the parquet import and export page, the csv auto-detection page, and the c api appender page, install.duckdb.org for the 1.5.5 binaries, and pkg.go.dev for the duckdb-go v2 driver, all accessed 2026-09-20. Verified by the ch11 probe suite, 10 tests behind the `duckdb_use_lib` tag against the pinned 1.5.5 dll with zig as the cgo compiler, green under `make verify-go` 2026-09-20.

12. duckdb and sqlite together: choosing the engine

This book now embeds two engines, and the interesting question is which workload each one owns. This chapter runs both in one process, attached to each other through duckdb's sqlite extension, pins what crosses the boundary and what breaks on it, proves the two engines agree on this repo's real schema, and closes with the selection rule the capstone follows and the v2.0 preview already visible on the horizon.

12.1. attach, two engines in one process

Duckdb speaks sqlite through an extension, and the whole integration is two statements:

```
INSTALL sqlite;
ATTACH 'chat.db' AS s (TYPE sqlite);
```

fragment, illustrative, not executed

`make duckdb-tools` warms the extension into the home cache so the install is a no-op offline, and the attach gives full read and write access to the file, transactions included, with the tables addressable as `s.things`. The samples keep the ownership split deliberate: modernc creates and owns the sqlite files, duckdb attaches them:

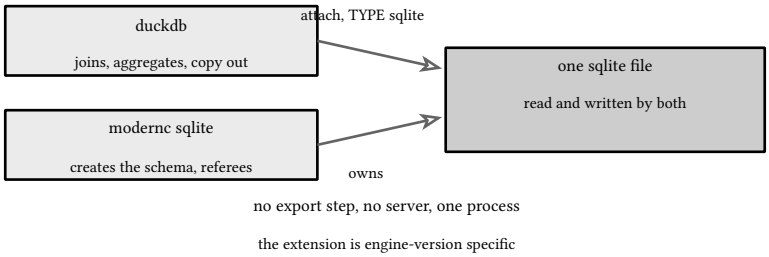


figure 34: one process, two engines, one file, no server in sight

The canonical shape is dimensions in sqlite, facts in duckdb, and one query reading both without either engine copying data to the other by hand:

```
// The join across the engine boundary: dimensions live in the sqlite
// file, facts live in duckdb, and one query reads both without either
// engine copying data to the other by hand.
func TestAttachJoinAcrossEngines(t *testing.T) {
  dir := t.TempDir()
  sp := pathOf(dir, "dims.db")

  regions := openSqlite(t, sp, "pragma foreign_keys=on")
  exec(t, regions, `create table regions(
    code text primary key,
    name text not null)`)
  exec(t, regions, `insert into regions values ('eu', 'europe'), ('na', 'north america')`)
  regions.Close()

  db := openDuck(t)
  attachSqlite(t, db, "s", sp)
  exec(t, db, `create table charges(region varchar, amount bigint)`)
  exec(t, db, `insert into charges values
    ('eu', 100), ('eu', 200), ('na', 50), ('eu', 30), ('na', 70), ('ap', 10)`)

  // ap has no row in sqlite: an inner join drops it, a left join
```

```
  // keeps it with a null name, and both behaviors are assertable
  rows, err := db.Query(`select r.name, count(*) as n, sum(c.amount) as total
    from charges c join s.regions r on c.region = r.code
    group by r.name order by r.name`)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer rows.Close()
  type row struct {
    name  string
    n     int64
    total int64
  }
  var got []row
  for rows.Next() {
    var r row
    if err := rows.Scan(&r.name, &r.n, &r.total); err ≠ nil {
      t.Fatal(err)
    }
    got = append(got, r)
  }
  if err := rows.Err(); err ≠ nil {
    t.Fatal(err)
  }
  want := []row{{"europe", 3, 330}, {"north america", 2, 120}}
  if len(got) ≠ len(want) {
    t.Fatalf("join rows = %d, want %d", len(got), len(want))
  }
  for i := range want {
    if got[i] ≠ want[i] {
      t.Fatalf("row %d = %+v, want %+v", i, got[i], want[i])
    }
  }

  var unmatched int64
  if err := db.QueryRow(`select count(*) from charges c
    left join s.regions r on c.region = r.code
    where r.code is null`).Scan(&unmatched); err ≠ nil {
    t.Fatal(err)
  }
  if unmatched ≠ 1 {
    t.Fatalf("charges with no region = %d, want 1 (the 'ap' row)", unmatched)
  }
}
```

listing 41: the join across the engine boundary, inner drops the unmatched region, left join counts it (source: infrastructure/samples/ch12/attach_test.go, lines 64-128)

The ap charge with no row in sqlite is the join semantics made assertable: an inner join drops it, a left join keeps it with a null name, and the probe checks both.

12.2. writing back through the attach

Reads are half the claim. The attach is a full citizen, duckdb can insert into the sqlite file, and the referee is a fresh modernc connection opened after, one that never talked to duckdb and only reads the file:

```
func TestWriteThroughAttachLandsInSqlite(t *testing.T) {
  dir := t.TempDir()
  sp := pathOf(dir, "notes.db")

  notes := openSqlite(t, sp, "pragma foreign_keys=on")
  exec(t, notes, `create table notes(
    id integer primary key,
    body text not null)`)
  exec(t, notes, `insert into notes(body) values ('first')`)
  notes.Close()

  db := openDuck(t)
  attachSqlite(t, db, "s", sp)
  exec(t, db, `insert into s.notes(body) values ('written by duckdb'), ('also duckdb')`)

  // a fresh modernc connection is the referee: it never talked to
  // duckdb, it only reads the file
  check := openSqlite(t, sp)
  rows, err := check.Query(`select id, body from notes order by id`)
  if err ≠ nil {
    t.Fatal(err)
  }
  defer rows.Close()
  var got []struct {
    id   int64
    body string
  }
  for rows.Next() {
    var r struct {
      id   int64
      body string
    }
    if err := rows.Scan(&r.id, &r.body); err ≠ nil {
      t.Fatal(err)
    }
    got = append(got, r)
  }
  if err := rows.Err(); err ≠ nil {
    t.Fatal(err)
  }
  if len(got) ≠ 3 {
    t.Fatalf("sqlite rows after duckdb writes = %d, want 3", len(got))
  }
  wantBodies := []string{"first", "written by duckdb", "also duckdb"}
  for i, w := range wantBodies {
    if got[i].body ≠ w {
      t.Fatalf("row %d body = %q, want %q", got[i].id, got[i].body, w)
    }
  }

  // the sqlite-assigned rowids continued where modernc left off
  if got[1].id ≠ 2 || got[2].id ≠ 3 {
    t.Fatalf("duckdb-written ids = %d, %d, want 2, 3", got[1].id, got[2].id)
  }
}
```

listing 42: duckdb writes two rows, modernc reads three, and the sqlite rowids continue where modernc left off (source: infrastructure/samples/ch12/writeback_test.go, lines 12-66)

The rowid detail matters more than it looks: duckdb's inserts continue sqlite's `integer primary key` sequence from where the owner left it, so the file's keying discipline survives a second writer engine. The same transactionality the sqlite chapters probed belongs to the file, and both engines see it.

12.3. affinity, loose typing meets strict columns

Sqlite's typing is advisory, `INT` suggests, storage decides, and duckdb's columns are strict, so the attach needs a mapping. The probe pins it:

```
func TestSqliteTypeAffinityMapping(t *testing.T) {
  dir := t.TempDir()
  sp := pathOf(dir, "aff.db")

  aff := openSqlite(t, sp)
  exec(t, aff, `create table typed(a INT, b TEXT, c REAL)`)
  exec(t, aff, `insert into typed values (42, 'x', 1.5)`)
  aff.Close()

  db := openDuck(t)
  attachSqlite(t, db, "s", sp)
  cols := describeAttached(t, db, "s.typed")
  for name, want := range map[string]string{
    "a": "BIGINT",
    "b": "VARCHAR",
    "c": "DOUBLE",
  } {
    if cols[name] ≠ want {
      t.Fatalf("column %s described as %s, want %s", name, cols[name], want)
    }
  }

  // the mapped types are what the scan delivers: int64, string, float64
  var a int64
  var b string
  var c float64
  if err := db.QueryRow(`select * from s.typed`).Scan(&a, &b, &c); err ≠ nil {
    t.Fatal(err)
  }
  if a ≠ 42 || b ≠ "x" || c ≠ 1.5 {
    t.Fatalf("typed row = (%d, %s, %v)", a, b, c)
  }
}
```

listing 43: INT, TEXT, REAL become BIGINT, VARCHAR, DOUBLE, and the scan delivers int64, string, float64 (source: infrastructure/samples/ch12/affinity_test.go, lines 38-70)

The mapped types are not cosmetic, they are what the go scan delivers: int64, string, float64, exactly as if the table had been created in duckdb. The mapping is also where ragged data breaks, because sqlite happily stores text in an int column and duckdb will not:

```
// Ragged data, both outcomes. Default attach: the read of the text
// value in an int column fails, and the error itself names the escape
// hatch. The escape hatch: sqlite_all_varchar set before attaching
// makes the column readable as varchar.
func TestRaggedColumnAndSqliteAllVarchar(t *testing.T) {
  dir := t.TempDir()
```

```
  sp := pathOf(dir, "ragged.db")

  ragged := openSqlite(t, sp)
  exec(t, ragged, `create table mixed(a INT)`)
  // sqlite takes all three: an int, text in an int column, another int
  for _, v := range []any{1, "oops", 3} {
    exec(t, ragged, `insert into mixed values (?)`, v)
  }
  ragged.Close()

  // outcome one: plain attach, the read fails with the conversion
  // error, and duckdb's message names the setting that fixes it
  strict := openDuck(t)
  attachSqlite(t, strict, "s", sp)
  _, err := strict.Query(`select * from s.mixed`)
  if err == nil {
    t.Fatal("ragged int column read succeeded, want the mismatch type error")
  }
  msg := err.Error()
  if !strings.Contains(msg, "Mismatch Type Error") {
    t.Fatalf("ragged read error = %v, want a mismatch type error", err)
  }
  if !strings.Contains(msg, "sqlite_all_varchar") {
    t.Fatalf("error does not name sqlite_all_varchar: %v", err)
  }
  t.Logf("ragged read error: %v", err)

  // outcome two: the setting must come before the attach; every
  // column then reads as varchar and the rows come through as text
  loose := openDuck(t)
  exec(t, loose, `SET GLOBAL sqlite_all_varchar = true`)
  attachSqlite(t, loose, "s2", sp)
  cols := describeAttached(t, loose, "s2.mixed")
  if cols["a"] != "VARCHAR" {
    t.Fatalf("column a under sqlite_all_varchar = %s, want VARCHAR", cols["a"])
  }
  rows, err := loose.Query(`select a from s2.mixed`)
  if err != nil {
    t.Fatalf("read under sqlite_all_varchar: %v", err)
  }
  defer rows.Close()
  var got []string
  for rows.Next() {
    var v string
    if err := rows.Scan(&v); err != nil {
      t.Fatal(err)
    }
    got = append(got, v)
  }
  if err := rows.Err(); err != nil {
    t.Fatal(err)
  }
  want := []string{"1", "oops", "3"}
  if len(got) != len(want) {
    t.Fatalf("rows = %v, want %v", got, want)
  }
  for i := range want {
```

```
    if got[i] != want[i] {
      t.Fatalf("row %d = %q, want %q", i, got[i], want[i])
    }
  }
}
```

listing 44: the ragged column fails with an error that names its own fix, then sqlite_all_varchar reads it as text (source: infrastructure/samples/ch12/affinity_test.go, lines 72-139)

Two outcomes, both asserted. A plain attach fails the read with a Mismatch Type Error whose message names `sqlite_all_varchar`, an error that documents its own escape hatch. And the setting must come before the attach: `SET GLOBAL sqlite_all_varchar = true` on a fresh connection, then attach, and every column reads as varchar, the three rows arriving as the strings `1`, `oops`, `3`. The operational reading: when a legacy file is ragged, take it as text and cast the columns you trust, one at a time, where the values are used.

12.4. one statement to parquet

The archival path for a live operational database is a single `COPY` through the attach, sqlite table in, parquet file out:

```
func TestCopyAttachedSqliteTableToParquet(t *testing.T) {
  dir := t.TempDir()
  sp := pathOf(dir, "ledger.db")
  pp := pathOf(dir, "ledger.parquet")

  ledger := openSqlite(t, sp, "pragma foreign_keys=on")
  exec(t, ledger, `create table entries(
    id integer primary key,
    account text not null,
    cents integer not null,
    ts integer not null)`)
  for i, e := range []struct {
    account string
    cents   int64
    ts      int64
  }{
    {"cash", 1500, 100},
    {"cash", -200, 200},
    {"bank", 9900, 300},
    {"bank", -900, 400},
    {"cash", 75, 500},
  } {
    exec(t, ledger, `insert into entries(id, account, cents, ts) values (?, ?, ?, ?)`,
      i+1, e.account, e.cents, e.ts)
  }
  ledger.Close()

  db := openDuck(t)
  attachSqlite(t, db, "s", sp)
  exec(t, db, `copy s.entries to '`+pp+`' (FORMAT parquet, COMPRESSION zstd)`)
```

listing 45: the ledger built by modernc, then one copy statement through the attach (source: infrastructure/samples/ch12/copyout_test.go, lines 12-41)

The probe reads the parquet back row for row against a referee read straight from sqlite after the copy, and the two row sets must match exactly. Then the part sqlite never could do on its own:

```
  // the referee read straight from sqlite, after the copy
  check := openSqlite(t, sp)
  sqlRows, err := check.Query(`select id, account, cents, ts from entries order by id`)
  if err ≠ nil {
    t.Fatal(err)
  }
  var fromSqlite []entry
  for sqlRows.Next() {
    var e entry
    if err := sqlRows.Scan(&e.id, &e.account, &e.cents, &e.ts); err ≠ nil {
      t.Fatal(err)
    }
    fromSqlite = append(fromSqlite, e)
  }
  sqlRows.Close()
  if err := sqlRows.Err(); err ≠ nil {
    t.Fatal(err)
  }

  if len(fromParquet) ≠ len(fromSqlite) {
    t.Fatalf("parquet %d rows vs sqlite %d rows", len(fromParquet), len(fromSqlite))
  }
  for i := range fromSqlite {
    if fromParquet[i] ≠ fromSqlite[i] {
      t.Fatalf("row %d: parquet %+v ≠ sqlite %+v", i, fromParquet[i], fromSqlite[i])
    }
  }

  // and the copy is queryable where sqlite never could: aggregate the
  // parquet directly and get the ledger balance per account
  var cash int64
  if err := db.QueryRow(`select sum(cents) from read_parquet('` + pp + `')
    where account = 'cash'`).Scan(&cash); err ≠ nil {
    t.Fatal(err)
  }
  if cash ≠ 1375 {
    t.Fatalf("cash balance from parquet = %d, want 1375", cash)
  }
}
```

listing 46: the referee agrees, and the archive itself answers aggregates (source: infrastructure/samples/ch12/copyout_test.go, lines 67-105)

The copy closes the loop this chapter keeps drawing: the operational engine owns the writes, the analytical engine owns the scans, and the boundary between them is a file format both speak.

12.5. two engines, one answer

Companionability earns trust when the two engines can be checked against each other on a real schema rather than a toy. The parity probe builds the capstone's actual migrations, `0001_init.sql` and `0002_rollup.sql` verbatim, in sqlite through modernc, loads a known fixture, and asserts the foreign key still rejects a ghost room:

```
// buildCapstone creates the migration schema verbatim and loads the
// fixture rows: 3 users, 3 rooms, 6 messages.
func buildCapstone(t *testing.T, path string) {
  db := openSqlite(t, path, "pragma foreign_keys=on")
```

```
  // 0001_init.sql
  exec(t, db, `create table users (
    id   integer primary key,
    name text not null unique)`)
  exec(t, db, `create table rooms (
    id   integer primary key,
    name text not null unique)`)
  exec(t, db, `create table messages (
    id      integer primary key,
    room_id integer not null references rooms(id),
    user_id integer not null references users(id),
    body    text not null,
    ts      integer not null)`)

  // 0002_rollup.sql
  exec(t, db, `create index messages_room_id_id on messages(room_id, id)`)
  exec(t, db, `create view room_counts as
    select r.name as room,
           count(*) as messages,
           count(distinct m.user_id) as talkers
    from rooms r
    join messages m on m.room_id = r.id
    group by r.name`)

  for i, name := range []string{"alice", "bob", "carol"} {
    exec(t, db, `insert into users(id, name) values (?, ?)`, i+1, name)
  }
  for i, name := range []string{"general", "random", "ops"} {
    exec(t, db, `insert into rooms(id, name) values (?, ?)`, i+1, name)
  }
  messages := []struct {
    id, room, user int
    body           string
    ts             int64
  }{
    {1, 1, 1, "morning", 1},
    {2, 1, 2, "hello", 2},
    {3, 1, 1, "again", 3},
    {4, 2, 2, "meme", 4},
    {5, 2, 3, "lol", 5},
    {6, 3, 3, "deploy done", 6},
  }
  for _, m := range messages {
    exec(t, db, `insert into messages(id, room_id, user_id, body, ts)
      values (?, ?, ?, ?, ?)`, m.id, m.room, m.user, m.body, m.ts)
  }

  // the schema shape is the capstone's: the fk rejects a message for
  // a room that does not exist
  if _, err := db.Exec(`insert into messages(id, room_id, user_id, body, ts)
    values (7, 99, 1, 'ghost room', 7)`); err == nil {
    t.Fatal("foreign key accepted a message into a missing room")
  }
}
```

listing 47: the capstone schema and fixture, built exactly as the migrations build it (source: infrastructure/samples/ch12/parity_test.go, lines 31-89)

Then the same question, top rooms by message count, is answered twice: sqlite through the 0002 view, duckdb with its own group by over the attached tables, the sqlite-defined view never consulted:

```
func TestTwoEnginesAgreeOnTheCapstoneData(t *testing.T) {
  dir := t.TempDir()
  sp := pathOf(dir, "chat.db")
  buildCapstone(t, sp)

  // sqlite's answer: the 0002 view, read through modernc
  sqliteSide := openSqlite(t, sp)
  rows, err := sqliteSide.Query(`select room, messages, talkers
    from room_counts order by room`)
  if err != nil {
    t.Fatal(err)
  }
  var fromView []roomCount
  for rows.Next() {
    var r roomCount
    if err := rows.Scan(&r.room, &r.messages, &r.talkers); err != nil {
      t.Fatal(err)
    }
    fromView = append(fromView, r)
  }
  rows.Close()
  if err := rows.Err(); err != nil {
    t.Fatal(err)
  }

  // duckdb's answer: its own group by over the attached tables, the
  // view defined in sqlite never consulted
  db := openDuck(t)
  attachSqlite(t, db, "s", sp)
  dRows, err := db.Query(`select r.name as room,
      count(*) as messages,
      count(distinct m.user_id) as talkers
    from s.rooms r
    join s.messages m on m.room_id = r.id
    group by r.name
    order by r.name`)
  if err != nil {
    t.Fatal(err)
  }
  var fromGroup []roomCount
  for dRows.Next() {
    var r roomCount
    if err := dRows.Scan(&r.room, &r.messages, &r.talkers); err != nil {
      t.Fatal(err)
    }
    fromGroup = append(fromGroup, r)
  }
  dRows.Close()
  if err := dRows.Err(); err != nil {
    t.Fatal(err)
  }

  // three parties, one truth: the view, the group by, and the
  // hand-derived expectation
```

```
  for i, w := range wantRoomCounts {
    if fromView[i] ≠ w {
      t.Fatalf("sqlite view row %d = %+v, want %+v", i, fromView[i], w)
    }
    if fromGroup[i] ≠ w {
      t.Fatalf("duckdb group by row %d = %+v, want %+v", i, fromGroup[i], w)
    }
  }

  // and duckdb can read the sqlite-defined view too, the schema
  // crossing the boundary with the data
  var viewVisible int
  if err := db.QueryRow(`select count(*) from s.room_counts`).Scan(&viewVisible); err ≠ nil {
    t.Fatalf("view over attach: %v", err)
  }
  if viewVisible ≠ len(wantRoomCounts) {
    t.Fatalf("s.room_counts rows = %d, want %d", viewVisible, len(wantRoomCounts))
  }
}
```

listing 48: the view, the group by, and the hand-derived expectation must all agree (source: infrastructure/samples/ch12/parity_test.go, lines 91-163)

Three parties, one truth: the view, the group by over the attach, and the counts derived by hand in the test. The schema crosses the boundary with the data, duckdb can even read `s.room_counts`, the sqlite-defined view, directly. One dialect edge is worth naming while it is cheap: this works because sqlite runs the schema verbatim. Porting DDL the other way, running sqlite's `create table` statements in duckdb directly, hits small disagreements, `at` is a reserved word in duckdb DDL where sqlite accepts it as a column name, so parity through the attach is the safer instrument than translation.

12.6. the cli lane, same binary

The pinned `duckdb.exe` from `make duckdb-tools` is the same 1.5.5 the dll lane links, and it runs plain sql scripts with csv output, the shape a shell pipeline consumes:

```
func TestCliScriptEmitsCsv(t *testing.T) {
  // samples/ch12 -> samples -> infrastructure -> books -> repo root
  _, thisFile, _, _ := runtime.Caller(0)
  root := filepath.Dir(filepath.Dir(filepath.Dir(filepath.Dir(filepath.Dir(thisFile)))))
  exe := filepath.Join(root, "tools", "duckdb", "build", "bin", "duckdb.exe")
  if _, err := os.Stat(exe); err != nil {
    t.Fatalf("duckdb.exe not at %s: run make duckdb-tools", exe)
  }

  dir := t.TempDir()
  script := pathOf(dir, "top_rooms.sql")
  // csv mode, then the same rollup the parity probe answers: headers
  // on, one row per group, sorted
  os.WriteFile(filepath.Join(dir, "top_rooms.sql"), []byte(`.mode csv
create table rooms(id int, name varchar);
create table messages(id int, room_id int);
insert into rooms values (1,'general'), (2,'random'), (3,'ops');
insert into messages values (1,1),(2,1),(3,1),(4,2),(5,2),(6,3);
select r.name as room, count(*) as messages
from rooms r join messages m on m.room_id = r.id
group by r.name order by r.name;
select version() as v;
`), 0o644)

  out, err := osexec.Command(exe, "-no-stdin", ":memory:", "-c", ".read "+script).Output()
  if err != nil {
    if ee, ok := err.(*osexec.ExitError); ok {
      t.Fatalf("duckdb cli: %v\n%s", err, ee.Stderr)
    }
    t.Fatal(err)
  }

  // the windows cli writes crlf; normalize before comparing lines
  lines := strings.Split(strings.TrimSpace(strings.ReplaceAll(string(out), "\r\n", "\n")), "\n")
  want := []string{
    "room,messages",
    "general,3",
    "ops,1",
    "random,2",
    "v",
    "v1.5.5",
  }
  if len(lines) != len(want) {
    t.Fatalf("cli output = %q, want %q", string(out), strings.Join(want, "\n"))
  }
  for i, w := range want {
    if lines[i] != w {
      t.Fatalf("cli line %d = %q, want %q", i, lines[i], w)
    }
  }
}
```

listing 49: one script, csv mode, headers and rows on stdout, version asserted (source: infrastructure/samples/ch12/cli_test.go, lines 17-67)

Two small truths are pinned there. The script's last statement is `select version()`, so the cli lane asserts the same pin the go lane does. And the windows cli emits CRLF, which the probe normalizes before comparing lines, because a pipeline that forgot would match nothing and fail loudly, which is the right direction for that surprise.

12.7. choosing the engine

Four durable stores appear across this book's capstone, and the selection rule is a question about the workload:

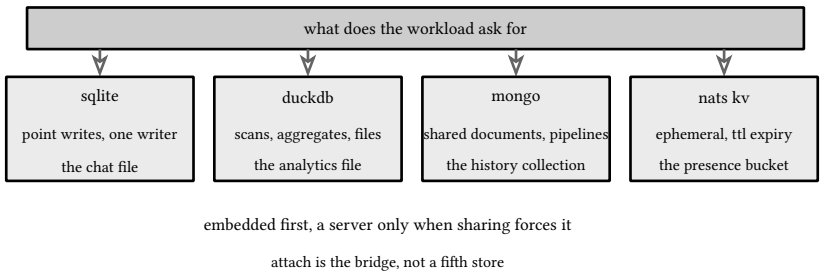


figure 35: the workload picks the engine, embedded before server

Sqlite owns the transactional seat: point access, single digit writes, one writer with the busy ladder from book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation) keeping commits honest. Duckdb owns the analytical seat: scans, group bys, parquet and csv, and a reader concurrency model where nobody parks, the mvcc contrast book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine) probed against that same ladder. Mongo earns a server when the documents are shared by many services and the aggregation runs where the data lives, the book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines) trade. And nats kv is not a database at all: presence wants ttl expiry and watch semantics, and holding it in a durable store would mean sweeping stale rows forever, the exact sweep book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) had to write anyway.

The rule the diagram encodes: embedded first, and a server only when sharing forces it. The attach is a bridge between the two embedded engines, useful for the parity and archival shapes this chapter probed, and it is not a fifth home for data.

12.8. the capstone, four stores named

The capstone's topology is the decision applied. The chat service writes sqlite, one file, one writer, `busy_timeout` for the administrative second writer. The analytics service, new beside it, consumes the same CHAT stream through its own `analysts` durable and writes one duckdb file, its own offset, its own engine, lagging or replaying without history's cursor moving. History keeps mongo and its pipelines, because its consumers share the collection. Presence stays in the nats kv bucket with the ttl doing the expiry. Each store has exactly one writer service, which is the whole reason no two writers share a lock domain, and book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 1: services and topology) draws the picture.

12.9. the v2.0 horizon

Version 2.0, "Cyanoptera", exists today only as a feature-frozen preview: the `v2.0-cyanoptera` branch publishes rolling tarball artifacts to artifacts.duckdb.org with no tagged release, and the final is announced for October 2026. What it brings is a client-server protocol named Quack, async I/O, rewritten recursive CTEs, and faster parquet scans. Two gaps matter for this repo specifically: the windows extensions are not yet published for the 2.0 client, which would leave the sqlite attach lane of this chapter dead on windows, and the go module tag will move to a v2.20000.x-style encoding of the core version. Until the release lands, 1.5.5 stays the pin and 1.4.5 is the line if a longer support window ever matters more than the features.

NOTE: the re-pin checklist, written while it is cheap

Five things make a future re-pin a chore instead of an incident. Keep the exact-string `version()` asserts centralized, one per module, so a drift breaks in one obvious place. Re-run the sqlite extension warm in `tools/build-duckdb.sh`, because the extension is engine-version specific and a new engine will not load yesterday's

> binary. Move the go.mod tag to the new v2.20000.x-style encoding and let the version test confirm the decoding. Re-record the cli and libduckdb sha256s in `tools/build-duckdb.sh` against the release's published digests. And recheck windows extension availability before promising the attach lane there, because today's 2.0 preview ships none.

sources: duckdb.org documentation for the sqlite extension and attach scanner settings, the csv and parquet pages already cited in the previous chapter, install.duckdb.org for the 1.5.5 binaries, the duckdb-go repository and tag list, and the artifacts.duckdb.org v2.0-cyanoptera preview channel, all accessed 2026-09-20. Verified by the ch12 probe suite, 7 tests behind the `duckdb_use_lib` tag against the pinned 1.5.5 dll and modernc 3.53.4, green under `make verify-go` 2026-09-20.

13. mongodb and aggregation pipelines

The stats the history service answers live in mongo, and the reason is the opposite of the sqlite chapters' reason: chat messages are append-only documents with shifting query shapes, and the questions asked of them, top talkers, room volumes, busiest hour, trending words, are group-bys computed on demand. Mongo's aggregation pipeline is a language for exactly that, stages flowing left to right, each transforming the document stream.

13.1. the seam first

The history service never imports the driver in its logic. It declares what it needs from a store:

```
type Store interface {
    Insert(ctx context.Context, m wire.Message) error
    Stats(ctx context.Context) (wire.Stats, error)
}
```

fragment, illustrative, not executed

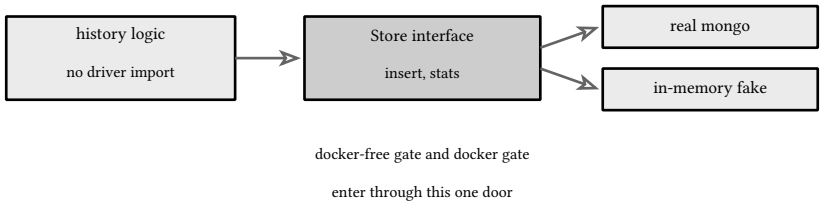


figure 36: the Store interface is the only door, and both gates use it

Two methods, one for the consumer to feed, one for the stats responder to answer. The real mongo implementation satisfies it, and so does the in-memory fake the docker-free gate runs on, book 13, ch 17 (infrastructure: docker, databases, and testing, mocks, seeding, contract tests). The chapter's pipelines are the real implementation.

13.2. idempotent writes

The consumer acks after inserting, so a crash between insert and ack means one redelivery, and the insert must tolerate it:

```
func (m *MongoStore) Insert(ctx context.Context, msg wire.Message) error {
  _, err := m.coll.UpdateOne(ctx,
    bson.M{"id": msg.ID},
    bson.M{"$set": docOf(msg)},
    options.UpdateOne().SetUpsert(true),
  )
  return err
}

func docOf(m wire.Message) bson.M {
  return bson.M{
    "id": m.ID, "room": m.Room, "sender": m.Sender,
    "body": m.Body, "ts": m.TS,
  }
}
```

listing 50: an upsert keyed on message id, with docOf as the one shaper (source: infrastructure/capstone/internal/history/mongo.go, lines 59-73)

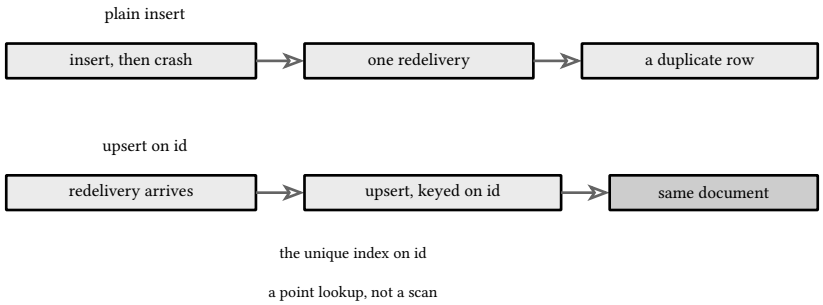


figure 37: one redelivery, two inserts: the plain insert doubles, the upsert lands on the same document

The unique index on `id` is created at connect time. Without it the upsert is a scan, and with a redelivery it would still be correct, but correctness by full-collection-scan is not a plan.

13.3. the pipeline vocabulary

The stats answer is four pipelines over the same collection. The simplest one is parameterized:

```
func (m *MongoStore) topGroup(ctx context.Context, on any, limit int) ([]wire.Stat, error) {
  pipeline := mongo.Pipeline{
    {{Key: "$group", Value: bson.M{
      "_id":   on,
      "count": bson.M{"$sum": 1},
    }}},
    {{Key: "$sort", Value: bson.D{
      {Key: "count", Value: -1}, {Key: "_id", Value: 1},
    }}},
    {{Key: "$limit", Value: int64(limit)}},
    {{Key: "$project", Value: bson.M{
      "_id": 0, "key": "$_id", "count": 1,
    }}},
  }
  cursor, err := m.coll.Aggregate(ctx, pipeline)
  if err ≠ nil {
    return nil, err
  }
  var rows []wire.Stat
  if err := cursor.All(ctx, &rows); err ≠ nil {
    return nil, err
  }
  return rows, nil
}
```

listing 51: group, sort, limit, project: the four stages that answer top talkers and top rooms (source: infrastructure/capstone/internal/history/mongo.go, lines 112-135)

`$group` collapses the stream on an expression, here a field path or the hour label below, `$sort` orders, `$limit` truncates, `$project` reshapes into the wire.Stat rows the responder marshals. The sort carries a tiebreaker on the key so equal counts order deterministically, the same rule the go fake enforces.

The busiest hour bucket turns a unix millisecond timestamp into a label inside the pipeline:

```
var hourExpr = bson.D{
  {Key: "$dateToString", Value: bson.M{
    "format": "%H:00", "date": bson.M{"$toDate": "$ts"}, "timezone": "UTC",
  }},
}
```

listing 52: `$toDate` then `$dateToString` with a format and timezone (source: infrastructure/capstone/internal/history/mongo.go, lines 105-110)

And the word count is the pipeline with the most moving parts, split, unwind, trim, filter, then the same group-sort-limit tail:

```
func (m *MongoStore) topWords(ctx context.Context) ([]wire.Stat, error) {
  pipeline := mongo.Pipeline{
    {{Key: "$project", Value: bson.M{
      "words": bson.M{"$split": bson.A{
        bson.M{"$toLower": "$body"}, " ",
      }},
    }}},
    {{Key: "$unwind", Value: "$words"}},
    {{Key: "$set", Value: bson.M{
      "words": bson.M{"$trim": bson.M{
        "input": "$words", "chars": ".,!?;:'\"()[]-",
      }},
    }}},
    {{Key: "$match", Value: bson.M{
      "$expr": bson.M{"$gt": bson.A{
        bson.M{"$strLenCP": "$words"}, 3,
      }},
    }}},
    {{Key: "$group", Value: bson.M{
      "_id":   "$words",
      "count": bson.M{"$sum": 1},
    }}},
    {{Key: "$sort", Value: bson.D{
      {Key: "count", Value: -1}, {Key: "_id", Value: 1},
    }}},
    {{Key: "$limit", Value: int64(5)}},
    {{Key: "$project", Value: bson.M{
      "_id": 0, "key": "$_id", "count": 1,
    }}},
  }
  cursor, err := m.coll.Aggregate(ctx, pipeline)
  if err ≠ nil {
    return nil, err
  }
  var rows []wire.Stat
  if err := cursor.All(ctx, &rows); err ≠ nil {
    return nil, err
  }
  return rows, nil
}
```

listing 53: `$split`, `$unwind`, `$trim`, an `$expr` length filter, then the counting tail (source: infrastructure/capstone/internal/history/mongo.go, lines 139-178)

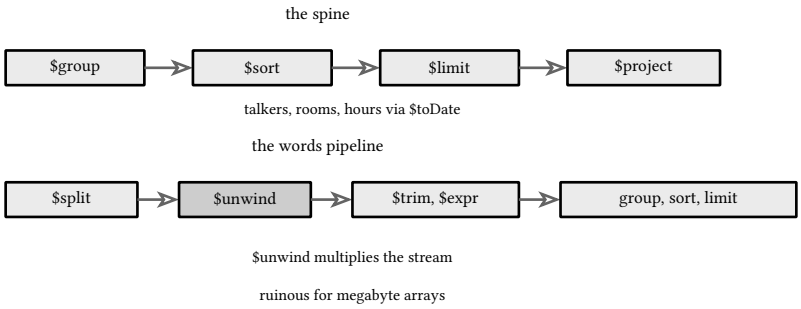


figure 38: documents flow left to right: the spine, and the words pipeline that multiplies first

`$unwind` is the stage to respect: it multiplies the stream by the array length before the group collapses it again, which is fine for chat lines and ruinous for megabyte arrays.

13.4. proven against the real engine

Every pipeline here is exercised twice, and the chapter is honest about which test proves what. The docker-free suite proves the plumbing against the go fake, whose Stats is a deliberate transcription of these stages. The real pipelines run in the docker gate:

```go
func TestChatNotificationsAndAnalyticsEndToEnd(t *testing.T) {
  suffix := fmt.Sprintf("%04d", rand.Intn(10000))
  body := "docker gate says " + suffix

  waitContains(t, "/fragment/roster?user=alice"+suffix+"&room=general", "alice"+suffix)
  waitContains(t, "/fragment/roster?user=bob"+suffix+"&room=general", "bob"+suffix)

  postForm(t, "/send", url.Values{
    "user": {"alice" + suffix}, "room": {"general"}, "body": {body},
  })
  waitContains(t, "/fragment/messages?room=general", body)
  waitContains(t, "/fragment/notifications?user=bob"+suffix, body)
  waitContains(t, "/stats?user=alice"+suffix+"&room=general", "total messages")
  // the duckdb panel answers the same stream, so both engines agree
  waitContains(t, "/analytics?user=alice"+suffix+"&room=general", "duckdb total messages")
}
```

listing 54: the docker-gated walk: notifications, mongo stats, then the duckdb analytics panel (source: infrastructure/capstone/integration/stack_test.go, lines 97-112)

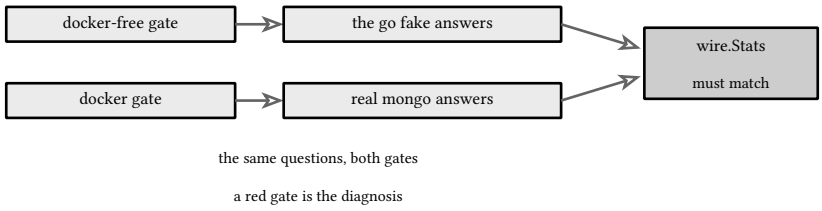


figure 39: the two-gate contract: same questions, both must fill wire.Stats identically

WARNING: schemaless does not mean thoughtless

Mongo will store anything. The first document decides the field names every later query must agree with, which is a schema with no enforcement, the inverse trade of sqlite's strict tables in book 13, ch 9 (infrastructure: docker, databases, and testing, sqlite: the 3.53 feature set). The capstone keeps one shaper, `docOf`, as the single place a message becomes a document, so drift has one door to walk through.

sources: mongodb.com aggregation manual for the stage semantics, accessed 2026-09-10, and pkg.go.dev for the v2 driver signatures, verified against the vendored module source. Verified by the history suite, 5 tests, under `make verify`, and by the docker-gated stats assertion under `make verify-infra-docker`, both green 2026-09-10.

14. nats: pub sub, request reply, queue groups

NATS core is a subject tree with a router in the middle. Publishers send to subjects, subscribers declare interest in subjects, and the broker matches, with no persistence, no delivery guarantee beyond at-most-once to a connected subscriber, and correspondingly no configuration. It is the smallest useful messaging primitive, and the capstone's request paths run on it entirely.

14.1. subjects are a hierarchy

A subject is dot separated tokens, `chat.room.general`, and the wildcards are token wildcards: `*` matches exactly one token, `>` matches one or more to the end. The probe runs all three subscription shapes against one publish and counts who saw what:

```
func TestWildcardSubjects(t *testing.T) {
  _, nc := runServer(t)
  var mu sync.Mutex
  seen := map[string]int{}
  mark := func(name string) func(*nats.Msg) {
    return func(*nats.Msg) {
      mu.Lock()
      seen[name]++
      mu.Unlock()
    }
  }
  subs := []struct{ subject, name string }{
    {"chat.room.general", "exact"},
    {"chat.room.*", "star"},
    {"chat.>", "gt"},
  }
  for _, s := range subs {
    if _, err := nc.Subscribe(s.subject, mark(s.name)); err != nil {
      t.Fatal(err)
    }
  }
  if err := nc.Flush(); err != nil {
    t.Fatal(err)
  }
  nc.Publish("chat.room.general", []byte("x"))
  nc.Flush()
  time.Sleep(100 * time.Millisecond)

  mu.Lock()
  for _, want := range []string{"exact", "star", "gt"} {
    if seen[want] != 1 {
      mu.Unlock()
      t.Fatalf("%s saw %d deliveries, want 1 (all: %v)", want, seen[want], seen)
    }
  }
  mu.Unlock()

  // a subject one level deeper only matches the > wildcard
  nc.Publish("chat.room.general.mods", []byte("x"))
  nc.Flush()
  time.Sleep(100 * time.Millisecond)

  mu.Lock()
  defer mu.Unlock()
  if seen["exact"] != 1 || seen["star"] != 1 {
    t.Fatalf("deep subject hit exact or star: %v", seen)
  }
  if seen["gt"] != 2 {
    t.Fatalf("gt wildcard missed the deep subject: %v", seen)
  }
}
```

listing 55: exact, star, and gt against two publishes of different depth (source: infrastructure/samples/ch14/nats_test.go, lines 66-116)

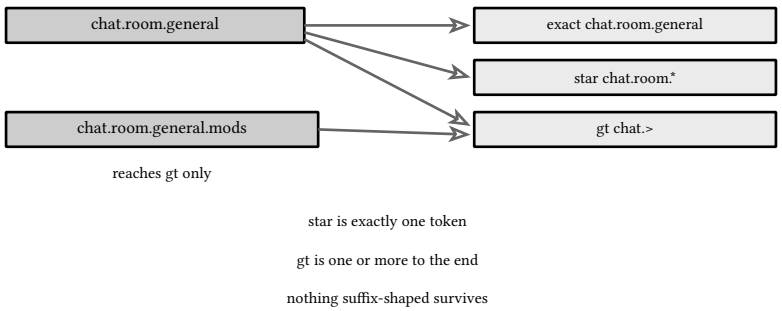


figure 40: token wildcards: who sees each publish

The second publish is the one that teaches: `chat.room.general.mods` misses the exact subscription and the star, one token stands for one token, and reaches the > subscription only. Nothing string-suffix shaped survives this probe, which is the point of running it.

14.2. request reply is one publish with a return address

A request publishes with an inbox subject attached as the reply field and waits on a fresh subscription to that inbox. The responder publishes to the inbox it was handed:

```
func TestRequestReply(t *testing.T) {
  _, nc := runServer(t)
  if _, err := nc.Subscribe("presence.online", func(m *nats.Msg) {
    m.Respond([]byte(`[{"user":"alice"}]`))
  }); err ≠ nil {
    t.Fatal(err)
  }
  if err := nc.Flush(); err ≠ nil {
    t.Fatal(err)
  }
  msg, err := nc.RequestWithContext(context.Background(), "presence.online", nil)
  if err ≠ nil {
    t.Fatal(err)
  }
  if string(msg.Data) ≠ `[{"user":"alice"}]` {
    t.Fatalf("reply = %s", msg.Data)
  }
}
```

listing 56: responder and requester, one round trip, no broker config (source: infrastructure/samples/ch14/nats_test.go, lines 120-137)

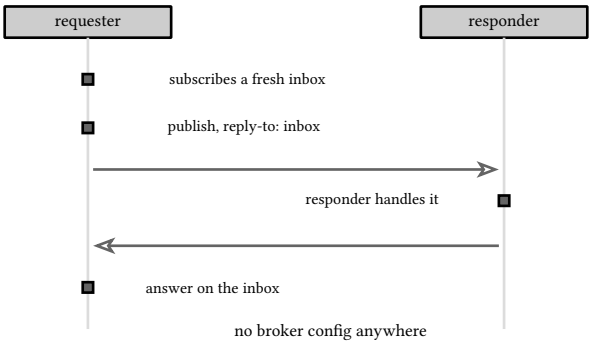


figure 41: request reply: one publish carrying a return address

The capstone wraps exactly this in two functions, one per side, so every service boundary uses the same encoding and error envelope:

```
func CallJSON(ctx context.Context, nc *nats.Conn, subject string, in, out any) error {
  payload, err := json.Marshal(in)
  if err ≠ nil {
    return err
  }
  cctx, cancel := context.WithTimeout(ctx, callTimeout)
  defer cancel()
  msg, err := nc.RequestWithContext(cctx, subject, payload)
  if err ≠ nil {
    if errors.Is(err, context.DeadlineExceeded) {
      return context.DeadlineExceeded
    }
    return err
  }
  var env envelope
  if err := json.Unmarshal(msg.Data, &env); err ≠ nil {
    return err
  }
  if env.Error ≠ "" {
    return errors.New(env.Error)
  }
  if out == nil {
    return nil
  }
  return json.Unmarshal(env.Data, out)
}
```

listing 57: the caller side: marshal, request, error envelope, decode (source: infrastructure/capstone/internal/bus/rpc.go, lines 24-49)

A modern nicety the capstone leans on: a request with no responder fails fast, the broker answers with a no-responders status instead of silence, so a missing service is an immediate error rather than a timeout to stare at:

```
func TestNoRespondersFailsFast(t *testing.T) {
  _, nc := runServer(t)
  _, err := nc.RequestWithContext(context.Background(), "nobody.home", nil)
  if !errors.Is(err, nats.ErrNoResponders) {
    t.Fatalf("err = %v, want ErrNoResponders", err)
  }
}
```

listing 58: nobody home is an error now, not a hang (source: infrastructure/samples/ch14/nats_test.go, lines 142-148)

14.3. queue groups: load balancing with no broker config

Plain subscriptions are broadcast, every subscriber sees everything. Adding a queue name turns a set of subscribers into one logical subscriber, and the broker delivers each message to exactly one member:

```
func TestQueueGroupDeliversExactlyOnce(t *testing.T) {
  _, nc := runServer(t)
  const messages = 50
  var mu sync.Mutex
  handled := make([]int, 3)
  for worker := 0; worker < 3; worker++ {
    w := worker
    if _, err := nc.QueueSubscribe("jobs.run", "workers", func(*nats.Msg) {
      mu.Lock()
      handled[w]++
      mu.Unlock()
    }); err != nil {
      t.Fatal(err)
    }
  }
  if err := nc.Flush(); err != nil {
    t.Fatal(err)
  }
  for i := 0; i < messages; i++ {
    if err := nc.Publish("jobs.run", nil); err != nil {
      t.Fatal(err)
    }
  }
  if err := nc.Flush(); err != nil {
    t.Fatal(err)
  }

  deadline := time.Now().Add(5 * time.Second)
  for time.Now().Before(deadline) {
    mu.Lock()
    total := handled[0] + handled[1] + handled[2]
    mu.Unlock()
    if total == messages {
      break
    }
    time.Sleep(50 * time.Millisecond)
  }
  mu.Lock()
  defer mu.Unlock()
  total := handled[0] + handled[1] + handled[2]
  if total != messages {
    t.Fatalf("queue group delivered %d of %d", total, messages)
  }
  idle := 0
  for _, n := range handled {
    if n == 0 {
      idle++
    }
  }
  if idle > 0 {
    t.Fatalf("distribution left a worker idle: %v", handled)
  }
}
```

listing 59: fifty messages across three workers: all delivered, none twice, nobody idle (source: infrastructure/samples/ch14/nats_test.go, lines 153-205)

That is worker pooling with zero configuration, and it is the primitive the web tier would scale on, every replica sees every notification because plain, while notify replicas share the stream work because queue semantics arrive with JetStream's durable consumers in book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared).

14.4. the broker in process

Everything above runs in the test binary. The embedded server is the same code the compose stack runs in a container, minus the container:

```
func Run(t *testing.T) (*server.Server, *nats.Conn) {
  t.Helper()
  opts := &server.Options{
    JetStream: true,
    StoreDir:  t.TempDir(),
    Port:      -1,
    NoLog:     true,
    NoSigs:    true,
  }
  srv, err := server.NewServer(opts)
  if err ≠ nil {
    t.Fatalf("embedded nats: %v", err)
  }
  go srv.Start()
  if !srv.ReadyForConnections(5 * time.Second) {
    t.Fatal("embedded nats never became ready")
  }
  nc, err := nats.Connect(srv.ClientURL(), nats.NoReconnect())
  if err ≠ nil {
    srv.Shutdown()
    t.Fatalf("dial embedded nats: %v", err)
  }
  t.Cleanup(func() {
    nc.Drain()
    srv.Shutdown()
  })
  return srv, nc
}
```

listing 60: one function: random port, jetstream on, temp store, cleanup registered (source: infrastructure/capstone/internal/testnats/server.go, lines 18-45)

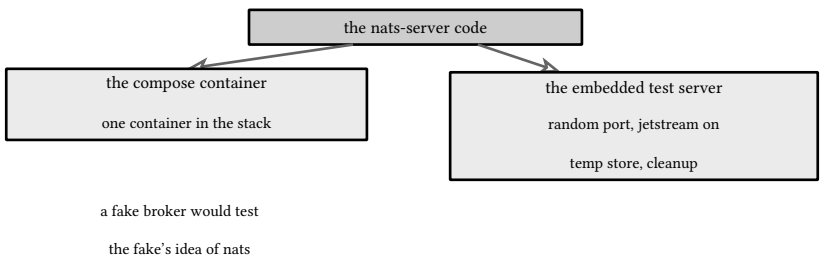


figure 42: the same nats-server code in two homes

Every capstone test calls this instead of mocking the broker. The distinction is the chapter's quiet argument: a fake broker would test the fake's idea of nats, while the embedded server is the real dispatcher with real subject routing, at the cost of an import.

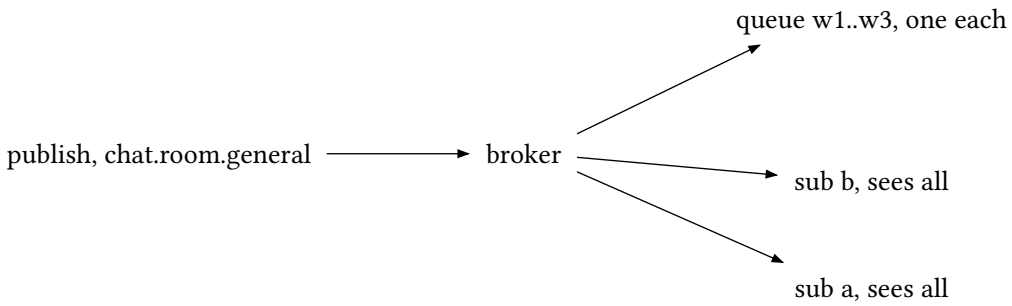


figure 43: who gets what, plain versus queue

NOTE: core nats forgets

A message published with no connected subscriber is gone. That is the design: the capstone's durable paths, the chat log and the presence bucket, live one chapter over in JetStream, while the request reply and notification fanout, where a lost message means a missed ping not lost data, stay on core and stay simple.

sources: docs.nats.io subject and wildcard semantics, accessed 2026-09-10, plus pkg.go.dev for the embedded server options. Verified by the ch14 probe suite, 6 tests, and the capstone bus suite, 5 tests, all green under `make verify` 2026-09-10.

15. jetstream: streams, consumers, kv, kafka compared

Core NATS forgets. JetStream is the layer that remembers: a stream declares a set of subjects the server durably records, and consumers read that recording at their own pace with their own position. One small config object turns the capstone's fire-and-forget chat publish into the system's durable log:

```
const StreamName = "CHAT"

// Subjects is what the stream captures.
const Subjects = "chat.room.>"

// NotifierDurable fans notifications out; HistorianDurable persists to
// mongo. Distinct durables mean distinct offsets over the same log.
const (
  NotifierDurable  = "notifiers"
  HistorianDurable = "historians"
)

// EnsureChatStream creates the CHAT stream if absent and returns it.
// Both consumers and the producer call it; CreateOrUpdate is the
// idempotent form.
func EnsureChatStream(ctx context.Context, js jetstream.JetStream) (jetstream.Stream, error) {
  return js.CreateOrUpdateStream(ctx, jetstream.StreamConfig{
    Name:     StreamName,
    Subjects: []string{Subjects},
    MaxAge:   24 * time.Hour,
  })
}
```

listing 61: the whole jetstream agreement of the capstone: one stream, two durables, in one file (source: infrastructure/capstone/internal/jet/chat.go, lines 16-37)

The chat service keeps publishing to `chat.room.general` exactly as before. The stream captures it because the subjects match, and two services read the same log independently, each with its own offset. The analytics service adds a third durable of its own, book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 1: services and topology).

15.1. durable consumers are offsets with a name

A consumer with a durable name holds its position on the server. That position is the ack floor, the earliest message not yet acknowledged: everything below it is settled, and acking the floor message advances it. The notify service is one, and its handler is the whole delivery discipline:

```
func (s *Service) handle(msg jetstream.Msg) {
  if err := s.fanout(msg); err ≠ nil {
    msg.NakWithDelay(s.ackWait)
    return
  }
  msg.Ack()
}

func (s *Service) fanout(msg jetstream.Msg) error {
  var m wire.Message
  if err := json.Unmarshal(msg.Data(), &m); err ≠ nil {
    // a malformed payload will never parse on any redelivery; ack
    // it rather than poison the consumer with retries
    return nil
  }
  roster, err := s.roster()
  if err ≠ nil {
    return err
  }
  for _, p := range roster {
    if p.User == m.Sender {
      continue
    }
    if err := s.nc.Publish("notify.user."+p.User, msg.Data()); err ≠ nil {
      return err
    }
  }
  return nil
}
```

listing 62: settle the message: fan out and ack, or delay the nak and retry (source: infrastructure/capstone/internal/notify/notify.go, lines 89-117)

Two hard-won details live in those lines. The malformed payload is acked, not retried, because it will never parse better, and a poisoned consumer is worse than a dropped bad message. And the retry is `NakWithDelay`, not a plain `Nak`, and the reason is a debugging story worth telling: the first implementation used a plain nak, and the redelivery test burned through the entire delivery budget (`MaxDeliver`, the consumer config's redelivery cap, pinned at 50) in two milliseconds. A nak means redeliver immediately, so a dependency that is down for half a second eats every retry at once. Delaying the nak spaces redeliveries by the ack wait, which is what the failing test actually wanted:

```go
func TestNakRedelivers(t *testing.T) {
  _, nc := testnats.Run(t)
  ctx := context.Background()
  wait := 250 * time.Millisecond
  svc, err := New(nc, wait)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { svc.Close() })

  // no presence responder yet: the first delivery fails and naks
  seed(t, nc, wire.Message{ID: 1, Room: "general", Sender: "alice", Body: "retry me", TS: 1})

  // give the nak time to round trip, then start presence
  time.Sleep(2 * wait)
  pres, err := presence.New(nc, time.Minute)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { pres.Close() })
  if err := pres.Beat(ctx, "bob", "general"); err ≠ nil {
    t.Fatal(err)
  }
  bobCh := deliver(t, nc, "bob")
  select {
  case got := ←bobCh:
    if got.Body ≠ "retry me" {
      t.Fatalf("bob got %+v", got)
    }
  case ←time.After(3 * time.Second):
    t.Fatal("redelivery never reached bob")
  }
}
```

listing 63: presence down, then up: the delayed nak redelivers and the message survives (source: infrastructure/capstone/internal/notify/notify_test.go, lines 131-163)

The history service is the second durable over the same stream, same shape, different name, and its restart test proves the offset is server side: the service closes with the first message acked, a second message lands while nobody is consuming, and the restarted service receives only the second.

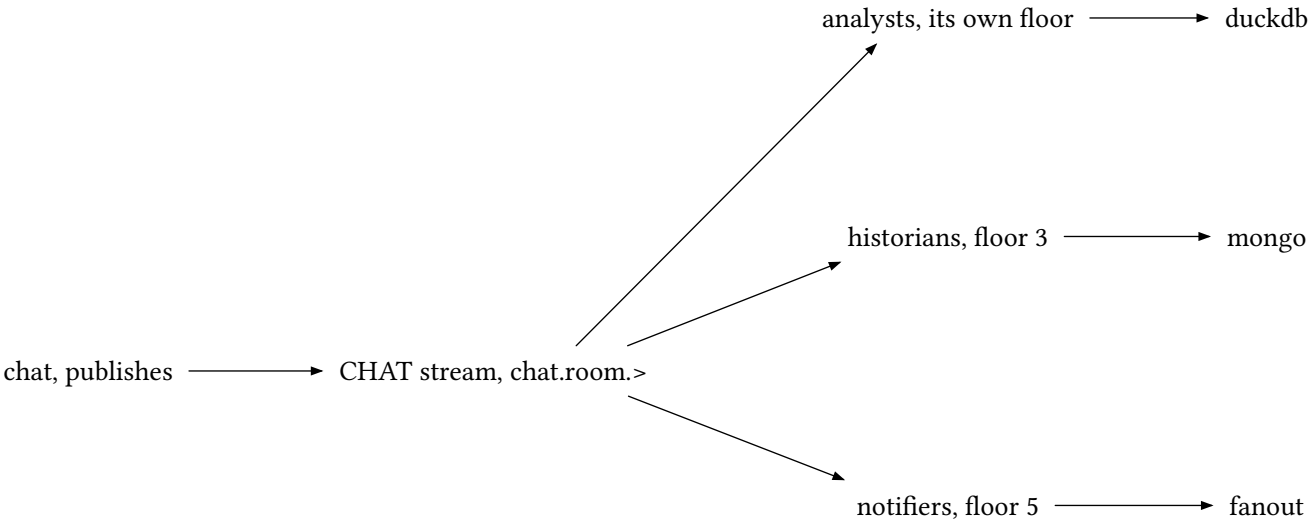


figure 44: one stream, three durables, three independent offsets

15.2. kv: a bucket with a ttl

A key value bucket is a stream wearing an api, keys as subjects, values as payloads. Presence is one: heartbeats put the room under the user key, and the bucket's TTL expires stale users server side:

```
func (s *Service) Beat(ctx context.Context, user, room string) error {
  if user == "" {
    return ErrValidation
  }
  _, err := s.kv.Put(ctx, user, []byte(room))
  return err
}
```

listing 64: a heartbeat is one put, and the put refreshes the ttl (source: infrastructure/capstone/internal/presence/presence.go, lines 134-140)

The chapter's second debugging story came from trusting the watch api here. A KV watch delivers puts and deletes, but a TTL expiry is neither: it is the stream quietly dropping an aged message, and the watcher saw nothing, tested nothing, shipped nothing. The fix is the sweep, a periodic roster diff that publishes the joins and leaves the watch cannot see:

```
func (s *Service) sweep(ttl time.Duration) {
  defer close(s.done)
  cadence := ttl / 3
  if cadence < 100*time.Millisecond {
    cadence = 100 * time.Millisecond
  }
  ticker := time.NewTicker(cadence)
  defer ticker.Stop()
  last := map[string]string{}
  for {
    select {
    case <-s.stop:
      return
    case <-ticker.C:
    }
    roster, err := s.Online(context.Background())
    if err != nil {
      continue
    }
    now := map[string]string{}
    for _, p := range roster {
      now[p.User] = p.Room
    }
    for _, p := range roster {
      if _, was := last[p.User]; !was {
        s.publish(wire.Presence{User: p.User, Room: p.Room, Online: true})
      }
    }
    for user := range last {
      if _, still := now[user]; !still {
        s.publish(wire.Presence{User: user, Online: false})
      }
    }
    last = now
  }
}
```

listing 65: reconcile, at a third of the ttl: what appeared is a join, what vanished is a leave (source: infrastructure/capstone/internal/presence/presence.go, lines 87-122)

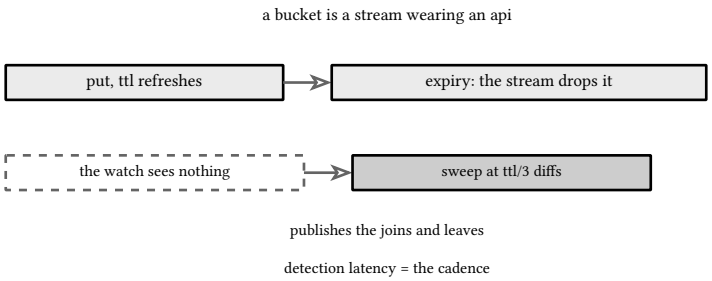


figure 45: a bucket with a ttl: the silent drop, and the sweep that sees it

The pattern generalizes past NATS: any TTL system whose expiry is invisible to notification needs a reconciler, and the reconciler's cadence is the detection latency you signed up for.

15.3. kafka, compared honestly

The shapes map almost one to one. Kafka's partitioned log is the stream, the consumer group is the durable consumer, committed offsets are the ack floor, and compaction is what a KV bucket's per-key history is doing.

The differences are in defaults and operations: kafka's partition count is fixed at topic creation and keys route to partitions, so parallelism is planned up front, while a JetStream stream parallelizes through consumers and leaves subject structure to the application. Kafka's retention is the product, built for long replay and heavy fan-in at high throughput; JetStream's defaults favor smaller systems that already speak NATS. The capstone implements the NATS side and treats kafka as this paragraph, because the second implementation would teach the api, and the api is not the hard part, the delivery semantics are, and those are what the tests here pin.

kafka		jetstream
partitioned log	=	stream
consumer group	=	durable consumer
committed offsets	=	ack floor
compaction	=	per-key kv history

partitions fixed up front vs consumers scale

retention as product vs smaller defaults

figure 46: kafka to jetstream: the shapes map, the defaults differ

WARNING: exactly once is a pipeline property, not a flag

The consumer acks after the fanout, so a crash between publish and ack redelivers, and a recipient can see a notification twice. The capstone accepts this because notifications are idempotent to read. The history consumer pays for stronger semantics instead, the upsert keyed on message id from book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines), which is the standard answer: make the sink idempotent, and at-least-once delivery is enough.

sources: docs.nats.io jetstream concepts, subjects, and the kv developer page, accessed 2026-09-10, and the nats.go jetstream package api verified against the vendored module source, v1.53.1. Verified by the notify suite, 5 tests, the history suite, 5 tests, and the presence suite, 4 tests, green under `make verify` 2026-09-10.

16. testing without docker

Chapter 1 drew the line: `make verify` never needs the daemon, and anything that does is gated behind `make verify-infra-docker` where absence is a loud red. This chapter is what makes that split honest rather than aspirational, because the hard part is not the rule, it is building a system whose tests mostly do not want docker.

The numbers first: the docker-free gate runs 131 go tests, 61 in the capstone module and 70 in the chapter probe module, and the probe module splits by lane, 53 compiler-free tests under plain `go test ./...` and 17 duckdb-tagged tests behind the pinned dll, zero skipped, and none of them ask for the engine. The docker gate runs 4 more that ask for nothing but the web port.

16.1. the three substitutions

Every technology the capstone uses has a docker-free stand-in that is not a mock of the technology:

```
func Run(t *testing.T) (*server.Server, *nats.Conn) {
  t.Helper()
  opts := &server.Options{
    JetStream: true,
    StoreDir:  t.TempDir(),
    Port:      -1,
    NoLog:     true,
    NoSigs:    true,
  }
  srv, err := server.NewServer(opts)
  if err ≠ nil {
    t.Fatalf("embedded nats: %v", err)
  }
  go srv.Start()
  if !srv.ReadyForConnections(5 * time.Second) {
    t.Fatal("embedded nats never became ready")
  }
  nc, err := nats.Connect(srv.ClientURL(), nats.NoReconnect())
  if err ≠ nil {
    srv.Shutdown()
    t.Fatalf("dial embedded nats: %v", err)
  }
  t.Cleanup(func() {
    nc.Drain()
    srv.Shutdown()
  })
  return srv, nc
}
```

listing 66: the broker: the real server, embedded, on a random port with a temp store (source: infrastructure/capstone/internal/testnats/server.go, lines 18-45)

Sqlite is its own substitution, a file in `t.TempDir()` that the test framework deletes. Mongo is the one fake, and it is a fake at a seam, the `Store` interface, not a fake mongodb, and book 13, ch 17 (infrastructure: docker, databases, and testing, mocks, seeding, contract tests) puts it under the microscope. Duckdb needs no substitution at all: it embeds the way sqlite does, one file per test in a temp directory, so the analytics service runs its real sql in the same one-process stack.

With those three, the whole system assembles in one process. The web suite is the demonstration: real chat service with a real sqlite file, real presence and notify and history and analytics over the embedded broker, fake store behind history, real http handlers, real clients:

```
func newStack(t *testing.T) *httptest.Server {
  t.Helper()
  db, err := store.Open(filepath.Join(t.TempDir(), "chat.db"))
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { db.Close() })
  _, nc := testnats.Run(t)
  if _, err := chat.New(db, nc); err ≠ nil {
    t.Fatal(err)
  }
  pres, err := presence.New(nc, 30*time.Second)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { pres.Close() })
  if _, err := notify.New(nc, notify.AckWait); err ≠ nil {
    t.Fatal(err)
  }
  histStore := history.NewFakeStore()
  hist, err := history.New(nc, histStore, history.AckWait)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { hist.Close() })
  anaStore, err := analytics.Open(filepath.Join(t.TempDir(), "analytics.duckdb"))
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { anaStore.Close() })
  ana, err := analytics.New(nc, anaStore, analytics.AckWait)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { ana.Close() })
  ts := httptest.NewServer(New(nc).Handler())
  t.Cleanup(ts.Close)
  return ts
}
```

listing 67: the capstone minus containers: every service real except the mongo seam (source: infrastructure/capstone/internal/web/web_test.go, lines 27-65)

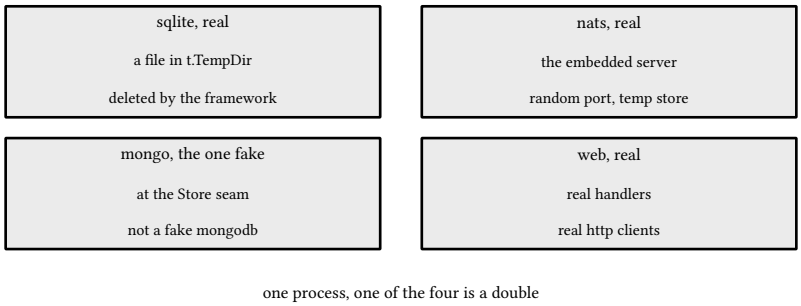


figure 47: the whole system in one process, one of the four a double

That one stack answers the questions a mock-heavy suite cannot: does a send reach sqlite and the stream and the notifier and another user's fragment, through the actual wire formats. It runs in about two seconds, which is why there is no pressure to skip it locally.

16.2. the gate itself is a build tag

The docker-gated suite is the same module with one file carrying a tag:

```
//go:build docker

// The docker-gated suite: these tests run only under
// `make verify-infra-docker`, against the compose stack it brings up
// on localhost:8080. They never run under `make verify`, which is the
// entire point of the two-gate rule in chapter 1.
package integration
```

listing 68: the tag that keeps these tests out of every run that does not ask for them (source: infrastructure/capstone/integration/stack_test.go, lines 1-8)

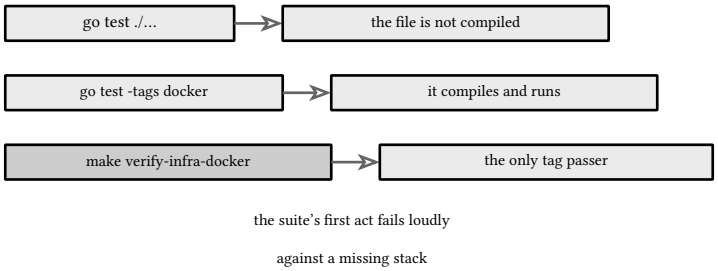


figure 48: the compiler is the gatekeeper: three ways to run, one passes the tag

`go test ./...` never compiles the file, `go test -tags docker` only runs it, and the make target is the only thing that passes the tag. The suite's first act is to reach the web port and fail loudly if the stack is not there, so running it bare, without compose, is a red test with a clear message, not a misleading green. The two-gate rule enforced by a compiler switch and a preflight, not by convention.

16.3. what each gate proves

The split is not arbitrary, it follows what each dependency can honestly verify. The docker-free gate proves the capstone's logic: schema and migrations, transaction and locking behavior, fanout and ack discipline, offset resumption, http fragments, escaping, the fake store's plumbing. The docker gate proves the seams the fakes cannot: the real mongo pipelines behind `history.stats`, the compose healthcheck graph, service discovery through the bridge, the published image behavior end to end. A red in either gate points at a different class of bug, and knowing which gate is red is the first line of diagnosis.

docker-free proves	docker proves
schema, migrations	real mongo pipelines
transactions, locking	the healthcheck graph
fanout, offsets	bridge discovery
fragments, escaping	published images

a red in each column is a different bug class

figure 49: what each gate proves, and what a red in each one means

WARNING: flaky is a bug, not a setting

Every wait in the docker-free suite is a bounded eventually with a condition, five seconds at the outside, and the conditions assert content, not just progress. A test that intermittently times out is telling you about a race the capstone shipped, and the answer is to find it, the way the nak and ttl stories in book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) were found by exactly such a timeout.

16.4. where expected values come from

Every assertion in both gates compares an observed value against an expected one, and the suite is only as honest as the source of the expected side. The chapter's bounded waits assert content, not just progress, and content is only as trustworthy as the place it was written down from. The three substitutions assemble a system whose answers are worth comparing, and the build tag keeps the gates honest about when they run, but neither earns its green if the values on the right of the comparison were lifted from the code on the left. Expected values have a rank order. Hand-derived from the requirement is best, arithmetic a person did away from the code, the closed form row sums and the hand computed statistics of book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine). Captured from a reference system comes next, an answer recorded from an engine already trusted, with the capture and its date written beside it. A domain property ranks last and still holds, an invariant any correct answer satisfies, the way the analytics store keeps its total at 1 across a redelivered message, because a redelivery is not a new message in any correct system.

```
// TestStatsMatchAManualCount round trips the history corpus: totals,
// top 3 talkers and rooms, the busiest UTC hour, and the top 5 words
// longer than three letters, each against values counted by hand.
func TestStatsMatchAManualCount(t *testing.T) {
  s := openStore(t)
  insertAll(t, s, manualCount...)
  stats, err := s.Stats(context.Background())
  if err ≠ nil {
    t.Fatal(err)
  }
  if stats.Total ≠ 3 {
    t.Fatalf("total = %d", stats.Total)
  }
  wantTalkers := []wire.Stat{{Key: "alice", Count: 2}, {Key: "bob", Count: 1}}
  if !reflect.DeepEqual(stats.Talkers, wantTalkers) {
    t.Fatalf("talkers = %+v, want %+v", stats.Talkers, wantTalkers)
  }
  wantRooms := []wire.Stat{{Key: "general", Count: 2}, {Key: "random", Count: 1}}
  if !reflect.DeepEqual(stats.Rooms, wantRooms) {
    t.Fatalf("rooms = %+v, want %+v", stats.Rooms, wantRooms)
  }
  // every timestamp sits inside hour zero, utc
  if stats.Busiest ≠ (wire.Stat{Key: "00:00", Count: 3}) {
    t.Fatalf("busiest = %+v", stats.Busiest)
  }
  // "for" and "are" die on the length filter, the rest tie at one
  wantWords := []wire.Stat{
    {Key: "waffles", Count: 2},
    {Key: "breakfast", Count: 1},
    {Key: "pancakes", Count: 1},
    {Key: "serious", Count: 1},
    {Key: "though", Count: 1},
  }
  if !reflect.DeepEqual(stats.TopWords, wantWords) {
    t.Fatalf("top words = %+v, want %+v", stats.TopWords, wantWords)
  }
}
```

listing 69: 3 documents anyone can read, every expected value counted from them by hand (source: infrastructure/capstone/internal/analytics/store_test.go, lines 55-91)

The analytics store runs the same discipline against its engine. Three documents, and every want literal derived from them by a person, away from the engine: the talker ranking from counting senders, the busiest hour from

the timestamps, the word ranking from applying the length filter by eye. The corpus is small enough to count in one glance, and that is what makes it an oracle. The same discipline reaches the fake store in the docker-free gate: the web suite posts 1 message and wants the fake's total and the duckdb panel to agree on 1, a number written down from the send itself, and the contract book 13, ch 17 (infrastructure: docker, databases, and testing, mocks, seeding, contract tests) holds from both sides.

The failure mode has one mechanic. Run the implementation, paste its output into the test, and the suite becomes a mirror: a green certifies that the code still does whatever it did on the day it was pasted, and a bug present at capture time is certified along with everything else. Mutation scores change meaning on such a suite. A mirror can kill 100 percent of mutants, because every mutant changes some output and the mirror flags each change, and the perfect score proves only that the suite tracks the code, not that the code meets the requirement. The requirement never entered the comparison. Mutation results mean something exactly when the oracle is independent of the implementation.

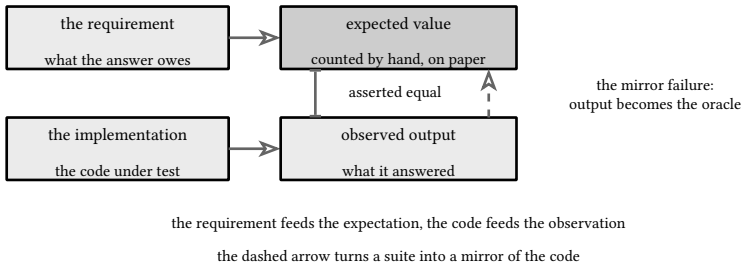


figure 50: where the expected value comes from, and the arrow that makes the suite a mirror

PITFALL: the direction of fit runs one way

Expectations are written from the requirement, before or while the code is written, never harvested from the finished output. The moment a wanted value is copied from what the implementation printed, the test stops being evidence about the requirement and becomes a record of the code's behavior, and no later green can tell the two apart. When the hand-derived oracle is out of reach, capture from a reference system and record the capture, so the next reader can tell an independent answer from a copied one.

sources: pkg.go.dev for the embedded nats server options and go's build constraints, accessed 2026-09-10. Verified by `make verify`, 131 go tests green, re-counted for the ch 19 samples at the v2.2 wave, and `make verify-infra-docker`, 4 tagged tests green, run 2026-09-20.

17. mocks, seeding, contract tests

The word mock covers too much. This chapter separates three tools that get confused: the embedded real thing, the fake at a seam, and the mock proper, and takes a position on when each one is honest.

17.1. the ladder, best first

The strongest test double is no double. The embedded nats server from book 13, ch 14 (infrastructure: docker, databases, and testing, nats: pub sub, request reply, queue groups) is the real dispatcher in-process, so subject routing, queue semantics, and jetstream behavior in the tests are the production behavior. Second best is a fake at a seam: the history `Store` interface is two methods, and the in-memory fake implements them with real aggregation logic, real grouping, real sorting. Weakest is the interaction mock, an object that records calls and replays expectations, and the capstone contains none, because every place a mock would go either has the real thing or has a fake with behavior.

The rule of thumb the capstone follows: mock or fake only at boundaries you do not own and cannot embed, and when you fake, fake behavior, not call sequences. A fake whose `Stats` groups and sorts the same way the real one does can be wrong in interesting ways, and those wrongs are assertions. A mock that expects "Insert then Stats" can only be wrong about what you already typed.

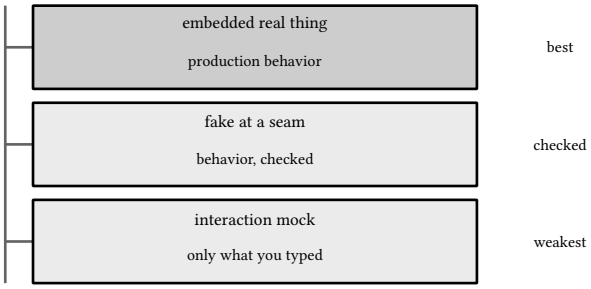


figure 51: the test-double ladder, best first, and where the capstone stands

17.2. the fake earns its keep by being checked

The fake is not trusted. Its aggregation is a transcription of the mongo pipelines, and two tests hold it to that. The first is a manual count against known input:

```
func TestFakeStatsMatchAManualCount(t *testing.T) {
  store := NewFakeStore()
  ts := time.UnixMilli(0).UTC() // hour 0
  for _, m := range []wire.Message{
    {ID: 1, Room: "general", Sender: "alice", Body: "waffles for breakfast", TS: ts.Add(0 *
time.Minute).UnixMilli()},
    {ID: 2, Room: "general", Sender: "bob", Body: "waffles are serious", TS: ts.Add(2 *
time.Minute).UnixMilli()},
    {ID: 3, Room: "random", Sender: "alice", Body: "pancakes though", TS: ts.Add(3 *
time.Minute).UnixMilli()},
  } {
    if err := store.Insert(context.Background(), m); err ≠ nil {
      t.Fatal(err)
    }
  }
  stats, err := store.Stats(context.Background())
  if err ≠ nil {
    t.Fatal(err)
  }
  if stats.Total ≠ 3 {
    t.Fatalf("total = %d", stats.Total)
  }
  if stats.Busiest.Key ≠ "00:00" || stats.Busiest.Count ≠ 3 {
    t.Fatalf("busiest = %+v", stats.Busiest)
  }
  // waffles appears twice, everything else once
  if stats.TopWords[0].Key ≠ "waffles" || stats.TopWords[0].Count ≠ 2 {
    t.Fatalf("top words = %+v", stats.TopWords)
  }
}
```

listing 70: three documents, hand computed totals, hour bucket, and word ranking (source: infrastructure/capstone/internal/history/history_test.go, lines 113-139)

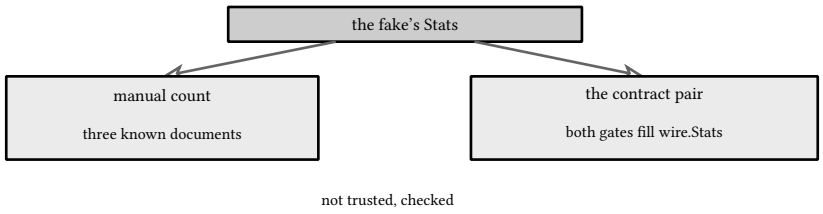


figure 52: the fake is not trusted: two checks hold its Stats honest

The second is the contract pair from book 13, ch 13 (infrastructure: docker, databases, and testing, mongodb and aggregation pipelines): the docker-free suite proves plumbing against the fake, the docker gate runs the same questions against real aggregation, and the wire shape, `wire.Stats`, is what both must fill identically.

17.3. seeding to reach the rare paths

Some behaviors only happen with history or damage already present, and waiting for a test to grow them is slow and flaky. Seeding is the answer, and the capstone seeds through the real interfaces, never behind their backs:

```
func TestConsumerSkipsAlreadyStoredIDs(t *testing.T) {
  _, nc := testnats.Run(t)
  store := NewFakeStore()
  // simulate a crash after insert but before ack by pre-seeding one
  // row that the stream will deliver again
  store.Insert(context.Background(), wire.Message{
    ID: 1, Room: "general", Sender: "alice", Body: "hello", TS: 1,
  })
  svc, err := New(nc, store, AckWait)
  if err ≠ nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { svc.Close() })

  seedChat(t, nc, wire.Message{ID: 1, Room: "general", Sender: "alice", Body: "hello", TS: 1})
  waitFor(t, "dedupe settles at 1", func() bool { return store.Len() == 1 })
}
```

listing 71: a pre-inserted row replays the crash between insert and ack (source: infrastructure/capstone/internal/history/history_test.go, lines 55-71)

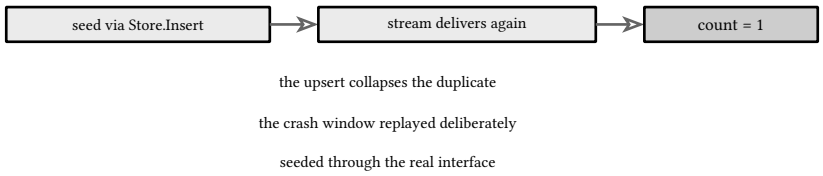


figure 53: seeding replays the crash window, through the real interface

That test simulates the exact crash window from book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) by inserting the row and then letting the stream deliver it again, and the assertion is that the count settles at one, not two. The seed went through `Store.Insert`, so the test reads as the scenario it is, a redelivery after a partial failure, instead of reaching into private state.

17.4. contract tests, names both sides answer to

Two services that talk over a subject agree on a string. A rename refactor will not break the string silently if the string is pinned once, in a constant, and asserted on both sides:

```
func TestRosterSubjectIsPresenceOnline(t *testing.T) {
  if RosterSubject ≠ "presence.online" {
    t.Fatalf("roster subject = %s", RosterSubject)
  }
  _ = bus.CallJSON // the same helper both sides use
}
```

listing 72: the subject the notifier calls is the subject the presence service serves (source: infrastructure/capstone/internal/notify/notify_test.go, lines 218-223)

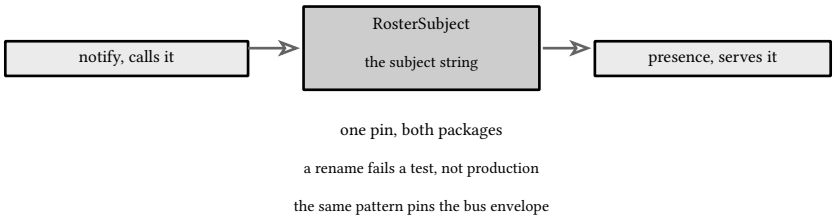


figure 54: one string, two packages, one pin

That is the whole contract test: cheap, honest, and it converted a would-be production outage, notify silently requesting a subject nobody serves, into a compile-adjacent failure. The same pattern pins the wire envelope in the bus package, where the round-trip test catches any encoding drift between caller and responder in one place.

NOTE: when a mock is the right call

The ladder has a top rung for mocks: third-party services you do not own, cannot embed, and pay by the call, an sms gateway, a payment provider. There the interface should be yours and thin, the fake minimal, and the mock confined to the adapter that speaks the vendor's protocol, because that adapter is the only code whose interaction with reality a mock can faithfully check. Nothing in the capstone reached that rung, which is itself the finding.

sources: no external sources, the positions here are argued from the capstone's own suites. Verified by the tests named in this chapter, the history suite's seeding and contract tests, 5 tests, and the notify roster pin, green under `make verify` 2026-09-10.

18. migrations, test driven

The migration runner is the one piece of the capstone built strictly test first in this book, red, green, refactor, one behavior at a time, and this chapter is that build order. The runner is not a sample: `store.Open` calls it on every open, so the capstone's schema lives or dies with these tests.

The contract, decided before any implementation: migrations are sql files named `NNNN_name.sql`, embedded in the binary, applied in order, each inside one transaction together with its version row, tracked in a table the runner creates, and a failure leaves the database at the previous version. Six tests encode it.

18.1. discovery, order, and the names that fail loudly

```
func TestPlanOrdersByVersionAndRejectsBadNames(t *testing.T) {
  fsys := fstest.MapFS{
    "0003_third.sql":  {Data: []byte("select 1;")},
    "0002_second.sql": {Data: []byte("select 1;")},
    "0001_first.sql":  {Data: []byte("select 1;")},
    "not_a_migration": {Data: []byte("junk")},
    "README.md":       {Data: []byte("docs")},
  }
  plan, err := Plan(fsys)
  if err != nil {
    t.Fatal(err)
  }
  if len(plan) != 3 {
    t.Fatalf("want 3 migrations, got %d: %+v", len(plan), plan)
  }
  want := []string{"0001", "0002", "0003"}
  for i, m := range plan {
    if m.Version != want[i] {
      t.Fatalf("plan[%d] version = %s, want %s", i, m.Version, want[i])
    }
  }

  bad := fstest.MapFS{"2_two.sql": {Data: []byte("select 1;")}}
  if _, err := Plan(bad); err == nil {
    t.Fatal("want error for name without 4 digit version")
  }
}
```

listing 73: shuffled files come out ordered, junk names are ignored, a misnamed sql file is an error (source: infrastructure/capstone/internal/migrate/migrate_test.go, lines 28-54)

The distinction in the last assertion is deliberate. A `README.md` among migrations is noise, skip it. A `2_two.sql` is a typo that would silently skip a migration forever, error. And a gap in the sequence, 0001 then 0003, is refused, because applied versions must be a prefix of a gapless sequence or resume logic cannot tell a missing file from a reordered one:

```
func TestUpRejectsVersionGaps(t *testing.T) {
  db, _ := openDB(t)
  gapped := fstest.MapFS{
    "0001_a.sql": {Data: []byte("create table a(x);")},
    "0003_c.sql": {Data: []byte("create table c(x);")},
  }
  _, err := Up(db, gapped)
  if err == nil {
    t.Fatal("want error for gap between 0001 and 0003")
  }
}
```

listing 74: gaps are errors, not surprises for later (source: infrastructure/capstone/internal/migrate/migrate_test.go, lines 152-162)

The implementation of discovery is one regexp and one sort:

```
func Plan(fsys fs.FS) ([]Migration, error) {
  entries, err := fs.ReadDir(fsys, ".")
  if err ≠ nil {
    return nil, err
  }
  var plan []Migration
  seen := map[string]bool{}
  for _, e := range entries {
    if e.IsDir() {
      continue
    }
    m := nameRe.FindStringSubmatch(e.Name())
    if m == nil {
      // a mistyped sql name must fail loudly, not silently skip the
      // migration inside it
      if strings.HasSuffix(e.Name(), ".sql") {
        return nil, fmt.Errorf("migrate: bad migration name %q, want NNNN_name.sql", e.Name())
      }
      continue
    }
    if seen[m[1]] {
      return nil, fmt.Errorf("migrate: duplicate version %s", m[1])
    }
    seen[m[1]] = true
    body, err := fs.ReadFile(fsys, e.Name())
    if err ≠ nil {
      return nil, err
    }
    plan = append(plan, Migration{Version: m[1], Name: m[2], SQL: string(body)})
  }
  sort.Slice(plan, func(i, j int) bool { return plan[i].Version < plan[j].Version })
  if err := checkGaps(plan); err ≠ nil {
    return nil, err
  }
  return plan, nil
}
```

listing 75: plan: read the fs, match names, reject bad sql names, order by version (source: infrastructure/capstone/internal/migrate/migrate.go, lines 35-70)

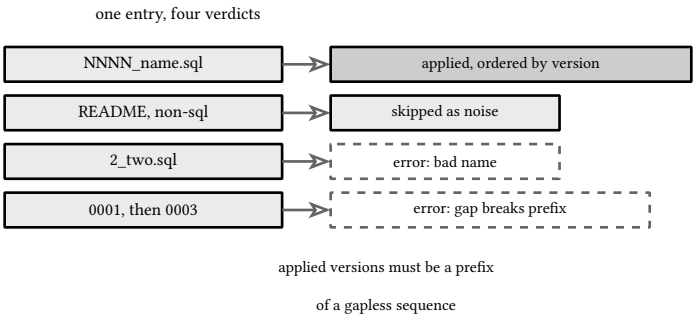


figure 55: what plan does to each file: two of the four verdicts are errors

18.2. apply, record, and the atomicity that matters

```
func Up(db *sql.DB, fsys fs.FS) ([]string, error) {
  if _, err := db.Exec(`create table if not exists schema_migrations (
    version    text primary key,
    applied_at text not null default (datetime('now'))
  )`); err != nil {
    return nil, err
  }
  appliedSet := map[string]bool{}
  rows, err := db.Query("select version from schema_migrations")
  if err != nil {
    return nil, err
  }
  for rows.Next() {
    var v string
    if err := rows.Scan(&v); err != nil {
      rows.Close()
      return nil, err
    }
    appliedSet[v] = true
  }
  if err := rows.Err(); err != nil {
    rows.Close()
    return nil, err
  }
  rows.Close()

  plan, err := Plan(fsys)
  if err != nil {
    return nil, err
  }
  var applied []string
  for _, m := range plan {
    if appliedSet[m.Version] {
      continue
    }
    tx, err := db.Begin()
    if err != nil {
      return nil, err
    }
    if _, err := tx.Exec(m.SQL); err != nil {
      tx.Rollback()
      return nil, fmt.Errorf("migrate %s_%s: %w", m.Version, m.Name, err)
    }
    if _, err := tx.Exec(
      "insert into schema_migrations(version) values (?)", m.Version); err != nil {
      tx.Rollback()
      return nil, fmt.Errorf("migrate %s_%s: record version: %w", m.Version, m.Name, err)
    }
    if err := tx.Commit(); err != nil {
      return nil, err
    }
    applied = append(applied, m.Version)
  }
  return applied, nil
}
```

listing 76: up: the tracking table, the applied set, each migration and its version row in one transaction (source: infrastructure/capstone/internal/migrate/migrate.go, lines 93-147)

The transaction holds two things at once, the schema change and the version insert. That pairing is the entire correctness claim, and the failure test is where it earns belief:

```
func TestFailedMigrationLeavesNoVersionRow(t *testing.T) {
  db, _ := openDB(t)
  broken := fstest.MapFS{
    "0001_init.sql":  {Data: []byte("create table keepme(x);")},
    "0002_broke.sql": {Data: []byte("this is not sql;")},
  }
  if _, err := Up(db, broken); err == nil {
    t.Fatal("want error from broken migration")
  }
  var n int
  if err := db.QueryRow(
    "select count(*) from schema_migrations").Scan(&n); err ≠ nil {
    t.Fatal(err)
  }
  if n ≠ 1 {
    t.Fatalf("recorded %d versions after failure, want 1", n)
  }
  var kept int
  if err := db.QueryRow("select count(*) from keepme").Scan(&kept); err ≠ nil {
    t.Fatalf("first migration not durable: %v", err)
  }
}
```

listing 77: a broken migration leaves one version row and the first migration's table durable (source: infrastructure/capstone/internal/migrate/migrate_test.go, lines 129-150)

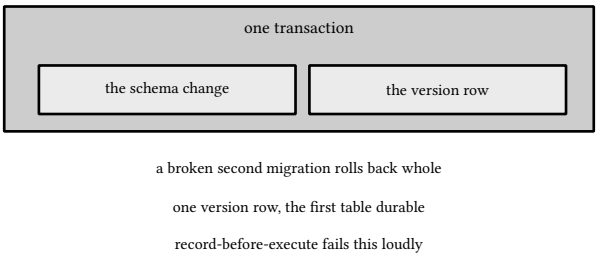


figure 56: one transaction holds both writes, so failure leaves no half state

A runner that recorded the version before executing, or executed outside the transaction, fails this test in the loudest way: the second run would skip a migration that never landed. Idempotence and resumption are the two remaining behaviors, one second call applying nothing, one partial database applying only the tail, and their tests drive the applied-set read at the top of `Up`.

18.3. why not a library

Go has migration libraries with more features, down migrations, checksums, dry runs. The capstone carries its own runner for the reason the chapter is in this book: schema evolution is a handful of policy rules, the rules are the interesting part, and the parts a library would add are the parts this system decided against. Down migrations, because forward-only with a copy-and-swap rebuild for the rare undo is safer than generated reverse sql. Checksums on applied files, because the gap rule plus the version table already answer the questions the schema asks.

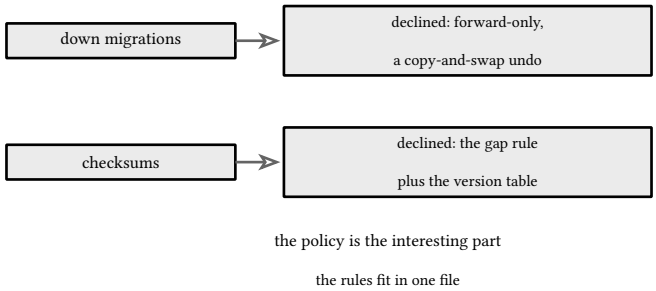


figure 57: own runner versus library: the ledger of declined features

WARNING: the migration is the deployment

`store.Open` runs `Up` on every open, which means every service start is a migration attempt. That is safe exactly because Up is idempotent and transactional, and it removes an entire class of "forgot to run migrations on the new node" incidents. The cost is that migrations must stay additive and fast, the discipline book 13, ch 6 (infrastructure: docker, databases, and testing, sqlite: schemas and normalization) already argued for.

sources: no external sources, the design follows from the sqlite transaction semantics probed in book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation). Verified by the migrate suite, 6 tests, red first then green, under `make verify` 2026-09-10.

19. version control at scale

The last storage engine in the book is the one holding the book's own source. Source control is a write-ahead log of changes, a content-addressed blob store, and derived read structures built off the log, three shapes this book has already built once each: the log in book 13, ch 8 (infrastructure: docker, databases, and testing, sqlite: wal, locking, crash semantics), the forward-only version sequence in book 13, ch 18 (infrastructure: docker, databases, and testing, migrations, test driven). This chapter builds them once more at the scale where git gives out, reads one system end to end, Anthropic's Scion, and carries its data model in a worked sample, an in-memory changeset store under `samples/ch19` held to the same suite discipline as every other engine chapter.

19.1. git's object model, in brief

Git keeps four object kinds, every one content-addressed. A blob is file bytes. A tree maps names to blob and tree digests, one whole directory snapshot. A commit points at one tree plus parent ids. Refs are the only mutable thing in the model, names pointing at commits. A clone carries all four kinds for the entire history, and that completeness is the design: diff, blame, log, and checkout all compute offline because the whole past is local.

This corpus rides that model on every build. The ci lanes of book 4, ch 29 (go 1.27, ship it) and book 6, ch 33 (c# 15, f# 11, and .net 11, ship it) start each Actions job with a checkout, and at one job per push the checkout is noise. The counts in the next section are what the same checkout becomes when most of the writers are agents.

19.2. the failure, measured

Anthropic's engineering post on Scion supplies the numbers, and every one below is transcribed from its bytes. Merged changes grew from around 2,500 per day in May 2026 to over 30,000 per day by September, and the post calls the latent demand for merging code practically infinite, so it expects no slowdown. Against that write rate sit four more counts: the history of the main branch alone is almost 100 gb, a compressed working copy is over 25 gb, ci performs on the order of 25 million code checkouts per day before counting developer environments, agent sandboxes, and research jobs, and fetched naively that is over 600 pb of git traffic per day.

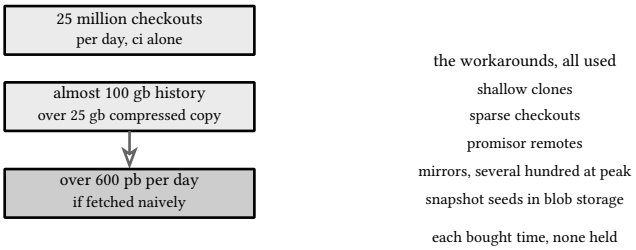


figure 58: one day of ci checkouts against one repo, priced the way a git clone pays

The workarounds ran out together. The post reports using all of them, shallow clones, sparse checkouts, promisor remotes, internal mirrors, repository snapshots kept in blob storage, with several hundred internal git mirrors at peak. Mirrors grew harder to operate as the merge rate rose, seeds fell further from the tip of main between refreshes, and history-dependent work like diff and blame kept needing more care on partial clones.

The post's diagnosis is the one to keep. The company was running git as a centralized system with one authoritative copy, while nearly all of its clients were agents and jobs that are online by definition. The properties that make git excellent for distributed offline work were the same properties undermining scale, so the design question became: what does source control look like when the client is always online?

19.3. the changeset, a wal entry

Scion's unit of work is the changeset, an entry in a write-ahead log. It carries what a git commit carries except the tree pointer: a list of the files that changed relative to its first parent, plus content-addressed pointers to

the compressed file contents in blob storage. Changesets are created server-side and are available everywhere instantly, and accepting one touches nothing global, no packfile to build, no lock, no cross-server coordination, no ref to move. The post's footnote is candid about the first-parent choice: in a trunk-based monorepo people only ask about history and blame on the main line, so recording changes against the first parent gives up nothing anyone misses.

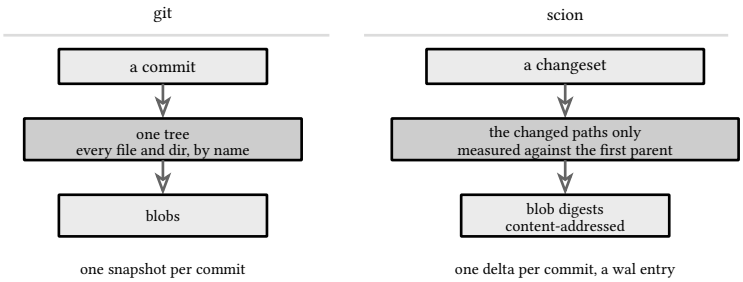


figure 59: what a commit points at, git on the left, scion on the right

The sample store is that data model made small, one mutex in place of spanner, a map in place of blob storage:

```
type Change struct {
  Path   string
  Digest Digest
}

type Changeset struct {
  ID      int64
  Parent  int64
  Changes []Change
}

type Store struct {
  mu         sync.Mutex
  blobs      map[Digest][]byte
  changesets []Changeset
  refs       map[string]int64
}

func NewStore() *Store {
  return &Store{
    blobs: make(map[Digest][]byte),
    refs:  make(map[string]int64),
  }
}

func (s *Store) Put(data []byte) Digest {
  sum := sha256.Sum256(data)
  d := Digest(hex.EncodeToString(sum[:]))
  s.mu.Lock()
  if _, ok := s.blobs[d]; !ok {
    buf := make([]byte, len(data))
    copy(buf, data)
    s.blobs[d] = buf
  }
  s.mu.Unlock()
  return d
}
```

listing 78: change, changeset, and put: sha256 over the bytes, stored once, defensive copy in and out (source: infrastructure/samples/ch19/store.go, lines 33-69)

`Put` is the whole addressing scheme. The digest is the sha256 of the content, identical content stores once, and the bytes are copied on the way in and on the way out. Immutability is what every cache tier later in this chapter lives on, in the post's words: "a piece of data never changes once it's stored, and any server can cache it without checking with the others."

```
func (s *Store) Commit(parent int64, changes []Change) (int64, error) {
  s.mu.Lock()
  defer s.mu.Unlock()
  if parent ≠ NullID && !s.stored(parent) {
    return NullID, ErrUnknownChangeset
  }
  sorted := slices.Clone(changes)
  slices.SortFunc(sorted, func(a, b Change) int { return cmp.Compare(a.Path, b.Path) })
  for i := 1; i < len(sorted); i++ {
    if sorted[i].Path == sorted[i-1].Path {
      return NullID, ErrDuplicatePath
    }
  }
  for _, c := range sorted {
    if _, ok := s.blobs[c.Digest]; !ok {
      return NullID, ErrMissingBlob
    }
  }
  id := int64(len(s.changesets)) + 1
  s.changesets = append(s.changesets, Changeset{ID: id, Parent: parent, Changes: sorted})
  return id, nil
}
```

listing 79: commit: validate the parent, sort by path, reject duplicates and unstored digests, append with a sequential id (source: infrastructure/samples/ch19/store.go, lines 83-104)

`Commit` validates, canonicalizes, appends. The parent must exist, the changes come out sorted by path, a duplicate path is a rejection, every digest must already be stored, and ids run sequentially from 1, the same gapless forward-only discipline the migration runner enforces in book 13, ch 18 (infrastructure: docker, databases, and testing, migrations, test driven). The append is the wal property verbatim: the cost of a commit scales with the size of the change, never with the size of the repository. No tree is materialized anywhere, not on commit, not on read.

19.4. refs as compare and swap

Refs are the one mutable thing in git and the one piece of global state in Scion, which is why the post discourages them: global state that every server has to agree on is a looming scaling problem, so the development lifecycle keeps to the main ref and release branches. Creating and fetching changesets never involves a ref, so moving one is a compare-and-swap on a single spanner row, off the hot path entirely.

```
func (s *Store) UpdateRef(name string, expect, next int64) error {
  s.mu.Lock()
  defer s.mu.Unlock()
  if !s.stored(next) {
    return ErrUnknownChangeset
  }
  have, ok := s.refs[name]
  if !ok {
    have = NullID
  }
  if have ≠ expect {
    return &RefConflict{Name: name, Have: have, Wanted: expect}
  }
  s.refs[name] = next
  return nil
}
```

listing 80: update ref: absent plus expect null creates, a mismatch reports the observed value (source: infrastructure/samples/ch19/store.go, lines 116-131)

An absent ref combined with `expect == NullID` creates it, anything else compares. A mismatch returns a `*RefConflict` carrying the observed `Have`, and that field is the entire retry protocol: the caller re-attempts the swap with the reported value instead of guessing. The race test pins the contract:

```
func TestConcurrentCASExactlyOneWinner(t *testing.T) {
  const racers = 8
  st := NewStore()
  ids := commitChain(t, st, racers+1)
  if err := st.UpdateRef("race", NullID, ids[0]); err ≠ nil {
    t.Fatal(err)
  }
  start := make(chan struct{})
  var wg sync.WaitGroup
  var mu sync.Mutex
  var nilErrs int
  var conflicts []*RefConflict
  for i := 0; i < racers; i++ {
    wg.Add(1)
    go func(i int) {
      defer wg.Done()
      <-start
      next := ids[i+1]
      err := st.UpdateRef("race", ids[0], next)
      mu.Lock()
      defer mu.Unlock()
      if err == nil {
        nilErrs++
        return
      }
      var conflict *RefConflict
      if !errors.As(err, &conflict) {
        t.Errorf("racer %d got %v, want *RefConflict", i, err)
        return
      }
      conflicts = append(conflicts, conflict)
    }(i)
  }
  close(start)
  wg.Wait()
  if nilErrs ≠ 1 {
    t.Fatalf("%d racers won the CAS, want exactly 1", nilErrs)
  }
  if len(conflicts) ≠ racers-1 {
    t.Fatalf("%d racers conflicted, want %d", len(conflicts), racers-1)
  }
  final, _ := st.Ref("race")
  for _, c := range conflicts {
    if c.Have ≠ final {
      t.Fatalf("conflict reported have %d, final ref %d: losers must see the winner's value", c.Have,
final)
    }
  }
}
```

listing 81: 8 racers, one cas expectation, exactly one winner, every loser told the winner's value (source: infrastructure/samples/ch19/refs_test.go, lines 139-186)

Eight goroutines race one expectation, exactly one wins, 7 read conflicts, and every conflict reports the winner's value, which is what lets a loser retry without a re-read. The mutex stands in for the spanner row, and the guarantee is the same one a single-row cas gives at fleet size.

PITFALL: retry with the reported value, never the cached one

> A retry that reuses its own stale expectation conflicts again, so a loop that never adopts the conflict's `Have` spins forever. The sample's `TestCASRetryWithReportedHave` pins exactly this path: second call with the old expectation fails, third call with `conflict.Have` lands. The conflict type exists to carry that field, an error string without `Have` forces every caller back to a read that can race again.

19.5. derived data, blame and path history

A changeset records only what changed, so on its own it cannot answer who last touched a path or which changesets touched it. Scion derives those answers from the log: dedicated structures for file blame and path history, and skip lists over the commit graph. Each type of derived data is defined recursively, its value at a changeset computed from its values at that changeset's parents, each type computed eagerly or deferred as needed, and none of it visible to clients, so the internals can be reoptimized without touching a single client. Copy detection and path-based file search joined later as new derived types, and the post reports adding new types this way more than once as the system grew.

The sample derives the same three answers from the same recursion:

```
func (s *Store) chain(id int64) ([]Changeset, error) {
  s.mu.Lock()
  defer s.mu.Unlock()
  if !s.stored(id) {
    return nil, ErrUnknownChangeset
  }
  var out []Changeset
  for cur := id; cur ≠ NullID; {
    cs := s.changesets[cur-1]
    out = append(out, cs)
    cur = cs.Parent
  }
  return out, nil
}

func (s *Store) Snapshot(id int64) (map[string]Digest, error) {
  history, err := s.chain(id)
  if err ≠ nil {
    return nil, err
  }
  snap := make(map[string]Digest)
  for i := len(history) - 1; i ≥ 0; i-- {
    for _, c := range history[i].Changes {
      snap[c.Path] = c.Digest
    }
  }
  return snap, nil
}
```

listing 82: chain walks first parents to the root, snapshot replays oldest to newest, each change overwriting its path (source: infrastructure/samples/ch19/query.go, lines 3-30)

`chain` walks first parents to the root, and `Snapshot` replays that chain from the oldest entry forward, each change overwriting its path's digest. The resulting map is the working copy: the state a checkout would materialize, computed from the log, stored nowhere.

```
func (s *Store) PathHistory(id int64, path string) ([]int64, error) {
  history, err := s.chain(id)
  if err != nil {
    return nil, err
  }
  var hits []int64
  for _, cs := range history {
    for _, c := range cs.Changes {
      if c.Path == path {
        hits = append(hits, cs.ID)
        break
      }
    }
  }
  return hits, nil
}

func (s *Store) Blame(id int64) (map[string]int64, error) {
  history, err := s.chain(id)
  if err != nil {
    return nil, err
  }
  blame := make(map[string]int64)
  for _, cs := range history {
    for _, c := range cs.Changes {
      if _, seen := blame[c.Path]; !seen {
        blame[c.Path] = cs.ID
      }
    }
  }
  return blame, nil
}
```

listing 83: path history names every changeset that touched a path, newest first, blame assigns each path its last writer (source: infrastructure/samples/ch19/query.go, lines 32-63)

`PathHistory` walks newest first and names every changeset that touched a path. `Blame` assigns each path the id of its last writer. Every expected value in the suite is counted by hand from the fixture, the oracle discipline of book 13, ch 16 (infrastructure: docker, databases, and testing, testing without docker):

```
func TestBlameLastWriterPerPath(t *testing.T) {
  fx := buildChain(t)
  tests := []struct {
    at   int64
    want map[string]int64
  }{
    {2, map[string]int64{"a": 2, "b": 1}},
    {3, map[string]int64{"a": 2, "b": 1, "c": 3}},
    {4, map[string]int64{"a": 4, "b": 4, "c": 3}},
  }
  for _, tt := range tests {
    got, err := fx.store.Blame(tt.at)
    if err ≠ nil {
      t.Fatal(err)
    }
    if !reflect.DeepEqual(got, tt.want) {
      t.Fatalf("Blame(%d) = %v, want %v", tt.at, got, tt.want)
    }
  }
}
```

listing 84: the blame oracle: 3 snapshots over a 4 changeset chain, each map derived from the fixture by eye (source: infrastructure/samples/ch19/query_test.go, lines 138-157)

NOTE: path granularity, on purpose

The sample's blame works at path granularity with whole-file digests, so every line of a path blames to that path's last writer. Line-level blame needs the blob diffed against its parent, and that is exactly the work Scion pushes into derived data beside the log rather than into the changeset itself. The sample also keeps the recursion and drops the increment: every query walks the whole chain, where Scion's derived structures are maintained per changeset. The recursion is the design, the index is the optimization, and the chapter's sample stops at the design.

19.6. three tiers, one rpc shape

Moving file contents is the data-intensive half, and Scion's answer is a cache hierarchy in front of the storage of record. The scn client talks over a socket to scion-daemon, one per node, which stores objects for the whole node, since jobs on one node reuse most of the same objects, and starts from a periodically refreshed snapshot. The daemon talks only to the tree and blob cache servers in its region, scion-cache, which stream objects straight from memory in the disk format the daemon consumes, so a cache spends no time computing a client's missing objects and none on delta compression. A cache missing an object asks the authoritative scion-servers, which read from spanner or cloud storage as needed. All three tiers answer identical rpcs, so the client never needs to know the cache topology, and the tiering is strictly enforced anyway to isolate failure domains: a failing regional cache must never be able to overwhelm the authoritative servers and take the whole system down.

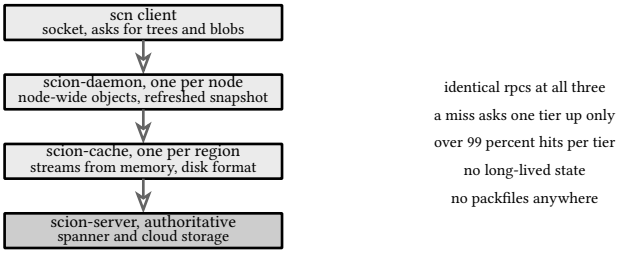


figure 60: the tier ladder, one rpc shape at every hop, misses go up exactly one tier

The protocol change that makes the hierarchy cheap is the negotiation shape. Git's fetch negotiates haves: the client announces what it holds, the server computes a packfile against that, one multi-gigabyte response that cannot resume. Scion's scn client negotiates wants: starting from a root tree, the client checks its local store for missing trees and requests them recursively, then repeats for blobs. Every request names one immutable object, so any cache at any tier can answer it, and the fetch resumes wherever it stopped because there is no single response to lose. Sparse checkouts fall out of the same shape, the client just does not ask for trees outside its scope, and virtualized checkouts materialize files on demand through the same mechanism.

The working copy itself is optional. A ci job that needs the diff between a commit and its merge base asks the server for it, and write workloads can function entirely through the api, the post's example being automated bisect and revert pipelines that would be impractical if each needed its own repository copy. Permissions live at directory level inside the one monorepo, and the review tooling expresses semantic constraints on approvals, separating comment-only changes from sensitive logical ones, so human reviewers spend attention where it matters.

The workflow conclusion is the sharpest break in the post. A merge queue serializes merges behind a global repository lock, and at agent scale that lock is the bottleneck: hundreds or thousands of changes can merge while tests are still running. Anthropic could not replicate that lock at its merge rate, so it invests in post-merge and pre-deploy automated testing instead, accepts that concurrent changes conflict, and reports that Claude fixes those conflicts faster than humans can even identify them. The per-push ci gates of book 4, ch 29 (go 1.27, ship it) are the human-scale form of the same question, and the post's answer at fleet scale is to test after the merge and gate the deploy.

19.7. the migration, measured

The results, every count from the post's bytes. The Scion infrastructure footprint is 200x smaller than the git mirroring infrastructure it replaced, in cpu and memory terms. Every tier holds a cache hit rate over 99 percent with no long-lived state. Ci and research jobs check out code two orders of magnitude faster, in the post's own phrasing "in seconds rather than minutes", and the bytes never print a literal 100x for checkouts, so neither does this chapter. Ci alone realized a 25 percent cpu capacity win. Neither clients nor caches perform repository maintenance, there are no packfiles, so there is nothing to gc.

The build is itself a data point about agent-scale engineering. 7 people built Scion, the Antique code review tool, and half a dozen related systems, and fully migrated the monorepo off git in 5 months, while the volume of code merged per day grew 10x under them. Asked to rewrite the earlier handwritten git-to-Scion ingestion pipeline, an internal model snapshot produced a version on the first try that consumed 20x less memory and ran 4x faster while preserving the ingestion-ordering invariants. The post is equally plain about the division of labor: agents wrote the code, and the engineers supplied the design-level judgment, very often the judgment to tell Claude no when it over-engineered in response to hypothetical concerns from adversarial review.

19.8. piper, sapling, and what oss can adopt

Google hit the same wall a decade earlier and published the answer. Potvin and Levenberg's cacm article, 59(7), july 2016, describes Piper: one repository of roughly 1 billion files with a history of roughly 35 million commits, 86 tb uncompressed, shared by more than 25,000 developers who commit 16,000 changes on a typical workday while automated systems commit another 24,000. No commercial or open source version-control system could hold that scale in one repository, so Piper runs on standard Google infrastructure, originally bigtable, now spanner, distributed over 10 data centers and kept consistent with paxos. Developers work in CitC, cloud storage under a fuse file system, where an average workspace holds fewer than 10 files against a seamless view of the whole codebase, and over 80 percent of Piper users work that way. Trunk-based development, one main line, review before commit: Scion holds every one of these positions, and the post names Piper and Sapling as the prior art it built on.

Meta's Sapling, open sourced 2022-11-15 by Goode and Bolin, attacked the client first: 10 years of work that started as an extension to Mercurial, now serving an internal repository with tens of millions of files, tens of millions of

commits, and tens of millions of branches. Two of its ideas reappear in Scion nearly whole. Lazy history, a clone downloads almost none of it, because three-quarters of the 5.5 gb Linux kernel repo is history, with the segmented changelog shipping the shape of the commit graph in a few megabytes, ancestry queries in o(number-of-merges) time, and file log and blame in o(log n) instead of o(n). And the working copy, virtual or sparse: a virtual file system that materializes files on first access, plus org-committed sparse profiles so thousands of engineers share one shifting subset of the repo without configuring it themselves. That first release open sourced the client only, the server and the virtual file system stayed internal.

The post closes with what transfers today. Scion is no git replacement for general use, it only pays off at truly enormous monorepos with tens of thousands of engineers or swarms of agents, and adopting it means migrating review, search, and ci along with the vcs, which is why Anthropic shared the design instead of shipping a product. Inside git the ideas still apply: a git server does not have to work the way a git client does, server architectures exist that scale much better, forges can expose the full source-control query surface as server-side rpcs, perhaps standardized across forges, negotiation can become want-shaped, and the sparse and shallow features can be made to compose. The post expects this class of problems to become commonplace within 6 to 12 months. The checkout at the top of every Actions job is where it arrives first.

VERIFY: the store is pinned by its suite

The sample carries 31 test functions across 4 files, 7 in store_test.go, 8 in changeset_test.go, 8 in refs_test.go, 8 in query_test.go, green under `go vet` and `go test -race` run 2026-10-15. The counts are measured from the live tree, and the suite keeps this book's oracle discipline: every expected digest, snapshot, path history, and blame value is derived from the fixture by hand, never harvested from output.

sources: Anthropic engineering, "Redesigning source control at Anthropic", published 2026-10-13, https://www.anthropic.com/engineering/redesigning-source-control, accessed 2026-10-15. Potvin and Levenberg, "Why Google Stores Billions of Lines of Code in a Single Repository", cacm 59(7), july 2016, https://cacm.acm.org/practice/why-google-stores-billions-of-lines-of-code-in-a-single-repository/, accessed 2026-10-15. Goode and Bolin, "Sapling: Source control that's user-friendly and scalable", published 2022-11-15, https://engineering.fb.com/2022/11/15/open-source/sapling-source-control-scalable/, accessed 2026-10-15. All three cached verbatim under `ref/scion/` with sha256 receipts in its README. Absent from the bytes and therefore from this chapter: any literal checkout multiplier beyond "two orders of magnitude", any scion fleet size, and any per-tier cache cost, the post gives none.

20. capstone part 1: services and topology

Everything before this chapter was one layer at a time. The capstone composes them: six Go services, one broker, one database server, two embedded engines, one connection each, brought up by a single compose command. This chapter is the topology and the service interiors, the next is the interface on top.

20.1. the shape

service	speaks	does	owns
chat	chat.send, chat.recent, publishes chat.room.<room>	validates, persists, publishes after commit	the sqlite file
presence	presence.beat, presence.online, presence.events	kv puts with ttl, roster, join and leave sweep	the kv bucket
notify	durable notifiers on CHAT	fans notifications to everyone but the sender	no state
history	durable historians on CHAT, history.stats	upserts documents, aggregates	the mongo collection
analytics	durable analysts on CHAT, analytics.stats	upserts rows into duckdb, sql aggregates	the duckdb file
web	http on 8080, all fragments	the only public surface, per user inboxes	nothing durable

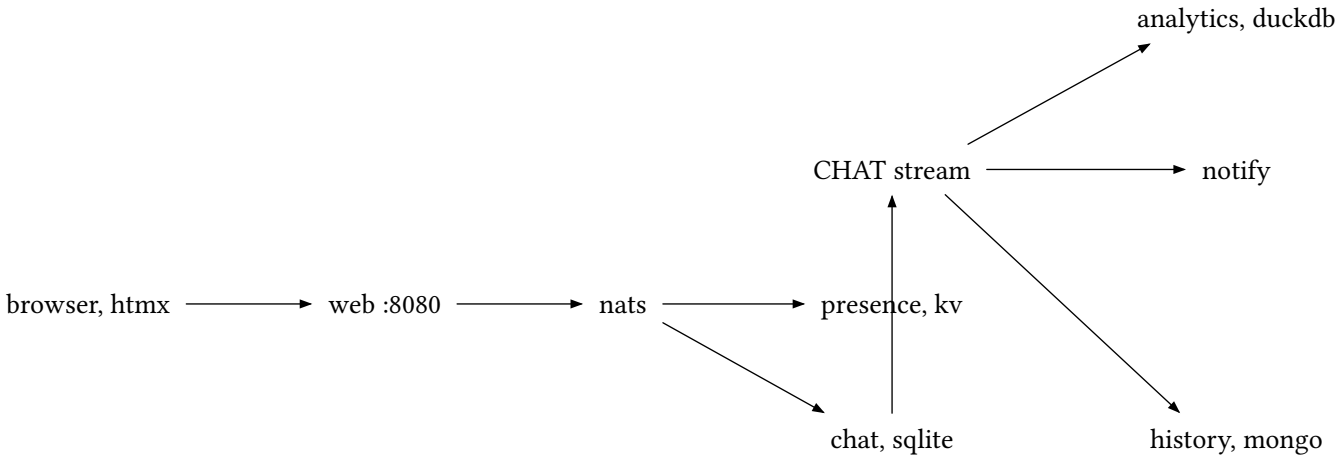


figure 61: the system, one message's path

The load-bearing decisions are three. Every service boundary is a subject, so services never address each other, the broker does. Everything durable is behind a service that owns exactly one store, one file for chat, one duckdb file for analytics, one bucket for presence, one collection for history, so no two writers share a lock domain. And the web tier owns nothing durable at all, which is why it can be scaled, restarted, or replaced without a migration in sight.

20.2. the wire, one page

Every request and reply crossing those subjects is one of six shapes, defined once:

```
type Message struct {
  ID     int64  `json:"id"`
  Room   string `json:"room"`
  Sender string `json:"sender"`
  Body   string `json:"body"`
  TS     int64  `json:"ts"`
}
```

listing 85: the message, the unit the whole system moves (source: infrastructure/capstone/internal/wire/wire.go, lines 10-17)

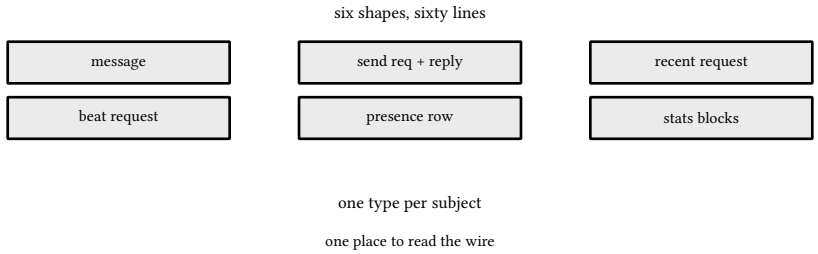


figure 62: the six wire shapes, one type per subject

The full set lives in sixty lines: the send and recent requests, the heartbeat, the roster entry, and the stats blocks. One type per subject keeps the contract tests from book 13, ch 17 (infrastructure: docker, databases, and testing, mocks, seeding, contract tests) meaningful, because there is exactly one place to read what crosses the wire.

20.3. the chat interior

The chat service is the book's sqlite argument in miniature. One transaction resolves or inserts the names, inserts the message, and commits. Only after the commit does the publish happen:

```
func (s *Service) Send(user, room, body string) (id int64, ts int64, err error) {
  if user == "" || room == "" || body == "" {
    return 0, 0, ErrValidation
  }
  ts = time.Now().UnixMilli()
  tx, err := s.db.Begin()
  if err ≠ nil {
    return 0, 0, err
  }
  defer tx.Rollback()
  uid, err := ensureName(tx, "users", user)
  if err ≠ nil {
    return 0, 0, err
  }
  rid, err := ensureName(tx, "rooms", room)
  if err ≠ nil {
    return 0, 0, err
  }
  res, err := tx.Exec(
    "insert into messages(room_id, user_id, body, ts) values (?, ?, ?, ?)",
    rid, uid, body, ts)
  if err ≠ nil {
    return 0, 0, err
  }
  id, err = res.LastInsertId()
  if err ≠ nil {
    return 0, 0, err
  }
  if err := tx.Commit(); err ≠ nil {
    return 0, 0, err
  }
  // only after commit does the event exist, and only the committed
  // shape of it
  payload, err := json.Marshal(wire.Message{
    ID: id, Room: room, Sender: user, Body: body, TS: ts,
  })
  if err ≠ nil {
    return id, ts, err
  }
  if err := s.nc.Publish("chat.room."+room, payload); err ≠ nil {
    return id, ts, err
  }
  return id, ts, nil
}
```

listing 86: validate, one transaction, then publish the committed shape (source: infrastructure/capstone/internal/chat/chat.go, lines 92-135)

The order is the concurrency contract: anything reacting to the event can read the row, a property the chat suite pins directly by querying inside the subscription callback. Reversing the two lines would create a consumer that races its own database, the kind of bug that shows up as a flaky downstream test and nowhere else.

The reads walk the normalized join from book 13, ch 6 (infrastructure: docker, databases, and testing, sqlite: schemas and normalization):

```
func (s *Service) Recent(room string, limit int) ([]wire.Message, error) {
  if room == "" {
    return nil, ErrValidation
  }
  if limit <= 0 || limit > 200 {
    limit = 50
  }
  rows, err := s.db.Query(`
    select m.id, r.name, u.name, m.body, m.ts
    from messages m
    join rooms r on r.id = m.room_id
    join users u on u.id = m.user_id
    where r.name = ?
    order by m.id desc
    limit ?`, room, limit)
  if err ≠ nil {
    return nil, err
  }
  defer rows.Close()
  msgs := []wire.Message{}
  for rows.Next() {
    var m wire.Message
    if err := rows.Scan(&m.ID, &m.Room, &m.Sender, &m.Body, &m.TS); err ≠ nil {
      return nil, err
    }
    msgs = append(msgs, m)
  }
  if err := rows.Err(); err ≠ nil {
    return nil, err
  }
  // newest first from the index, oldest first for the reader
  for i, j := 0, len(msgs)-1; i < j; i, j = i+1, j-1 {
    msgs[i], msgs[j] = msgs[j], msgs[i]
  }
  return msgs, nil
}
```

listing 87: newest of one room off the (room_id, id) index, reversed for the reader (source: infrastructure/capstone/internal/chat/chat.go, lines 169-204)

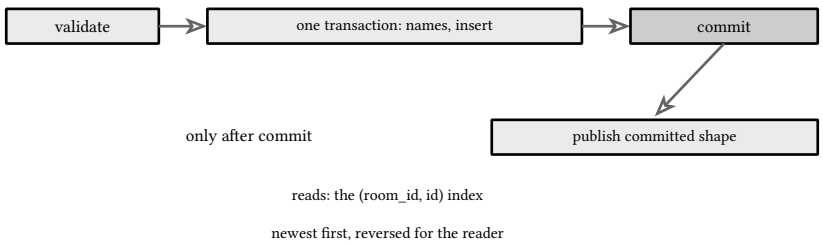


figure 63: send is a strict sequence, and the publish waits for the commit

20.4. the analytics interior, the columnar twin

History mirrors the stream into mongo documents and answers `history.stats` with pipelines. Analytics is the same shape pointed at the other engine: its own durable consumer over the same CHAT stream, rows landing in one duckdb file, stats answered by sql the engine folds vectorized. The store has no interface seam, because the engine itself is cheap to embed, so the tests run the real sql against a real file and never a fake of it. The whole schema and query set is constants:

```
const schemaSQL = `CREATE TABLE IF NOT EXISTS messages (
  id     BIGINT PRIMARY KEY,
  room   VARCHAR NOT NULL,
  sender VARCHAR NOT NULL,
  body   VARCHAR NOT NULL,
  ts     BIGINT NOT NULL
);
CREATE TABLE IF NOT EXISTS schema_meta (
  name  VARCHAR PRIMARY KEY,
  value VARCHAR NOT NULL
);`

// insertSQL upserts by id: a redelivery after a crash between insert
// and ack rewrites the same row instead of appending a duplicate.
const insertSQL = `INSERT INTO messages (id, room, sender, body, ts)
VALUES (?, ?, ?, ?, ?)
ON CONFLICT (id) DO UPDATE SET
  room = excluded.room,
  sender = excluded.sender,
  body = excluded.body,
  ts = excluded.ts`

// talkersSQL and roomsSQL are the $group/$sort/$limit counterpart:
// group, count descending with the key as tiebreaker, cut at three.
const talkersSQL = `SELECT sender, count(*) AS n
FROM messages
GROUP BY sender
ORDER BY n DESC, sender
LIMIT 3`

const roomsSQL = `SELECT room, count(*) AS n
FROM messages
GROUP BY room
ORDER BY n DESC, room
LIMIT 3`

// busiestSQL buckets the unix millisecond ts into a "HH:00" UTC label,
// the strftime analog of mongo's $toDate plus $dateToString pair.
const busiestSQL = `SELECT strftime(epoch_ms(ts), '%H:00') AS hour, count(*) AS n
FROM messages
GROUP BY hour
ORDER BY n DESC, hour
LIMIT 1`

// wordsSQL is the top-words pipeline in one statement: lowercase, split
// on the space, unnest the list into rows, trim the punctuation mongo
// trims, drop words of three letters or fewer, group, sort, cut at
// five.
const wordsSQL = `WITH split AS (
  SELECT unnest(string_split(lower(body), ' ')) AS raw
  FROM messages
)
SELECT word, count(*) AS n
FROM (
  SELECT trim(raw, '.,!?;:''"()[]-') AS word
  FROM split
) trimmed
```

```
WHERE length(word) > 3
GROUP BY word
ORDER BY n DESC, word
LIMIT 5`
```

listing 88: one table mirroring the wire message, an upsert by id, and the four aggregations as plain sql (source: infrastructure/capstone/internal/analytics/store.go, lines 27-87)

The insert upserts by message id for the same reason mongo's does: a redelivery after a crash between insert and ack rewrites the same row instead of appending a duplicate. Open is where the engine gets its seat:

```
// Open creates or opens the database at path and applies the schema,
// refusing files stamped with a schema version newer than this code.
func Open(path string) (*Store, error) {
  db, err := sql.Open("duckdb", path+"?threads=2")
  if err ≠ nil {
    return nil, err
  }
  db.SetMaxOpenConns(1)
  if _, err := db.Exec(schemaSQL); err ≠ nil {
    db.Close()
    return nil, fmt.Errorf("analytics: schema: %w", err)
  }
  var version string
  err = db.QueryRow(`SELECT value FROM schema_meta WHERE name = 'schema_version'`).Scan(&version)
  switch {
  case err == sql.ErrNoRows:
    if _, err := db.Exec(
      `INSERT INTO schema_meta (name, value) VALUES ('schema_version', ?)`,
      schemaVersion); err ≠ nil {
      db.Close()
      return nil, fmt.Errorf("analytics: stamp schema: %w", err)
    }
  case err ≠ nil:
    db.Close()
    return nil, fmt.Errorf("analytics: read schema version: %w", err)
  case version ≠ schemaVersion:
    db.Close()
    return nil, fmt.Errorf("analytics: schema version %s, code understands %s", version, schemaVersion)
  }
  return &Store{db: db}, nil
}
```

listing 89: open: threads=2 in the dsn, one pooled connection, and a schema version that is refused when newer than the code (source: infrastructure/capstone/internal/analytics/store.go, lines 95-125)

`threads=2` is the honest size for a service sharing a machine with five peers, and the schema version stamp is the migrate discipline from book 13, ch 18 (infrastructure: docker, databases, and testing, migrations, test driven) in miniature: a file carrying anything newer than this code understands is refused, not guessed at. The stats side returns the same `wire.Stats` shape history returns, so the web tier renders both engines identically:

```
// Stats computes the same three answers the mongo pipelines compute:
// top talkers and rooms by volume, the busiest UTC hour bucket, and
// the top words longer than three letters. The shapes are wire.Stat
// and wire.Stats so the web tier renders both engines identically.
func (s *Store) Stats(ctx context.Context) (wire.Stats, error) {
  stats := wire.Stats{}
  if err := s.db.QueryRowContext(ctx, `SELECT count(*) FROM messages`).Scan(&stats.Total); err != nil {
    return wire.Stats{}, err
  }
  var err error
  if stats.Talkers, err = s.topN(ctx, talkersSQL); err != nil {
    return wire.Stats{}, err
  }
  if stats.Rooms, err = s.topN(ctx, roomsSQL); err != nil {
    return wire.Stats{}, err
  }
  hours, err := s.topN(ctx, busiestSQL)
  if err != nil {
    return wire.Stats{}, err
  }
  if len(hours) > 0 {
    stats.Busiest = hours[0]
  }
  if stats.TopWords, err = s.topN(ctx, wordsSQL); err != nil {
    return wire.Stats{}, err
  }
  return stats, nil
}
```

listing 90: stats wiring: count, top talkers and rooms, the busiest hour bucket, top words (source: infrastructure/capstone/internal/analytics/store.go, lines 155-182)

The service half is a mirror of history's consumer with its own name. The durable name lives in this package, so `internal/jet` stays untouched and a new reader adds its own offset instead of editing the shared one:

```
// AckWait spaces redeliveries of a message the store rejected.
const AckWait = config.AckWait

// AnalystDurable is this package's durable, beside jet's notifiers and
// historians. It lives here so internal/jet stays untouched: a new
// reader adds its own name instead of editing the shared one.
const AnalystDurable = "analysts"

// ErrValidation marks rejected input.
var ErrValidation = errors.New("analytics: invalid message")

// Service holds the store, the consumer, and the stats responder.
type Service struct {
  store   *Store
  cons    jetstream.Consumer
  cc      jetstream.ConsumeContext
  stats   *nats.Subscription
  ackWait time.Duration
}
```

listing 91: the analysts durable, its ack wait, and the service holding store, consumer, responder (source: infrastructure/capstone/internal/analytics/service.go, lines 22-40)

```
// New ensures the CHAT stream, creates or updates the analysts
// durable, and starts consuming and answering analytics.stats.
func New(nc *nats.Conn, store *Store, ackWait time.Duration) (*Service, error) {
  js, err := jetstream.New(nc)
  if err ≠ nil {
    return nil, err
  }
  ctx, cancel := context.WithTimeout(context.Background(), config.SetupTimeout)
  defer cancel()
  stream, err := jet.EnsureChatStream(ctx, js)
  if err ≠ nil {
    return nil, err
  }
  cons, err := stream.CreateOrUpdateConsumer(ctx, jetstream.ConsumerConfig{
    Durable:       AnalystDurable,
    FilterSubject: jet.Subjects,
    AckWait:       ackWait,
    MaxDeliver:    config.MaxDeliver,
  })
  if err ≠ nil {
    return nil, err
  }
  s := &Service{store: store, cons: cons, ackWait: ackWait}
  stats, err := bus.ServeJSON(nc, "analytics.stats", s.handleStats)
  if err ≠ nil {
    return nil, err
  }
  cc, err := cons.Consume(s.handle)
  if err ≠ nil {
    stats.Unsubscribe()
    return nil, err
  }
  nc.Flush()
  s.cc = cc
  s.stats = stats
  return s, nil
}
```

listing 92: new: ensure the chat stream, create or update the analysts durable, serve analytics.stats, consume (source: infrastructure/capstone/internal/analytics/service.go, lines 42-78)

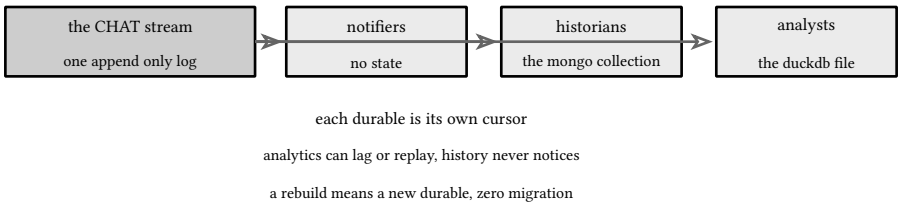


figure 64: three durables fan off one stream, each with its own offset

The handler is the ack discipline from book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared) applied once more:

```
func (s *Service) handle(msg jetstream.Msg) {
  var m wire.Message
  if err := json.Unmarshal(msg.Data(), &m); err ≠ nil {
    // unparseable now means unparseable on every retry
    msg.Ack()
    return
  }
  if err := s.store.Insert(context.Background(), m); err ≠ nil {
    msg.NakWithDelay(s.ackWait)
    return
  }
  msg.Ack()
}

func (s *Service) handleStats([]byte) (any, error) {
  return s.store.Stats(context.Background())
}
```

listing 93: unparseable now means unparseable on every retry, so ack it; a failed insert naks with delay (source: infrastructure/capstone/internal/analytics/service.go, lines 90-106)

The pair of lines at the top is the whole poison message policy: an unparseable body would fail identically on every redelivery, so it is acknowledged and dropped, while a store failure is a `NakWithDelay`, spaced by `AckWait`, so the retry budget burns slowly.

20.5. boot, shared by all six

Every main is the same dozen lines, connect with retries, construct, serve healthz, park on signal, close in reverse, and the shared parts live in one package:

```
func Connect(url string) (*nats.Conn, error) {
  var lastErr error
  for attempt := 0; attempt < 60; attempt++ {
    nc, err := nats.Connect(url,
      nats.Name(os.Args[0]),
      nats.MaxReconnects(-1),
      nats.ReconnectWait(time.Second),
    )
    if err == nil {
      return nc, nil
    }
    lastErr = err
    time.Sleep(500 * time.Millisecond)
  }
  return nil, lastErr
}
```

listing 94: dial with retries: a service starting beside its broker still comes up (source: infrastructure/capstone/internal/boot/boot.go, lines 22-37)

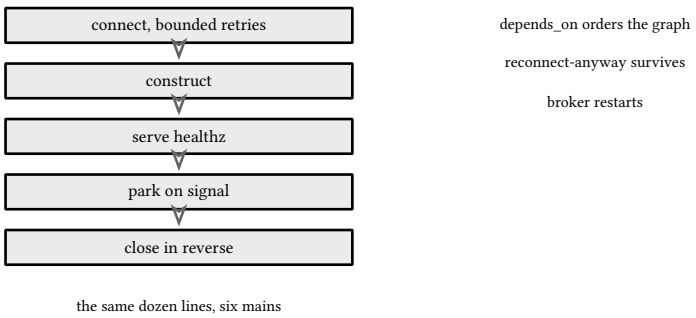


figure 65: the shared main, one timeline every service walks

The retry matters in compose specifically: `depends_on` orders readiness for the declared graph, but reconnect-anyway is what keeps a broker restart from becoming a service restart.

NOTE: where the failure modes went

Each mechanism this capstone leans on failed somewhere first: the nak that burned its budget and the ttl that watches cannot see in book 13, ch 15 (infrastructure: docker, databases, and testing, jetstream: streams, consumers, kv, kafka compared), the commit that waits for readers in book 13, ch 7 (infrastructure: docker, databases, and testing, sqlite: transactions and isolation). The topology does not add new failure modes, it composes known ones, and the suite is arranged so each keeps its own test.

sources: the compose file and the code listings above are the specification. Verified by the full module suite, 61 tests in the duckdb lane, and the docker-gated end to end walk, green 2026-09-20 under `make verify` and `make verify-infra-docker`.

21. capstone part 2: chat and notifications, one click

Part 1 built the services. This chapter is the face on them and the proof they compose: an htmx page that polls fragments, and a gated suite that proves the whole stack works from one compose command.

21.1. htmx, vendored and pinned

The ui is one page plus five fragments, and htmx is the only script, vendored into the binary with its sri digest inline:

```
const HtmxSri = "sha384-BvJpBiO8Kh31EqtJe5DRIeWrHWnCGkwytKs9NKFi86Hhw96dEqdEMzZDeK9iEGTc"
```

listing 95: the digest of the vendored htmx.min.js, the same recipe as the game-systems companion (source: infrastructure/capstone/internal/web/page.go, lines 16-16)

The script tag carries the digest and the file is embedded through `go:embed`, so the running binary cannot serve a script other than the one pinned, and swapping the file breaks the page loudly instead of silently. The page itself is a template function with everything escaped through the html package:

```
func indexHTML(user, room string) string {
  esc := html.EscapeString
  var b strings.Builder
  b.WriteString("<!doctype html>\n<html>\n<head>\n<meta charset=\"utf-8\">\n")
  b.WriteString("<title>chat</title>\n")
  fmt.Fprintf(&b,
    "<script src=\"/static/htmx.min.js\" integrity=\"%s\" crossorigin=\"anonymous\"></script>\n",
    HtmxSri)
  b.WriteString(pageStyle)
  b.WriteString("</head>\n<body>\n")
  fmt.Fprintf(&b, "<h1>room %s as %s</h1>\n", esc(room), esc(user))
  b.WriteString("<main>\n")
  b.WriteString("<section id=\"messages\" hx-get=\"/fragment/messages?room=" +
    urlQuery(room) + "\" hx-trigger=\"every 2s\" hx-swap=\"innerHTML\">\n")
  b.WriteString(renderMessages(nil))
  b.WriteString("</section>\n")
  b.WriteString("<section id=\"side\">\n")
  fmt.Fprintf(&b, "<h2>online</h2>\n<div id=\"roster\" hx-get=\"/fragment/roster?user=%s&room=%s\" "+
    "hx-trigger=\"every 4s\" hx-swap=\"innerHTML\">loading</div>\n",
    urlQuery(user), urlQuery(room))
  fmt.Fprintf(&b, "<h2>notifications</h2>\n<div id=\"notifications\" "+
    "hx-get=\"/fragment/notifications?user=%s\" hx-trigger=\"every 2s\" "+
    "hx-swap=\"innerHTML\">none</div>\n", urlQuery(user))
  b.WriteString("<h2>send</h2>\n")
  fmt.Fprintf(&b, "<form hx-post=\"/send\" hx-target=\"#messages\" hx-swap=\"innerHTML\">\n"+
    "<input type=\"hidden\" name=\"user\" value=\"%s\">\n"+
    "<input type=\"hidden\" name=\"room\" value=\"%s\">\n"+
    "<input type=\"text\" name=\"body\" autocomplete=\"off\">\n"+
    "<button>send</button>\n</form>\n", esc(user), esc(room))
  b.WriteString("<h2>stats</h2>\n")
  fmt.Fprintf(&b, "<div id=\"stats\" hx-get=\"/stats?user=%s&room=%s\" "+
    "hx-trigger=\"every 10s\" hx-swap=\"innerHTML\">loading</div>\n",
    urlQuery(user), urlQuery(room))
  fmt.Fprintf(&b, "<h2>analytics</h2>\n<div id=\"analytics\" "+
    "hx-get=\"/analytics?user=%s&room=%s\" "+
    "hx-trigger=\"every 10s\" hx-swap=\"innerHTML\">loading</div>\n",
    urlQuery(user), urlQuery(room))
  b.WriteString("</section>\n</main>\n</body>\n</html>\n")
  return b.String()
}
```

listing 96: the index: five polling targets, the send form, user and room carried in the query (source: infrastructure/capstone/internal/web/page.go, lines 35-74)

Every fragment endpoint is one NATS request, book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 1: services and topology) showed the handler side. The one piece of server-side state is the notification inbox, a per user ring fed by a lazy subscription:

```
func (i *Inboxes) Drain(user string) []wire.Message {
  if user == "" {
    return nil
  }
  i.mu.Lock()
  defer i.mu.Unlock()
  i.subscribe(user)
  msgs := i.rings[user]
  i.rings[user] = nil
  return msgs
}
```

listing 97: drain: subscribe on first poll, hand over the ring, clear it (source: infrastructure/capstone/internal/web/inbox.go, lines 36-46)

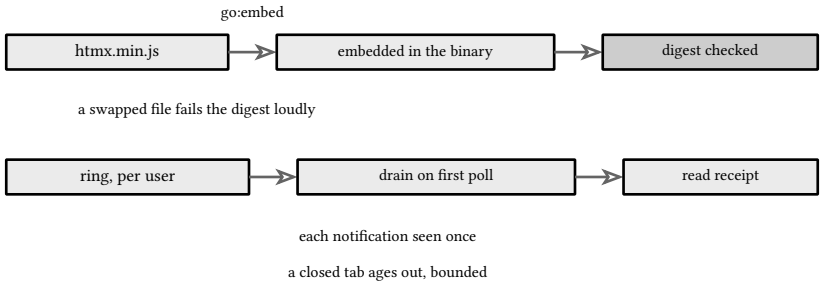


figure 66: the script's chain of custody, and the ring as read receipt

The drain is the read receipt. A polling tab sees each notification once, and a closed tab's notifications age out with the ring bound. The roster fragment doubles as the heartbeat, so a live tab stays online by doing what it was doing anyway, and the browser needs no javascript beyond htmx itself.

The escaping is a test, not a hope: the suite sends a script tag as a message body and asserts it arrives entity-encoded in the fragment, in both the in-process and the docker-gated runs.

The stats and analytics panels are the engine pair made visible. Both are one NATS request per poll, `history.stats` to mongo and `analytics.stats` to duckdb, both every ten seconds, and both render the same `wire.Stats` shape, so the only visible difference between the panels is which engine answered:

```
  b.WriteString("<h2>stats</h2>\n")
  fmt.Fprintf(&b, "<div id=\"stats\" hx-get=\"/stats?user=%s&room=%s\" "+
    "hx-trigger=\"every 10s\" hx-swap=\"innerHTML\">loading</div>\n",
    urlQuery(user), urlQuery(room))
  fmt.Fprintf(&b, "<h2>analytics</h2>\n<div id=\"analytics\" "+
    "hx-get=\"/analytics?user=%s&room=%s\" "+
    "hx-trigger=\"every 10s\" hx-swap=\"innerHTML\">loading</div>\n",
    urlQuery(user), urlQuery(room))
```

listing 98: the two ten second panels: mongo answers stats, duckdb answers analytics (source: infrastructure/capstone/internal/web/page.go, lines 64-71)

`renderAnalytics` is `renderStats` with a duckdb label on the headings, deliberately. If the two panels ever disagree, a bug in one of the engines or in the stream fanout is on the table, and this page is the fastest place to see it.

21.2. the one command

`docker compose up -d --wait` brings up nats, mongo, and the six services, every container behind a healthcheck, the web tier waiting on all five below it. The make target owns the lifecycle:

```
verify-infra-docker: verify-duckdb-race
  @docker info >/dev/null 2>&1 || { \
    echo "docker daemon not reachable." >&2; \
    echo "start Docker Desktop (or the engine), then rerun: make verify-infra-docker" >&2; \
    exit 1; \
  }
  @cd books/infrastructure/capstone && trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    go test -tags docker ./integration/...; rc=$$?; exit $$rc
```

fragment, illustrative, not executed

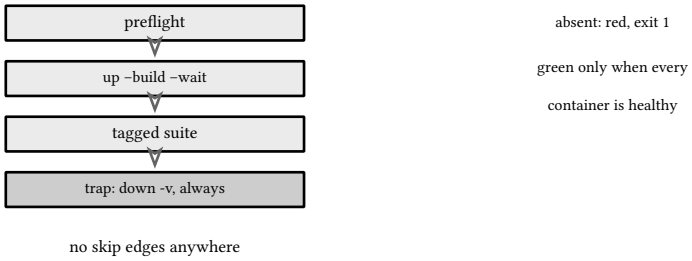


figure 67: the one command as a state machine, no skip edges anywhere

Three properties worth naming. The preflight exits 1 with two plain-English lines when the daemon is absent, a red gate, never a skip. The trap tears down with volumes even on failure, so a red run leaves nothing behind. And the suite runs only after `--wait` returned, which is why its first request can be the web port with no sleep anywhere.

The analytics service is the one deliberate exception inside that graph. Its image is debian rather than the shared alpine, because the duckdb driver is cgo and its default static bindings want the glibc toolchain the golang image already carries, a plain `go build` with no tag. The same file carries the module's race pass: tsan plus cgo has no supported path on windows regardless of compiler, so `make verify-duckdb-race` builds the selftest stage and runs `go test -race` on linux over the exact source snapshot the image ships, while the runtime stage stays compose's default build target. The data lands on the `analyticsdata` volume, the fourth named volume, and the compose seat is otherwise identical to its peers:

```
# the analytics image is the one deliberate exception to the shared
# alpine image: the duckdb driver is cgo, and its default static
# bindings compile with the debian toolchain the golang image already
# carries, plain go build, no tags. the runtime stays debian too,
# because alpine cannot host the glibc-linked duckdb build, and it
# installs wget and ca-certificates so the compose healthcheck keeps
# probing localhost like every other service.
FROM golang:1.27 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=1 go build -o /out/analytics ./cmd/analytics

# the race pass for this module. tsan plus cgo has no supported path
# on windows regardless of compiler: the link misses the synch apiset
# imports tsan needs, patching them in still dies reserving the shadow
# map (error 87), and gcc crt-mixes into heap corruption. so the race
# gate runs here: linux, gcc, the driver's default static bindings,
# still no duckdb_use_lib tag, racing the exact COPY snapshot the
# image ships. the stage sits before the runtime stage so the runtime
# stays compose's default build target; make verify-duckdb-race
# builds it by name
FROM build AS selftest
RUN CGO_ENABLED=1 go test -race ./internal/... ./cmd/...

FROM debian:13-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    wget ca-certificates \
  && rm -rf /var/lib/apt/lists/*
# tini-free pid 1, same as the alpine image: the binary installs its
# own signal handlers, so exec form hands SIGTERM straight to go
COPY --from=build /out/analytics /service
EXPOSE 8085
ENTRYPOINT ["/service"]
```

listing 99: the one debian image: cgo build, the linux race stage, the slim runtime (source: infrastructure/capstone/analytics.Dockerfile, lines 1-35)

The gated walk is the system's whole story in one test: two users join through roster polls, one sends, the message persists to sqlite and appears in the room fragment, the notifier fans it to the other user's inbox, the stats fragment answers from real mongo aggregation, and the analytics fragment answers the same questions from real duckdb sql:

```
// and the duckdb analytics (analytics fragment).
func TestChatNotificationsAndAnalyticsEndToEnd(t *testing.T) {
  suffix := fmt.Sprintf("%04d", rand.Intn(10000))
  body := "docker gate says " + suffix

  waitContains(t, "/fragment/roster?user=alice"+suffix+"&room=general", "alice"+suffix)
  waitContains(t, "/fragment/roster?user=bob"+suffix+"&room=general", "bob"+suffix)

  postForm(t, "/send", url.Values{
    "user": {"alice" + suffix}, "room": {"general"}, "body": {body},
  })
  waitContains(t, "/fragment/messages?room=general", body)
  waitContains(t, "/fragment/notifications?user=bob"+suffix, body)
  waitContains(t, "/stats?user=alice"+suffix+"&room=general", "total messages")
  // the duckdb panel answers the same stream, so both engines agree
  waitContains(t, "/analytics?user=alice"+suffix+"&room=general", "duckdb total messages")
```

listing 100: the walk: presence, send, persistence, notification, analytics (source: infrastructure/capstone/integration/stack_test.go, lines 96-111)

21.3. topic matrix

Every topic maps to a chapter: 0 unmapped. The proof column is where the running evidence lives, a test, a listing, or a gate.

topic	chapter	cost or guarantee	proof
two engines pinned once, cgo confined to the duckdb lanes	ch 1	sqlite lanes stay compiler-free through modernc	duckdb_use_lib tag, analytics.Dockerfile, make duckdb-tools
multi-stage build, one dockerfile five services	ch 2	compiler stays out of the runtime layer	capstone Dockerfile, built by verify-infra-docker
layer caching, manifest before source	ch 2	dependency layer reused across code edits	Dockerfile lines 7-9
cgo_enabled=0 static binary	ch 2	no libc coupling, tiny base possible	Dockerfile line 11, go build
container is a process, pid 1	ch 3	exec form entrypoint or signals never arrive	Dockerfile line 18, boot.Signal
sigterm handling and drain order	ch 3	stop closes in reverse startup order	cmd/*/main.go
compose healthcheck anchors	ch 4	one yaml anchor, seven services	compose.yml lines 12-16
depends_on with service_healthy	ch 4	up returning means working, not merely started	compose.yml web stanza
down -v on failure	ch 4	no dirty volumes between runs	Makefile verify-infra-docker trap
bridge network and embedded dns	ch 5	service name resolves inside the project network	compose.yml NATS_URL values
one published port	ch 5	only the web tier is reachable from the host	compose.yml ports stanza
normalized schema, names stored once	ch 6	renames touch one row	migrations/0001_init.sql

topic	chapter	cost or guarantee	proof
foreign keys enforced per connection	ch 6	pragma per open, not per file	store.go Open
additive second migration, index and view	ch 6	live database absorbs it	migrations/0002_rollup.sql
snapshot at first read	ch 7	repeatable read with no configuration	samples/ch07 TestReaderSeesOnlyCommittedState
shared, reserved, exclusive ladder	ch 7	commit needs exclusive in rollback mode	samples/ch07 TestRollbackCommitNeedsExclusive
busy_timeout as wait not error	ch 7	bounded parking on lock contention	samples/ch07 TestBusyTimeoutWaitsForTheReader
wal reader and writer concurrency	ch 8	one sidecar file, checkpoint duty	samples/ch08 TestWalModeSticksAndPersists
wal_checkpoint truncate	ch 8	sidecar reset on demand	samples/ch08 TestCheckpointTruncateResetsTheSidecar
vacuum into backup	ch 8	consistent snapshot, no driver support needed	samples/ch08 TestVacuumIntoBackup
crash between begin and commit	ch 8	old state or new, never half	samples/ch08 subprocess test
json functions and paths	ch 9	queries into text columns	samples/ch09 TestJsonFunctions
alter column set and drop not null	ch 9	3.53 constraint edits, defaults still need a rebuild	samples/ch09 TestAlterColumnConstraints
reindex by name and collation	ch 9	expression reindex absent in this build	samples/ch09 TestReindex
money as integer cents	ch 9	no float drift, formatting at the edge	samples/ch09 TestMoneyAsIntegerCents
strict tables	ch 9	type gate, numeric text still converts	samples/ch09 TestStrictTables
sqlite as embedded store in three other books	ch 10	single writer discipline	raft store, game saves, lsm comparator
duckdb engine pin, version() exact string	ch 11	driver tag v2.10505.0 encodes 1.5.5	samples/ch11 TestEngineVersion
parquet round trip and hive partitioning	ch 11	the partition column lives in the directory names	samples/ch11 TestParquetPartitionedByHive
csv sniffing with explicit fallback	ch 11	delim, quote, nullstr pin the parse	samples/ch11 TestCsvExplicitOptionsFixTheParse
appender bulk load vs batched exec	ch 11	measured 48x on this machine	samples/ch11 TestAppenderVersusBatchedExec
reader snapshot beside an open writer	ch 11	no busy ladder, one writer at a time still	samples/ch11 TestReaderSnapshotWhileWriterHoldsTransaction
sqlite attach companionability	ch 12	one process, two engines, one file	samples/ch12 TestAttachJoinAcrossEngines, TestWriteThroughAttachLandsInSqlite
engine selection by workload	ch 12	transactional, analytical, shared, ephemeral	ch12 decision figure, capstone four stores

topic	chapter	cost or guarantee	proof
upsert by id, idempotent consumer writes	ch 13	redelivery cannot duplicate	history/mongo.go Insert
aggregation pipeline, group sort limit project	ch 13	grouping happens at the server	history/mongo.go topGroup
pipeline mirrored in go behind a fake	ch 13	docker-free gate proves plumbing	history/fake.go, docker gate proves pipelines
subjects and wildcards	ch 14	token hierarchy, not string matching	samples/ch14 TestWildcardSubjects
request reply over one round trip	ch 14	inbox per request	samples/ch14 TestRequestReply
no responders fails fast	ch 14	missing service is an error, not a hang	samples/ch14 TestNoRespondersFailsFast
queue groups, exactly one member	ch 14	load balancing with no broker config	samples/ch14 TestQueueGroupDeliversExactlyOnce
embedded server in tests	ch 14	same broker binary, no container	testnats/server.go
stream capturing subjects	ch 15	core publish becomes durable log	jet/chat.go
durable consumers as offsets	ch 15	resume where the ack floor stopped	notify tests, history tests
nak with delay, not plain nak	ch 15	plain nak burns max_deliver instantly	notify.go handle, TestNakRedelivers
kv bucket with ttl presence	ch 15	server-side expiry	presence.go New
ttl expiry is invisible to a kv watch	ch 15	reconcile sweep needed	presence.go sweep
kafka compared in prose	ch 15	same shapes, different defaults	chapter text, no second implementation
two gates, docker-free by default	ch 16	verify never needs the daemon	Makefile verify, 131 go tests
whole system in one process	ch 16	real services, embedded broker, fake mongo	web_test.go newStack
build-tagged docker suite	ch 16	docker-gated and never silently skipped	integration/stack_test.go
expected values hand-derived from the requirement	ch 16	the oracle independent of the implementation	analytics store_test.go TestStatsMatchAManualCount
harvested output makes a mirror suite	ch 16	green certifies self-consistency, not the requirement	ch16 oracle section, mutation caveat
direction of fit, requirement to expectation	ch 16	expectations written first, never pasted from output	analytics manualCount, history manual count
fake at the store seam	ch 17	contract test pins both sides	history fake against mongo shape
seeding to force the rare path	ch 17	pre-inserted row replays a crash	TestConsumerSkipsAlreadyStoredIDs
pinned subject names as contract	ch 17	rename fails a test, not production	notify_test RosterSubject
versioned sql migrations, one tx each	ch 18	failure leaves no partial schema	migrate.go Up

topic	chapter	cost or guarantee	proof
gapless version sequence enforced	ch 18	reorder or missing file is an error	TestUpRejectsVersionGaps
changeset as wal entry, files changed vs first parent	ch 19	commit cost scales with the change, never the repo	samples/ch19 store.go Commit, TestCommitRootAndSequentialIDs
content-addressed blobs, immutable and idempotent	ch 19	any cache serves any object with no coordination	samples/ch19 TestPutSameContentSameDigest, TestPutStoresDefensiveCopy
refs as one-row compare-and-swap	ch 19	global state off the hot path, the conflict carries Have	samples/ch19 TestConcurrentCASExactlyOneWinner, TestCASRetryWithReportedHave
derived blame and path history from the log	ch 19	deltas in the log, answers derived recursively from parents	samples/ch19 TestBlameLastWriterPerPath, TestPathHistoryNewestFirst
snapshot as first-parent chain replay	ch 19	the working copy computed from the log, stored nowhere	samples/ch19 TestSnapshotChainsThroughParents
want-based negotiation, resumable object fetches	ch 19	every request one immutable object, any tier answers	ch19 tier ladder, ref/scion/post.md scaling scion
merge queue replaced by post-merge pre-deploy testing	ch 19	no global repository lock at agent merge rates	ch19 lessons section
analytics service over the analysts durable	ch 20	own offset, own engine, no shared writer	analytics/service.go New, cmd/analytics
one request per fragment	ch 21	web holds no database state	web.go handlers
sri-pinned vendored htmx	ch 21	script swap fails the digest	page.go HtmxSri
roster poll doubles as heartbeat	ch 21	no background timer in the browser	web.go roster



figure 68: where the 70 topics landed, none unmapped

21.4. pinned sources

topic	url	accessed
sqlite change log, 3.53.4 released 2026-07-24	https://www.sqlite.org/changes.html	2026-09-09
docker compose file reference, healthchecks and depends_on	https://docs.docker.com/compose/compose-file/	2026-09-09
docker engine network drivers, bridge and embedded dns	https://docs.docker.com/engine/network/	2026-09-09

topic	url	accessed
nats jetstream concepts, streams and consumers	https://docs.nats.io/nats-concepts/jetstream	2026-09-09
modernc.org/sqlite, pure go sqlite driver	https://pkg.go.dev/modernc.org/sqlite	2026-09-09
dockerfile reference, build stages, entrypoint exec form	https://docs.docker.com/reference/dockerfile/	2026-09-10
sqlite write-ahead logging, checkpoints and reader concurrency	https://www.sqlite.org/wal.html	2026-09-10
sqlite file locking, shared, reserved, exclusive	https://www.sqlite.org/lockingv3.html	2026-09-10
sqlite json functions	https://www.sqlite.org/json1.html	2026-09-10
sqlite alter table, including the 3.53 constraint edits	https://www.sqlite.org/lang_altertable.html	2026-09-10
sqlite reindex	https://www.sqlite.org/lang_reindex.html	2026-09-10
sqlite pragma reference, busy_timeout, synchronous, foreign_keys	https://www.sqlite.org/pragma.html	2026-09-10
sqlite vacuum, the into clause for backups	https://www.sqlite.org/lang_vacuum.html	2026-09-10
sqlite strict tables	https://www.sqlite.org/stricttables.html	2026-09-10
sqlite create index, the statement and its semantics	https://www.sqlite.org/lang_createindex.html	2026-09-10
sqlite create view, the statement and its semantics	https://www.sqlite.org/lang_createview.html	2026-09-10
nats subjects and wildcards	https://docs.nats.io/nats-concepts/subjects	2026-09-10
nats jetstream key value stores	https://docs.nats.io/using-nats/developer/develop_jetstream/kv	2026-09-10
nats.go jetstream package api	https://pkg.go.dev/github.com/nats-io/nats.go/jetstream	2026-09-10
mongodb aggregation pipeline	https://www.mongodb.com/docs/manual/aggregation/	2026-09-10
htmx attribute reference, the vendored 4.x script	https://htmx.org/docs/	2026-09-10
duckdb parquet import and export, format options and partitioned writes	https://duckdb.org/docs/stable/data/parquet/overview	2026-09-20
duckdb csv auto-detection, the sniffer and its overrides	https://duckdb.org/docs/stable/data/csv/auto_detection	2026-09-20
duckdb sqlite extension, attach, affinity, sqlite_all_varchar	https://duckdb.org/docs/stable/extensions/sqlite	2026-09-20
duckdb c api, the appender interface the go driver exposes	https://duckdb.org/docs/stable/api/c/appender	2026-09-20
duckdb go client, driver name, dsn parameters	https://pkg.go.dev/github.com/duckdb/duckdb-go/v2	2026-09-20
duckdb 1.5.5 binaries, the sha256 pinned install channel	https://install.duckdb.org	2026-09-20
duckdb-go repository and tags, the 10505 encoding of 1.5.5	https://github.com/duckdb/duckdb-go	2026-09-20
duckdb v2.0 cyanoptera preview artifacts channel, no tagged release	https://artifacts.duckdb.org/v2.0-cyanoptera/	2026-09-20

topic	url	accessed
anthropic engineering, redesigning source control, the scion post	https://www.anthropic.com/engineering/redesigning-source-control, published 2026-10-13, local cache ref/scion/post.md sha256 83543194	2026-10-15
potvin and levenberg, why google stores billions of lines of code in a single repository	https://cacm.acm.org/practice/why-google-stores-billions-of-lines-of-code-in-a-single-repository/, cacm 59(7) july 2016, local cache ref/scion/piper.md	2026-10-15
goode and bolin, sapling source control that scales, the meta post	https://engineering.fb.com/2022/11/15/open-source/sapling-source-control-scalable/, published 2022-11-15, local cache ref/scion/sapling.md	2026-10-15

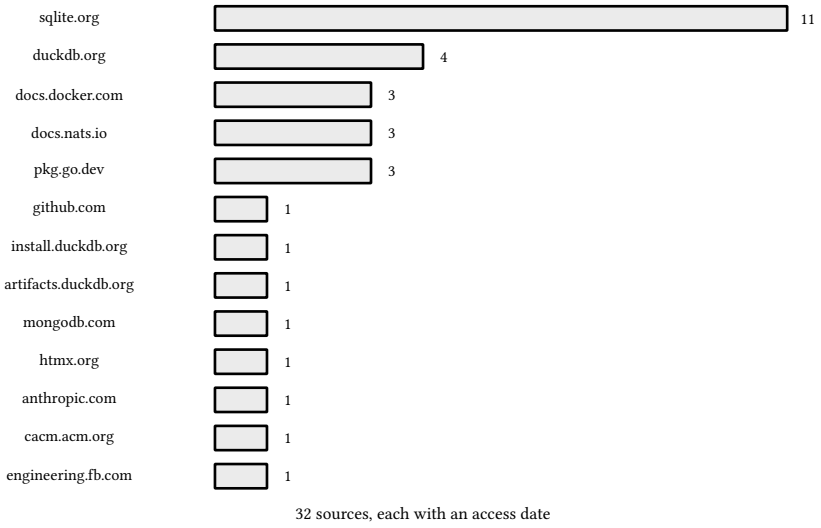


figure 69: the 32 pinned sources by origin, each with its access date

NOTE: the two gates, closed

`make verify` runs this book's 134 docker-free go tests, 61 in the capstone module and 73 in the probe module, the probes splitting into 56 compiler-free tests under plain `go test ./...` and 17 duckdb-tagged tests behind the pinned dll, and it compiles every listing you have read against the files that passed them. `make verify-infra-docker` adds the linux race stage for the analytics module and runs the four tagged tests against the composed system, absent daemon a red with instructions. The docker-free lane was green on 2026-10-16, the day this chapter was re-counted for the vcs chapter's 28 new tests, and the docker lane on 2026-09-20, the only kinds of claims this series makes.